

**CODA-EDA: *COMPACT DEPENDENCY-AWARE ESTIMATION
OF DISTRIBUTION ALGORITHM* UNTUK *INCREMENTAL
POLICY MINING ATTRIBUTE-BASED ACCESS CONTROL*
(ABAC) PADA *EDGE-IOT***

TESIS

**Karya tulis sebagai salah satu syarat
untuk memperoleh gelar Magister dari
Institut Teknologi Bandung**

**Oleh:
ZIADAN QOWI
NIM: 23524019
Program Studi Magister Informatika**



**INSTITUT TEKNOLOGI BANDUNG
September 2026**

ABSTRAK

CODA-EDA: *COMPACT DEPENDENCY-AWARE ESTIMATION OF DISTRIBUTION ALGORITHM* UNTUK INCREMENTAL POLICY MINING ATTRIBUTE-BASED ACCESS CONTROL (ABAC) PADA *EDGE-IOT*

Oleh
Ziadan Qowi
NIM: 23524019
Program Studi Magister Informatika

Kontrol akses berbasis atribut (ABAC) memerlukan penambangan kebijakan (*policy mining*) dari log akses karena penyusunan mahal dan rawan galat. Pada IoT, penambangan idealnya berjalan langsung pada perangkat *edge* bersumber daya terbatas sekaligus diperbarui secara berkala. Algoritma yang ada umumnya boros memori atau mengabaikan dependensi antar atribut. Penelitian ini merancang dan menguji CoDA-EDA (*Compact Dependency-Aware Estimation of Distribution Algorithm*) yang menyatukan pembaruan probabilistik tanpa populasi eksplisit, pemodelan dependensi orde satu melalui pohon Chow-Liu, dan pemisahan pembaruan struktur yang dipicu *drift* dari pembaruan parameter dan memiliki jejak memori $O(D)$. Dengan paradigma *Design Science Research*, CoDA-EDA diuji terhadap enam *base-line* pada Raspberry Pi 5 atas lima dataset ABAC Lab dan validasi antar-generator *Smart Building IoT* menggunakan uji Friedman dan *post-hoc* Wilcoxon berkoreksi Holm-Bonferroni. Hasilnya, kualitas mengungguli keluarga *compact* univariat yang menguat pada dimensi tinggi, setara iBOA/BOA, namun tertinggal moderat dari ACO/UBK. Jejak memorinya datar dengan rasio 1,00 berbanding 6,2-9,2 kali pada algoritma berpopulasi eksplisit. Pemisahan pembaruan berhasil menghemat waktu saat data stabil, serta strategi *warm-start* 2,0-2,6 kali lebih cepat dengan kualitas non-inferior. Kontribusinya adalah pertukaran rancangan yang koheren bagi *gateway edge-IoT*.

Kata kunci: ABAC, CoDA-EDA, dependensi, *edge-IoT*, inkremental, memori, penambangan kebijakan.

ABSTRACT

CODA-EDA: COMPACT DEPENDENCY-AWARE ESTIMATION OF DISTRIBUTION ALGORITHM FOR INCREMENTAL POLICY MINING ATTRIBUTE-BASED ACCESS CONTROL (ABAC) IN EDGE-IOT

By

Ziadan Qowi

NIM: 23524019

Master's Program in Informatics

Attribute-Based Access Control (ABAC) requires policy mining from access logs because manual policy construction is costly and error-prone. In IoT environments, policy mining should ideally be performed directly on resource-constrained edge devices while being periodically updated. Existing algorithms are generally memory-intensive or fail to account for dependencies among attributes. This study designs and evaluates CoDA-EDA (Compact Dependency-Aware Estimation of Distribution Algorithm), which integrates population-free probabilistic updates, first-order dependency modeling through a Chow-Liu tree, and the separation of drift-triggered structure updates from parameter updates, while maintaining an $O(D)$ memory footprint. Following the Design Science Research paradigm, CoDA-EDA is evaluated against six baselines on a Raspberry Pi 5 using five ABAC Lab datasets, together with cross-generator validation on a Smart Building IoT dataset. Statistical significance is assessed using the Friedman test followed by post-hoc Wilcoxon tests with Holm-Bonferroni correction. The results show that CoDA-EDA achieves higher solution quality than the univariate compact family, with the advantage becoming more pronounced at higher dimensionalities. Its performance is comparable to iBOA/BOA, although it remains moderately behind ACO/UBK. Its memory footprint remains effectively constant, with a ratio of 1.00 compared with 6.2-9.2 times higher memory usage for explicit population-based algorithms. The separation of update mechanisms successfully reduces computation time when the data distribution is stable, while the warm-start strategy is 2.0-2.6 times faster while maintaining non-inferior solution quality. The main contribution is a coherent design trade-off for policy mining on edge-IoT gateways.

Keywords: ABAC, CoDA-EDA, dependency, edge-IoT, incremental, memory, policy mining.

**CODA-EDA: *COMPACT DEPENDENCY-AWARE ESTIMATION
OF DISTRIBUTION ALGORITHM* UNTUK *INCREMENTAL
POLICY MINING ATTRIBUTE-BASED ACCESS CONTROL*
(ABAC) PADA *EDGE-IOT***

Oleh
Ziadan Qowi
NIM 23524019
(Program Studi Magister Informatika)

Institut Teknologi Bandung

Menyetujui
Tim Pembimbing
Tanggal 18 September 2026

Ketua

Anggota

Ir. Robithoh Annur, S.T., M.Eng.,
Ph.D.

Dr. Phil. Eng. Hari Purnama, S.Si.,
M.Si

Dipersembahkan kepada orang tua, keluarga, kekasih, dan semua pihak yang senantiasa memberikan doa, dukungan, dan semangat dalam perjalanan pendidikan ini.

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Allah SWT atas rahmat dan karunia-Nya sehingga tesis ini dapat diselesaikan sebagai salah satu syarat untuk menyelesaikan Program Magister Informatika di Institut Teknologi Bandung.

Penulis menyampaikan terima kasih kepada:

1. Bapak Sugimin dan Ibu Umilah, selaku orang tua penulis, atas doa, kasih sayang, dukungan, dan pengorbanan yang senantiasa diberikan.
2. Lembaga Pengelola Dana Pendidikan (LPDP), Kementerian Keuangan Republik Indonesia, atas dukungan pembiayaan selama penulis menempuh studi magister.
3. Ibu Robithoh Annur dan Bapak Hari Purnama, selaku dosen pembimbing, atas bimbingan, arahan, kritik, dan masukan dalam penyempurnaan penelitian ini.
4. Kekasih hati, Fitriani Luluk Ashari, atas dukungan, pengertian, semangat, dan pendampingan selama penyelesaian studi dan tesis ini.
5. Rekan-rekan IOTA Research Group, atas diskusi, bantuan, dan kebersamaan selama proses penelitian.
6. Rekan-rekan Magister Informatika Angkatan 2024 Ganjil, atas dukungan dan kebersamaan selama menjalani perkuliahan dan penyelesaian studi.

Penulis menyadari bahwa tesis ini masih memiliki keterbatasan. Kritik dan saran yang membangun diharapkan dapat menjadi masukan bagi pengembangan penelitian selanjutnya. Semoga tesis ini bermanfaat bagi pengembangan ilmu pengetahuan, khususnya pada bidang keamanan siber, *Attribute-Based Access Control*, dan komputasi *edge-IoT*.

Bandung, 18 September 2026

Penulis

DAFTAR ISI

ABSTRAK.....	i
ABSTRACT	ii
HALAMAN PENGESAHAN	iii
HALAMAN PERUNTUKAN	iv
KATA PENGANTAR	v
DAFTAR ISI.....	vi
DAFTAR LAMPIRAN.....	viii
DAFTAR GAMBAR DAN ILUSTRASI.....	ix
DAFTAR TABEL	x
DAFTAR SINGKATAN DAN LAMBANG	xi
 Bab I Pendahuluan	 1
I.1 Latar Belakang	1
I.2 Rumusan Masalah.....	3
I.3 Tujuan Penelitian.....	3
I.4 Batasan Masalah	4
I.5 Premis dan Hipotesis.....	4
I.6 Metodologi Penelitian	6
I.7 Sistematika Penulisan.....	7
 Bab II Tinjauan Pustaka dan Landasan Teori	 8
II.1 <i>Internet of Things</i> dan Keterbatasan Sumber Daya Perangkat <i>Edge</i>	8
II.2 Kontrol Akses dan <i>Attribute-Based Access Control</i> (ABAC) .	9
II.3 <i>Policy Mining</i> pada ABAC.....	10
II.4 Algoritma Metaheuristik untuk Optimasi Kombinatorial	12
II.5 <i>Estimation of Distribution Algorithm</i>	13
II.6 EDA Ringan/ <i>Compact</i>	14
II.7 Pemodelan Dependensi	16
II.8 Penelitian Terdahulu	18
II.9 Posisi dan Kebaruan Penelitian.....	20
II.10 Kerangka Berpikir	24
 Bab III Metodologi Penelitian.....	 26
III.1 Analisis Masalah	26
III.2 Analisis Kebutuhan	28
III.3 Variabel Penelitian	29
III.4 Rancangan Algoritma CoDA-EDA	31
III.5 Konstruksi Dataset Penelitian	37
III.6 Algoritma Pembandingan	43
III.7 Perangkat dan Lingkungan Eksperimen	46
III.8 Desain Eksperimen.....	47
III.9 Metrik dan Instrumen Evaluasi.....	51

III.10	Teknik Analisis Data	54
III.11	Ketertelusuran Rancangan Penelitian	56
III.12	Studi Pendahuluan	58
Bab IV	Hasil dan Pembahasan	63
IV.1	Kualitas Kebijakan	63
IV.2	Jejak Memori	67
IV.3	Efisiensi Pembaruan Kebijakan	70
IV.4	Validasi Eksternal	71
IV.5	Sintesis, Keterbatasan, dan Jawaban Rumusan Masalah	73
Bab V	Kesimpulan dan Saran	75
V.1	Kesimpulan	75
V.2	Saran	76
DAFTAR PUSTAKA.....		77
LAMPIRAN		81

DAFTAR LAMPIRAN

Lampiran A	Surat Pernyataan Penggunaan AI	82
Lampiran B	Contoh Kebijakan Hasil Mining Lengkap	83
	B.1 CoDA-EDA pada Dataset <i>Workforce</i> ($D = 436$)	83
	B.2 ACO pada Dataset <i>Workforce</i> (Pembanding)	83
	B.3 CoDA-EDA pada Dataset <i>Smart Building IoT</i> ($D = 428$)	84
Lampiran C	Tabel Hasil Eksperimen Lengkap	86
	C.1 Kualitas Kebijakan (H1) – Rerata Per Dataset dan Tingkat Korelasi	86
	C.2 Jejak Memori (H2) – Rasio Penskalaan $\text{alloc}(NP = 400)/\text{alloc}(NP = 25)$ Per Dataset	88
	C.3 Efisiensi Pembaruan (H3 dan H4) – Waktu Per Siklus dan Kualitas	89
	C.4 Validasi Eksternal <i>Smart Building IoT</i> (H1-H4)	91
Lampiran D	Hasil Uji Statistik Lengkap	92
	D.1 Uji H1 (Kualitas Kebijakan) Per Dataset	92
	D.2 Uji H3 dan H4 (Efisiensi Pembaruan) Per Dataset dan Skenario	95
	D.3 Uji Validasi Eksternal <i>Smart Building IoT</i> (H1)	97
Lampiran E	Analisis Karakteristik Dataset (Pra-Eksperimen)	98
Lampiran F	Konfigurasi dan Kalibrasi Algoritma	100
	F.1 Parameter Tiap Algoritma	100
	F.2 Anggaran Evaluasi	100
	F.3 Parameter Siklus Pembaruan (H3 dan H4)	101
	F.4 Ringkasan Uji Monotonisitas (Kalibrasi)	102
Lampiran G	Rincian Konstruksi Dataset	103
	G.1 Parameter Konstruksi	103
	G.2 Struktur Manifest	103
	G.3 Laporan Validasi — Kalibrasi Korelasi	105
	G.4 Laporan Validasi — Lantai Derau, Drift Bersih, dan Duplikasi	105
Lampiran H	Kutipan Kode Kunci	107
	H.1 Fungsi Fitness Selaras-F1	107
	H.2 Pohon Chow-Liu order-1 via Algoritma Prim (Memori $O(D)$)	108
	H.3 Mekanisme Deteksi Drift	109

DAFTAR GAMBAR DAN ILUSTRASI

II.1	Arsitektur Keputusan Akses ABAC	10
II.2	Siklus Kerja Algoritma <i>Compact</i> dan Mekanisme Pembaruan PV ...	15
II.3	Spektrum Pemodelan Dependensi pada EDA dan Posisi CoDA-EDA	17
II.4	Kerangka Berpikir Penelitian	25
III.1	Diagram Alir Arsitektur CoDA-EDA	34
III.2	<i>Pipeline</i> Konstruksi Dataset Penelitian.....	41
III.3	Peta Eksperimen: Pemetaan Hipotesis H1-H4 terhadap Perbandingan, Metrik, dan Uji Statistik	51
IV.1	Jejak Memori CoDA-EDA Linear Terhadap Dimensi Ruang Solusi D	69

DAFTAR TABEL

II.1	Contoh potongan log akses ($\mathcal{L}$)	11
II.2	Ringkasan Posisi Penelitian Terdahulu	19
II.3	Perbedaan CoDA-EDA terhadap Algoritma EDA yang Sudah Ada ..	23
III.1	Analisis Masalah pada Algoritma Pembandingan terhadap Kebutuhan Inti Edge-IoT	27
III.2	Pemetaan Analisis Masalah — Kebutuhan — Keputusan Rancangan	28
III.3	Definisi Operasional Variabel Penelitian	30
III.4	Perbandingan kompleksitas waktu dan memori per siklus/generasi ..	37
III.5	Ilustrasi distribusi bersama dua atribut sebelum injeksi korelasi	38
III.6	Ilustrasi distribusi bersama dua atribut setelah injeksi korelasi	38
III.7	Skema Dataset Penelitian	42
III.8	Algoritma Pembandingan dan Perlakuan Kesetaraan Implementasi	44
III.9	Spesifikasi Perangkat dan Lingkungan Eksperimen	47
III.10	Ringkasan Desain Eksperimen	49
III.11	Definisi Operasional Metrik dan Instrumen Evaluasi.....	53
III.12	Rancangan Uji Statistik per Hipotesis.....	56
III.13	Desain artefak dan pemetaan kebutuhan	57
III.14	Karakteristik Dataset dan Kecocokannya dengan Hipotesis (Analisis Pra-Eksperimen).....	58
III.15	Hasil Studi Pendahuluan: Kualitas Kebijakan (F-score) pada Dua Dataset Utama	61
IV.1	Rerata <i>F-score</i> Ketujuh Algoritma pada Lima Dataset ABAC Lab...	63
IV.2	Rincian Kualitas H1 pada e-document ($D = 1.907$; rerata lintas korelasi; post-hoc vs CoDA-EDA)	64
IV.3	Efek Anggaran Jumlah Aturan terhadap Kualitas CoDA-EDA pada <i>e-document</i>	65
IV.4	Contoh Kebijakan Hasil Mining (cuplikan aturan terbaca).....	67
IV.5	Penskalaan Jejak Memori terhadap Ukuran Populasi NP (rerata lintas lima dataset).....	68
IV.6	Hasil Pengujian H3 dan H4 pada Lima Dataset (rerata; warm/always/cold)	70
IV.7	Validasi Eksternal Smart Building IoT — Kualitas ($D = 428$; garis sepele $F1=0,858$; 5 seed).....	72
IV.8	Sintesis Status Hipotesis H1-H4 Lintas Dataset	73

DAFTAR SINGKATAN DAN LAMBANG

SINGKATAN	Nama	Pemakaian pertama kali pada halaman
ABAC	<i>Attribute-Based Access Control</i>	1
ACL	<i>Access Control List</i>	9
ACO	<i>Ant Colony Optimization</i>	1
ARM	Arsitektur/prosesor ARM	47
BIC	<i>Bayesian Information Criterion</i>	16
BOA	<i>Bayesian Optimization Algorithm</i>	3
CoDA-EDA	<i>Compact Dependency-Aware Estimation of Distribution Algorithm</i>	3
compact-GA	<i>Compact Genetic Algorithm</i>	3
compact-PSO	<i>Compact Particle Swarm Optimization</i>	4
CPU	<i>Central Processing Unit</i>	46
DAC	<i>Discretionary Access Control</i>	1
DSRM	<i>Design Science Research Methodology</i>	6
EDA	<i>Estimation of Distribution Algorithm</i>	5
F1	<i>F-measure (F-score)</i>	51
FN	<i>False Negative</i>	52
FP	<i>False Positive</i>	52
H1	Hipotesis Penelitian 1	5
H2	Hipotesis Penelitian 2	5
H3	Hipotesis Penelitian 3	5
H4	Hipotesis Penelitian 4	5
HAT	<i>Hardware Attached on Top</i>	47
iBOA	<i>Incremental Bayesian Optimization Algorithm</i>	5
IoT	<i>Internet of Things</i>	1
JSD	<i>Jensen-Shannon Divergence</i>	58
LLM	<i>Large Language Model</i>	18
MAC	<i>Mandatory Access Control</i>	9
MGGO	<i>Modified Greylag Goose Optimization</i>	2
NIST	<i>National Institute of Standards and Technology</i>	9
NMI	<i>Normalized Mutual Information</i>	38
OS	<i>Operating System</i>	47
PCIe	<i>Peripheral Component Interconnect Express</i>	47
PMBGA	<i>Probabilistic Model-Building Genetic Algorithm</i>	13
PV	<i>Probability Vector</i>	15
RAM	<i>Random Access Memory</i>	46
RBAC	<i>Role-Based Access Control</i>	1
rcGA	<i>Real-Valued Compact Genetic Algorithm</i>	16
RSS	<i>Resident Set Size</i>	52

SINGKATAN	Nama	Pemakaian pertama kali pada halaman
SDRAM	<i>Synchronous Dynamic Random-Access Memory</i>	47
SoC	<i>System on Chip</i>	47
SPEA2	<i>Strength Pareto Evolutionary Algorithm 2</i>	18
SSD	<i>Solid-State Drive</i>	47
TP	<i>True Positive</i>	52
UBK	<i>Upgraded Black-Winged Kite</i>	1
UcGA	<i>Uniform Compact Genetic Algorithm</i>	16
VmHWM	<i>Virtual Memory High Water Mark</i>	46
WSC	<i>Weighted Structural Complexity</i>	6
LAMBANG		
α	Parameter ACO; simbol ini kemudian juga digunakan sebagai parameter adaptif UBK dan taraf signifikansi statistik	45
β	Parameter/bobot heuristik ACO	45
<i>Coverage</i>	Proporsi permintaan akses yang dicakup oleh setidaknya satu aturan	52
D	Dimensi ruang solusi, yaitu jumlah pasangan atribut-nilai	4
δ	Margin non-inferioritas praktis	36
Δ	Besar pergeseran distribusi atau <i>drift</i> antar-jendela	33
$\Delta\text{Quality}$	Selisih kualitas antara <i>warm-start</i> dan <i>re-mining</i> penuh	53
$f(x)$	Nilai fungsi <i>fitness</i> kandidat aturan x	32
i	Indeks pasangan atribut-nilai atau variabel keputusan	32
k	Orde dependensi yang diizinkan pada model	18
L	Log akses yang menjadi masukan <i>policy mining</i>	10
M	Ukuran populasi pada EDA kanonik	14
MemPeak_alloc	Puncak alokasi struktur data algoritma yang diukur dengan <i>tracemalloc</i>	48
MemPeak_sys	Puncak penggunaan memori pada tingkat sistem operasi	49
n	Banyaknya dimensi/variabel dalam formulasi umum; kemudian juga digunakan untuk jumlah pasangan/blok statistik	36
NP	Ukuran populasi virtual	4
$O(D)$	Kompleksitas memori linear terhadap dimensi D	15

LAMBANG	Nama	Pemakaian pertama kali pada halaman
$O(D^2)$	Kompleksitas perhitungan <i>mutual information</i> seluruh pasangan variabel	28
$O(n^k)$	Kompleksitas jumlah tabel probabilitas marginal iBOA pada orde dependensi k	18
$O(NP \times D)$	Kompleksitas memori algoritma berpopulasi eksplisit terhadap ukuran populasi dan dimensi	15
p	Nilai-p pada pengujian statistik	55
p_{adj}	Nilai-p yang telah dikoreksi/di- <i>adjust</i> dengan prosedur Holm-Bonferroni	61
p_i	Probabilitas bit ke- i bernilai 1 pada PV	32
p_0	Probabilitas awal/inisialisasi PV	32
π	Kebijakan yang dihasilkan	32
<i>Precision</i>	Presisi keputusan kebijakan, $TP/(TP + FP)$	51
r	Aturan yang menjadi anggota kebijakan π ; simbol ini kemudian juga digunakan untuk ukuran efek dan korelasi Pearson	52
<i>Recall</i>	<i>Recall</i> keputusan kebijakan, $TP/(TP + FN)$	51
R^2	Koefisien determinasi pada regresi linear	69
T	Struktur dependensi berupa pohon Chow-Liu	33
T_{init}	Waktu pembelajaran/inisialisasi awal hingga elit pertama terbentuk	52
T_{param}	Waktu siklus yang hanya menjalankan pembaruan parameter	52
T_{reopt}	Waktu re-optimasi setelah <i>drift</i> terdeteksi hingga konvergensi	53
T_{struct}	Waktu satu siklus yang turut melakukan pembelajaran ulang struktur	53
τ	Ambang divergensi Jensen-Shannon untuk memicu pembelajaran ulang struktur	22
x	Vektor biner kandidat aturan ABAC	32
$ \text{attr}(r) $	Banyaknya pasangan atribut-nilai yang menjadi syarat pada aturan r	52
$\emptyset$	Struktur/himpunan kosong pada inisialisasi	35

Bab I Pendahuluan

I.1 Latar Belakang

Perkembangan *Internet of Things* (IoT) mendorong konektivitas perangkat, sensor, dan sistem dalam skala yang sangat besar. Jumlah perangkat IoT diproyeksikan melampaui 60 miliar unit pada 2030 dengan karakteristik yang heterogen dari sisi kemampuan komputasi, energi, dan konteks penggunaan (Ahsan dan Pathan, 2025). Pertumbuhan ini meningkatkan tantangan keamanan, terutama pada autentikasi, integritas data, dan kontrol akses. Keterbatasan sumber daya perangkat juga menyulitkan penerapan mekanisme keamanan konvensional secara langsung (Dritsas dan Trigka, 2025).

Kontrol akses menjadi salah satu komponen penting dalam keamanan IoT karena menentukan entitas yang dapat mengakses sumber daya pada kondisi tertentu. Model konvensional seperti *Role-Based Access Control* (RBAC) dan *Discretionary Access Control* (DAC) kurang sesuai untuk lingkungan IoT yang dinamis, berskala besar, dan kaya konteks (Ragothaman dkk., 2023). Sebaliknya, *Attribute-Based Access Control* (ABAC) memungkinkan keputusan akses berdasarkan kombinasi atribut subjek, objek, dan lingkungan sehingga lebih sesuai dengan heterogenitas dan dinamika IoT (Ahsan dan Pathan, 2025).

Namun, perancangan kebijakan ABAC secara manual kompleks dan mahal sehingga organisasi sering mempertahankan mekanisme kontrol akses lama daripada menanggung biaya migrasi (Díaz-Rodríguez dan Díaz-Pérez, 2025). *Policy mining* kemudian dikembangkan untuk mengekstraksi kebijakan ABAC secara otomatis dari log akses dan data atribut. Pendekatan metaheuristik telah digunakan untuk tujuan ini, antara lain *Ant Colony Optimization* (ACO) untuk migrasi kebijakan lintas sistem heterogen (Siyuan dkk., 2025) dan *Upgrade Black-Winged Kite* (UBK) untuk optimasi kebijakan ABAC pada berbagai data (Li dkk., 2026). Namun, keduanya ditujukan untuk pemrosesan *batch/offline* pada *server* dengan sumber daya memadai, bukan perangkat *edge-IoT* yang terbatas.

Keterbatasan memori, komputasi, dan energi pada perangkat IoT menuntut mekanisme keamanan yang ringan agar dapat dijalankan langsung pada *edge* tanpa bergantung sepenuhnya pada komputasi awan (Canavese dkk., 2024). Pada optimasi metaheuristik untuk *edge computing*, kebutuhan ini mulai dijawab melalui algoritma

ringan dan adaptif, seperti *Modified Greylag Goose Optimization* (MGGO) untuk alokasi layanan IoT yang tidak memerlukan pelatihan atau pemodelan eksplisit yang berat (Khosrowshahi dkk., 2025).

Prinsip tersebut belum banyak diterapkan pada *policy mining* ABAC. Metaheuristik seperti ACO dan UBK menggunakan pendekatan berbasis populasi sehingga tidak dirancang untuk eksekusi ringan dan berulang pada perangkat berdaya rendah. Di sisi lain, pendekatan *policy mining* inkremental yang lebih efisien masih didominasi metode berbasis aturan (Batra dkk., 2021 dan Bamberger dan Fernández, 2024), sehingga tidak memanfaatkan kemampuan eksplorasi kombinatorial yang menjadi keunggulan metaheuristik.

Kesenjangan ini semakin terlihat pada *edge gateway* di lingkungan seperti *smart building* yang harus mengelola akses terhadap ratusan sensor, aktuator, dan data untuk berbagai pengguna dan perangkat. Perancangan kebijakan secara manual bersifat mahal, lambat, subjektif, dan rentan kesalahan sehingga organisasi cenderung mempertahankan RBAC/DAC daripada melakukan migrasi (Díaz-Rodríguez dan Díaz-Pérez, 2025). Padahal, kelemahan kontrol akses ini dapat menimbulkan dampak yang signifikan. Pada studi IBM-Ponemon tahun 2024 yang dikutip oleh Díaz-Rodríguez dan Díaz-Pérez (2025), memperkirakan rata-rata biaya satu pelanggaran data dapat mencapai USD 4,88 juta.

Policy mining dapat mengotomatisasi pembentukan kebijakan, tetapi masih menghadapi dua keterbatasan utama pada *edge-IoT*. Pertama, metode berbasis *association rule* dan sejenisnya mengalami penurunan kinerja pada log berukuran besar serta peningkatan jumlah aturan ketika jumlah atribut bertambah (Xu dan Stoller, 2015 dan Cotrini Jimenez dkk., 2018). Karena itu, metode mutakhir umumnya masih dirancang untuk *server* dengan sumber daya yang memadai. Kedua, *policy mining* umumnya dilakukan secara *batch*, sedangkan lingkungan IoT terus berubah akibat munculnya perangkat, peran, dan pola akses baru sehingga kebijakan perlu diperbarui secara berkala. Pemrosesan di *server* juga mengharuskan pengiriman log keluar perangkat, yang menambah beban jaringan, latensi, dan risiko privasi, sementara algoritma berbasis populasi dengan jejak memori besar sulit dijalankan pada *edge gateway* terbatas. Atas dasar tersebut, penelitian ini merancang algoritma *policy mining* ABAC yang *compact*, mempertimbangkan dependensi antar-atribut, dan mampu memperbarui kebijakan secara inkremental sehingga dapat dijalankan langsung pada perangkat *edge-IoT* tanpa mengorbankan kualitas kebijakan.

I.2 Rumusan Masalah

Berdasarkan kesenjangan penelitian yang teridentifikasi, rumusan masalah yang ada pada penelitian ini disusun sebagai berikut:

1. Mekanisme pemodelan probabilistik dan pembelajaran struktur dependensi seperti apa yang memungkinkan korelasi antar-atribut ABAC ditangkap tanpa menyimpan populasi solusi eksplisit, sehingga jejak memorinya tidak tumbuh terhadap ukuran populasi?
2. Bagaimana kualitas kebijakan ABAC yang dihasilkan algoritma usulan dibandingkan terhadap algoritma berpopulasi (BOA, ACO, UBK) dan *compact* murni (*compact-GA/PSO*) pada berbagai tingkat korelasi atribut, dan *trade-off* apa yang muncul antara kualitas kebijakan dan kehematan sumber daya?
3. Sejauh mana pemisahan frekuensi pembaruan struktur dependensi dari pembaruan parameter, termasuk strategi *warm-start* setelah *drift*, memengaruhi waktu pembaruan dan kualitas kebijakan pada berbagai tingkat *drift* data akses, dibandingkan pembaruan struktur setiap siklus dan penambangan ulang dari awal?
4. Bagaimana jejak memori algoritma usulan berperilaku terhadap ukuran populasi virtual (NP) dan dimensi ruang solusi (D) ketika dijalankan pada perangkat *edge* target, dan bagaimana kelayakan sumber dayanya dibandingkan algoritma berpopulasi eksplisit (BOA, ACO, UBK)?

Ketersediaan dataset yang memenuhi karakteristik pada rumusan masalah di atas dibahas dan dikonstruksi pada metodologi penelitian.

I.3 Tujuan Penelitian

Secara umum, penelitian ini bertujuan untuk merancang dan mengevaluasi sebuah algoritma metaheuristik yang ringan, sadar terhadap korelasi antar-atribut, dan mampu diperbarui secara inkremental yang diberi nama CoDA-EDA (*Compact Dependency-Aware Estimation of Distribution Algorithm*), untuk mendukung *policy mining Attribute-Based Access Control* (ABAC) pada perangkat *edge*-IoT berdaya rendah. Secara khusus, tujuan penelitian ini adalah untuk:

1. Menghasilkan mekanisme CoDA-EDA yang memodelkan dependensi antar-atribut ABAC tanpa menyimpan populasi solusi eksplisit.
2. Memperoleh bukti mengenai kualitas kebijakan CoDA-EDA beserta *trade-off*-

nya terhadap algoritma berpopulasi (BOA, ACO, UBK) dan *compact* pada berbagai tingkat korelasi atribut.

3. Memperoleh bukti efektivitas mekanisme pembaruan kebijakan berkala terhadap waktu pembaruan dan kualitas kebijakan pada berbagai tingkat *drift*.
4. Memperoleh karakterisasi jejak memori CoDA-EDA terhadap ukuran populasi virtual (NP) dan dimensi ruang solusi (D), serta kelayakan sumber dayanya pada perangkat *edge* target dibanding algoritma berpopulasi eksplisit.

I.4 Batasan Masalah

Batasan berikut menetapkan ruang lingkup persoalan yang dikaji pada penelitian ini, antara lain:

1. Persoalan dibatasi pada penemuan aturan ABAC berbentuk konjungsi dari kondisi atribut subjek, objek, dan lingkungan yang memetakan permintaan akses ke keputusan izin/tolak. Bentuk kebijakan lain seperti kebijakan berprioritas/berbobot tidak termasuk persoalan yang dikaji.
2. Persoalan pemodelan ketergantungan antar-atribut yang ditelaah dibatasi pada keterkaitan berpasangan (orde satu). Ketergantungan orde tinggi berada di luar persoalan yang dijawab oleh penelitian ini.
3. Kualitas kebijakan dimaknai sebagai fidelitas rekonstruksi, yaitu seberapa tepat himpunan aturan hasil penambangan memetakan kembali keputusan izin/tolak pada log yang menjadi sumbernya. Kemampuan generalisasi ke permintaan akses yang belum pernah muncul maupun kebenaran semantik kebijakan terhadap maksud organisasi berada di luar lingkup persoalan.

I.5 Premis dan Hipotesis

Hipotesis penelitian ini disusun secara deduktif dari sejumlah premis yang berakar pada Latar Belakang Masalah dan Rumusan Masalah. Premis-premis tersebut adalah sebagai berikut:

1. **Premis 1:** Perangkat *edge-IoT* memiliki keterbatasan memori, daya komputasi, dan energi, sehingga mekanisme keamanan perlu dirancang ringan agar dapat dijalankan langsung pada perangkat tersebut (Canavese dkk., 2024).
2. **Premis 2:** Algoritma *compact* seperti *compact-GA* dan *compact-PSO* menghemat memori dengan mengasumsikan independensi antar-atribut, sedangkan

BOA dan iBOA dapat memodelkan dependensi tetapi tidak dirancang dengan batasan memori yang ketat. Dalam EDA, pemodelan dependensi antar-variabel diketahui dapat meningkatkan kualitas solusi pada masalah dengan korelasi antar-variabel yang tinggi dibandingkan model dengan asumsi independensi penuh.

3. **Premis 3:** Lingkungan IoT bersifat dinamis, berskala besar, dan memiliki konteks yang terus berubah (Ragothaman dkk., 2023), sehingga kebijakan ABAC perlu diperbarui secara berkala. Namun, pembelajaran ulang struktur dependensi pada setiap perubahan data seperti pada pendekatan BOA/iBOA memerlukan biaya komputasi yang tinggi sehingga kurang efisien untuk perangkat berdaya rendah.
4. **Premis 4:** Kualitas hasil *policy mining* dipengaruhi oleh kemampuan model probabilistik merepresentasikan dependensi dalam data akses. Oleh karena itu, algoritma *compact* yang tetap memodelkan dependensi antar-atribut diperkirakan dapat menghasilkan kualitas kebijakan yang lebih mendekati algoritma *population-based* seperti ACO dan UBK dibandingkan algoritma *compact* yang mengasumsikan independensi penuh.

Berdasarkan keempat premis tersebut, hipotesis penelitian yang diajukan adalah sebagai berikut:

1. **H1** – CoDA-EDA diharapkan dapat menghasilkan kualitas kebijakan (*F-score*) yang kompetitif dibandingkan dengan keenam algoritma pembanding lainnya (ACO, UBK, BOA, iBOA, *compact-GA*, *compact-PSO*).
2. **H2** – Jejak memori CoDA-EDA tetap datar terhadap ukuran populasi virtual (NP), sehingga diperkirakan secara signifikan lebih rendah dibanding algoritma berpopulasi pada NP besar.
3. **H3** – Pemisahan pembaruan struktur dari pembaruan parameter diharapkan dapat mengurangi total waktu komputasi dengan kualitas kebijakan yang non-inferior.
4. **H4** – Pada skenario *drift* rendah hingga sedang, CoDA-EDA dengan *warm-start* diharapkan dapat mencapai kualitas kebijakan non-inferior terhadap *re-mining* penuh dalam waktu re-optimasi yang jauh lebih singkat.

I.6 Metodologi Penelitian

Penelitian ini menggunakan pendekatan *Design Science Research Methodology* (DSRM), yaitu paradigma yang berorientasi pada perancangan dan evaluasi artefak untuk menyelesaikan permasalahan yang teridentifikasi, alih-alih sekadar menjelaskan atau memprediksi fenomena yang telah berlangsung (Peppers dkk., 2007). Titik masuk yang diikuti adalah *problem-centered initiation*, karena penelitian ini bermula dari kesenjangan yang teridentifikasi pada tinjauan pustaka. DSRM menjadi payung paradigma, sedangkan aktivitas *Demonstration* dan *Evaluation* dioperasionalkan melalui desain eksperimen kuantitatif terkontrol. Jadi artefak akan dibandingkan dengan enam algoritma *baseline* pada dataset yang dikonstruksi, diukur dengan metrik yang ditetapkan, dan diuji signifikansinya secara statistik. Proses penelitian mengikuti enam aktivitas DSRM (Peppers dkk., 2007) berikut:

1. *Problem identification and motivation* – Mengidentifikasi kesenjangan penelitian, yaitu belum tersedianya algoritma metaheuristik untuk *policy mining* ABAC yang sekaligus ringan, sadar dependensi, dan dapat diperbarui inkremental pada perangkat *edge-IoT* beserta urgensinya.
2. *Define objectives of a solution* – Menetapkan empat tujuan khusus beserta kriteria keberhasilan terukur yang dirumuskan sebagai Hipotesis H1-H4.
3. *Design and development* – Merancang dan mengimplementasikan artefak penelitian. Aktivitas ini paling ekstensif dan mencakup tiga komponen: (a) perancangan algoritma CoDA-EDA itu sendiri – representasi model probabilistik, mekanisme pembaruan kompetitif tanpa populasi eksplisit, dan strategi pembaruan struktur dependensi berbasis delta; (b) implementasi enam algoritma pembanding secara setara agar perbandingan pada tahap *Evaluation* tidak bias oleh perbedaan kualitas implementasi; dan (c) konstruksi dataset ABAC-Lab dengan tingkat korelasi atribut dan *drift* terkontrol, mengingat dataset semacam ini belum tersedia sebagai sumber daya siap pakai.
4. *Evaluation* – Mengukur dan membandingkan hasil demonstrasi terhadap metrik (kualitas kebijakan/*F-score* dan WSC, jejak memori, serta waktu komputasi), disertai uji signifikansi statistik perbandingan antar-algoritma terhadap H1-H4.
5. *Communication* – Mengomunikasikan permasalahan, artefak CoDA-EDA, dan hasil evaluasinya melalui buku tesis serta naskah publikasi ilmiah terkait.

Meskipun keenam aktivitas disajikan berurutan, proses DSRM bersifat iteratif, se-

hingga penulis dimungkinkan kembali ke aktivitas *Design and Development* apabila hasil *Demonstration* dan *Evaluation* menunjukkan kebutuhan penyempurnaan rancangan (Peffers dkk., 2007).

I.7 Sistematika Penulisan

Penulisan tesis ini disusun dalam lima bab sebagai berikut.

Bab I Pendahuluan membahas latar belakang, rumusan masalah, tujuan penelitian, hipotesis, batasan masalah, metodologi penelitian secara ringkas, dan sistematika penulisan.

Bab II Tinjauan Pustaka dan Landasan Teori membahas landasan teori mengenai *Internet of Things* dan keterbatasan sumber daya perangkat *edge*, kontrol akses berbasis atribut (ABAC) beserta penambangan kebijakannya, algoritma metaheuristik dan *Estimation of Distribution Algorithm* (EDA) beserta variannya yang ringan dan sadar dependensi, posisi penelitian terhadap literatur terdahulu, argumentasi kebaruan, serta kerangka berpikir penelitian.

Bab III Metodologi Penelitian menjelaskan pendekatan *Design Science Research* yang digunakan, tahapan penelitian, variabel penelitian, rancangan algoritma CoDA-EDA yang diusulkan, konstruksi dataset, algoritma pembandingan, perangkat dan lingkungan eksperimen, desain eksperimen, metrik dan instrumen evaluasi, teknik analisis data, serta studi pendahuluan yang mencakup analisis karakteristik dataset dan kalibrasi rancangan.

Bab IV Hasil dan Pembahasan menyajikan hasil pengujian empiris keempat hipotesis pada kelima dataset ABAC Lab, validasi eksternal pada dataset [Smart Building IoT], contoh kebijakan hasil *mining*, serta sintesis temuan, keterbatasan, dan jawaban atas rumusan masalah.

Bab V Kesimpulan dan Saran memuat kesimpulan penelitian yang menjawab tujuan dan rumusan masalah beserta kontribusi yang dicapai, serta saran penerapan praktis dan pengembangan lanjutan.

Bab II Tinjauan Pustaka dan Landasan Teori

II.1 *Internet of Things* dan Keterbatasan Sumber Daya Perangkat *Edge*

Internet of Things (IoT) merupakan jaringan objek fisik, seperti sensor, aktuator, dan perangkat pintar, yang saling terhubung dan mampu bertukar data melalui internet tanpa interaksi manusia secara langsung. Jumlah perangkat IoT diproyeksikan melampaui 60 miliar unit pada 2030 dengan karakteristik yang heterogen dari sisi kemampuan komputasi, energi, dan konteks penggunaan (Ahsan dan Pathan, 2025).

Untuk mengelola kompleksitas tersebut, arsitektur aplikasi IoT umumnya diklasifikasikan menjadi *cloud*, *edge*, dan *fog computing* (Dauda dkk., 2024). *Cloud* menyediakan pemrosesan dan penyimpanan terpusat untuk skala besar, tetapi dapat menghadapi latensi dan keterbatasan *bandwidth*. *Edge computing* memindahkan komputasi lebih dekat ke sumber data untuk mendukung pemrosesan *real-time* dan mengurangi beban jaringan, sedangkan *fog computing* mengombinasikan karakteristik *cloud* dan *edge* untuk lingkungan IoT yang lebih kompleks.

Pemindahan komputasi ke *edge* tidak menghilangkan keterbatasan sumber daya, tetapi memindahkannya ke perangkat tempat komputasi dijalankan. Perangkat IoT pada *edge* umumnya memiliki keterbatasan memori, daya komputasi, dan energi, sehingga mekanisme yang dijalankan perlu dirancang ringan agar tidak bergantung sepenuhnya pada komputasi awan (Canavese dkk., 2024).

Kebutuhan akan algoritma ringan dan adaptif telah mulai direspons dalam optimasi metaheuristik untuk *edge computing*. Salah satu contohnya adalah *Modified Greylag Goose Optimization* (MGGO) yang dirancang untuk alokasi layanan IoT pada *edge* dengan mekanisme adaptif tanpa pelatihan atau pemodelan eksplisit yang berat (Khosrowshahi dkk., 2025). Hal ini menunjukkan bahwa keterbatasan sumber daya *edge* dapat ditangani melalui desain algoritma yang sesuai, bukan hanya melalui peningkatan kapasitas perangkat keras.

Keterbatasan memori, komputasi, dan energi tersebut menjadi premis dasar penelitian ini dalam merancang algoritma metaheuristik yang ringan untuk tugas *policy mining* ABAC pada perangkat *edge-IoT*.

II.2 Kontrol Akses dan *Attribute-Based Access Control* (ABAC)

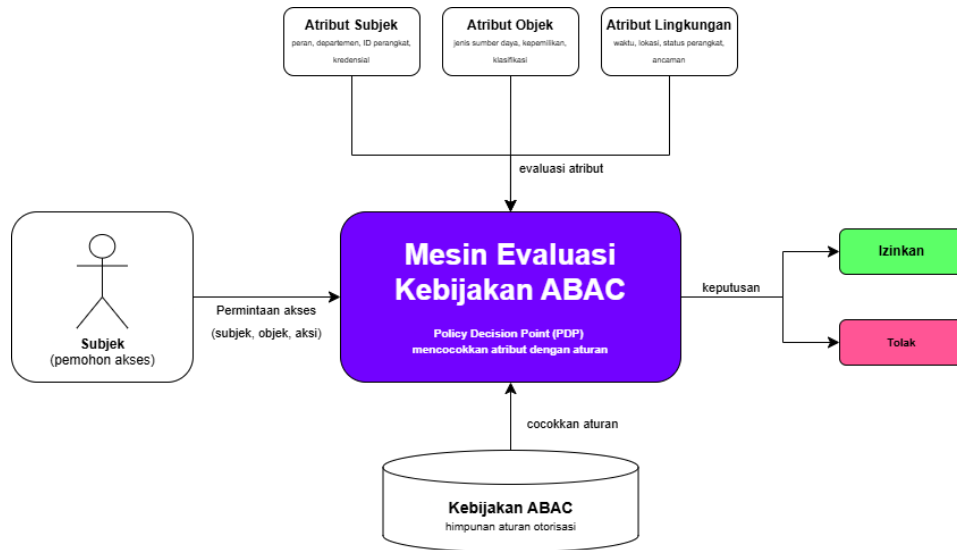
Kontrol akses merupakan mekanisme keamanan yang menentukan entitas yang berhak melakukan operasi tertentu terhadap suatu sumber daya beserta kondisi yang harus dipenuhi. Model kontrol akses klasik meliputi *Discretionary Access Control* (DAC), *Mandatory Access Control* (MAC), dan *Role-Based Access Control* (RBAC). Pada DAC, pemilik sumber daya mengatur hak akses pengguna, umumnya melalui *Access Control List* (ACL) yang perlu diperbarui ketika terjadi perubahan. Sifatnya yang relatif statis membuat DAC kurang sesuai untuk lingkungan IoT yang dinamis (Ragothaman dkk., 2023 dan Ahsan dan Pathan, 2025).

Berbeda dengan DAC, MAC menentukan akses melalui kebijakan terpusat berbasis label keamanan, seperti klasifikasi objek dan tingkat kewenangan subjek. Pendekatan ini memberikan kontrol yang ketat dan konsisten, tetapi pengelolaan label yang terpusat dan kaku menyulitkan penerapannya pada lingkungan IoT yang heterogen, berskala besar, dan terus berubah (Ragothaman dkk., 2023).

RBAC mengaitkan hak akses dengan peran pengguna sehingga pengelolaannya lebih sederhana dibandingkan pemberian izin per individu. Namun, pada lingkungan IoT yang sangat heterogen, kebutuhan untuk merepresentasikan berbagai kombinasi konteks dapat memicu *role explosion*. Selain itu, peran kurang fleksibel untuk merepresentasikan kondisi kontekstual seperti waktu, lokasi, dan status perangkat (Ahsan dan Pathan, 2025).

Sebagai alternatif, *Attribute-Based Access Control* (ABAC) menentukan otorisasi dengan mengevaluasi atribut subjek, objek, operasi, dan lingkungan terhadap kebijakan yang berlaku. Menurut NIST SP 800-162, pendekatan ini memungkinkan kombinasi input keputusan yang lebih luas serta mampu mengakomodasi konsep DAC dan MAC dalam suatu model berbasis atribut (Hu dkk., 2014).

Secara operasional, ABAC terdiri atas tiga komponen utama, yaitu atribut, aturan (*rule*), dan kebijakan *policy*. Atribut merepresentasikan karakteristik subjek, objek, dan lingkungan; aturan menentukan kondisi pemberian akses berdasarkan atribut tersebut; sedangkan kebijakan merupakan himpunan aturan otorisasi. Ketika permintaan akses diterima, atribut yang relevan dievaluasi terhadap kebijakan untuk menghasilkan keputusan izin atau penolakan akses (Díaz-Rodríguez dan Díaz-Pérez, 2025). Alur pengambilan keputusan tersebut diilustrasikan pada Gambar II.1.



Gambar II.1 Arsitektur Keputusan Akses ABAC

Gambar II.1 menunjukkan bahwa keputusan ABAC tidak bergantung langsung pada identitas subjek, tetapi pada kombinasi atribut subjek, objek, dan lingkungan yang dievaluasi terhadap aturan dalam kebijakan. Struktur ini memungkinkan ABAC merepresentasikan konteks akses yang kompleks, sekaligus menunjukkan bahwa kualitas keputusan akses bergantung pada kualitas aturan yang membentuk kebijakan.

Fleksibilitas tersebut menjadikan ABAC sesuai untuk lingkungan IoT yang heterogen dan dinamis karena keputusan akses dapat mempertimbangkan atribut perangkat, pengguna, dan lingkungan tanpa mendefinisikan peran untuk setiap kemungkinan konteks (Ahsan dan Pathan, 2025 dan Ragothaman dkk., 2023). Namun, fleksibilitas ini juga meningkatkan kompleksitas spesifikasi kebijakan, terutama ketika kebijakan dirancang secara manual (Díaz-Rodríguez dan Díaz-Pérez, 2025).

II.3 *Policy Mining* pada ABAC

Spesifikasi kebijakan ABAC secara manual merupakan proses yang kompleks, mahal, dan sulit diskalakan pada sistem besar. *Policy mining* dikembangkan untuk mengotomatisasi proses tersebut dengan mengekstraksi aturan akses dari pola atribut-nilai pada log akses, yaitu catatan permintaan akses yang diizinkan atau ditolak oleh sistem (Perez-Haro dan Diaz-Perez, 2024). Secara formal, jika diberikan log akses L , tujuan *policy mining* ABAC adalah menghasilkan kebijakan yang mampu merepresentasikan pola perizinan dalam L secara akurat (Díaz-Rodríguez dan Díaz-Pérez, 2025).

Sebagai ilustrasi, digunakan potongan log akses L pada sistem rekam medis rumah sakit. Setiap baris merepresentasikan satu permintaan akses yang memuat atribut subjek dan objek, aksi yang diminta, serta keputusan akses yang tercatat.

Tabel II.1 Contoh potongan log akses ($\mathcal{L}$)

#	peran (subjek)	unit (subjek)	jenis objek	unit (objek)	aksi	keputusan
1	perawat	ICU	rekam_me- dis	ICU	baca	permit
2	dokter	ICU	rekam_me- dis	ICU	baca	permit
3	perawat	UGD	rekam_me- dis	UGD	baca	permit
4	perawat	ICU	rekam_me- dis	UGD	baca	deny
5	staf_admin	ICU	rekam_me- dis	ICU	baca	deny
6	dokter	ICU	resep_obat	ICU	tulis	permit
7	perawat	ICU	resep_obat	ICU	tulis	deny

Dari pola perizinan pada L , *policy mining* berupaya menemukan kebijakan berupa himpunan aturan ringkas yang memetakan setiap permintaan ke keputusan yang sama seperti tercatat. Untuk potongan di atas, kebijakan berikut sudah memadai:

R1. (baca rekam medis oleh klinisi pada unit yang sama). JIKA peran $\in \{\text{perawat, dokter}\}$ DAN jenis objek = *rekam_medis* DAN aksi = *baca* DAN unit(subjek) = unit(objek) $\rightarrow$ izinkan.

R2. (penulisan resep oleh dokter). JIKA peran = *dokter* DAN jenis objek = *resep_obat* DAN aksi = *tulis* DAN unit(subjek) = unit(objek) $\rightarrow$ izinkan.

Permintaan yang tidak dicakup aturan mana pun ditolak secara baku (*default-deny*).

Kebijakan R1 dan R2 mengizinkan seluruh permintaan berlabel *permit* dan menolak seluruh permintaan berlabel *deny*, sehingga merekonstruksi log akses L secara sempurna. Akurasi rekonstruksi ini dapat diukur menggunakan *F1-score*, yaitu rata-rata harmonik antara presisi dan *recall* terhadap log akses asli (Díaz-Rodríguez dan Díaz-Pérez, 2025). Selain akurasi, kualitas kebijakan juga dinilai melalui *Weighted*

Structural Complexity (WSC), yaitu kompleksitas yang dihitung berdasarkan jumlah pasangan atribut-nilai yang membentuk seluruh aturan. WSC yang lebih rendah menunjukkan kebijakan yang lebih ringkas dan lebih mudah dikelola. Metrik ini awalnya diperkenalkan untuk *role mining* pada RBAC oleh Molloy dan Reed (1995), kemudian diadaptasi untuk ABAC oleh Xu dan Stoller (2015) (Perez-Haro dan Diaz-Perez, 2024).

Berbagai pendekatan telah dikembangkan untuk melakukan *policy mining* ABAC. Salah satunya adalah *association rule mining* yang membentuk aturan secara progresif dari *log permission*, tetapi mengalami penurunan kinerja pada log berukuran besar (Xu dan Stoller, 2015 dan Díaz-Rodríguez dan Díaz-Pérez, 2025). Pendekatan lain mencakup pembelajaran mesin berbasis PRISM (Iyer dan Masoumzadeh, 2018), kombinasi *data mining* dan statistik (Abu Jabal dkk., 2020), serta pengelompokan permintaan akses sebelum pembentukan aturan. Mayoritas pendekatan tersebut mengandalkan frekuensi kemunculan pola dengan ambang dukungan minimum. Ambang yang terlalu tinggi dapat menyebabkan sumber daya dengan frekuensi akses rendah tidak tercakup aturan, sedangkan ambang yang terlalu rendah dapat memicu *rule explosion* (Perez-Haro dan Diaz-Perez, 2024). Sebagai alternatif, Perez-Haro dan Diaz-Perez (2024) memodelkan log akses sebagai jaringan afiliasi dan menggunakan analisis *biclique* untuk mengekstraksi aturan tanpa ambang frekuensi.

Selain pendekatan tersebut, *policy mining* juga dapat dipandang sebagai masalah pencarian kombinatorial. Pendekatan metaheuristik berbasis populasi seperti ACO (Siyuan dkk., 2025) dan UBK (Li dkk., 2026) melakukan ekstraksi kebijakan melalui proses pencarian solusi, sehingga berbeda dari pendekatan berbasis *association rule*, pembelajaran mesin, klaster, maupun graf.

Dalam penelitian ini, kualitas kebijakan dievaluasi melalui dua dimensi, yaitu akurasi menggunakan *F1-score* dan kompleksitas menggunakan WSC. Kedua metrik tersebut digunakan pada Hipotesis H1 untuk membandingkan CoDA-EDA dengan *baseline compact-GA* dan *compact-PSO*, sedangkan ACO dan UBK digunakan sebagai *baseline* metaheuristik berbasis populasi.

II.4 Algoritma Metaheuristik untuk Optimasi Kombinatorial

Optimasi kombinatorial merupakan proses mencari solusi terbaik dari ruang solusi diskret yang sangat besar dan umumnya bersifat *NP-hard*, sehingga penyelesaian eksak menjadi tidak praktis untuk instansi berskala besar. Metaheuristik menawarkan pendekatan pencarian tingkat tinggi untuk memperoleh solusi mendekati optimal

dengan biaya komputasi yang lebih rendah, meskipun tanpa jaminan optimalitas formal. Perkembangan bidang ini sangat pesat; Rajwar dkk. (2023) mencatat lebih dari 500 algoritma metaheuristik telah dikembangkan, dengan lebih dari 350 di antaranya muncul dalam satu dekade terakhir.

Berdasarkan sumber inspirasinya, metaheuristik dapat dikelompokkan menjadi algoritma evolusioner, seperti *Genetic Algorithm* dan *Differential Evolution*; *swarm intelligence*, seperti *Particle Swarm Optimization* dan *Ant Colony Optimization*; algoritma berbasis fenomena fisika atau kimia, seperti *Simulated Annealing* dan *Gravitational Search Algorithm*; serta kategori lain yang terinspirasi perilaku manusia, permainan, atau konsep matematis (Rajwar dkk., 2023). Namun, klasifikasi metaheuristik belum sepenuhnya konsisten antar-penulis dan perlu terus diperbarui seiring munculnya metode baru (Jakšić dkk., 2023).

Dari sisi mekanisme pencarian, metaheuristik juga dapat dibedakan menjadi *trajectory-based* dan *population-based*. Algoritma *trajectory-based*, seperti *Simulated Annealing* dan *Tabu Search*, menggunakan satu agen yang bergerak melalui ruang solusi, sedangkan algoritma *population-based*, seperti *Genetic Algorithm*, *Particle Swarm Optimization*, dan *Ant Colony Optimization*, menggunakan banyak agen secara simultan untuk memperluas eksplorasi ruang solusi (Rajwar dkk., 2023).

ACO dan UBK yang digunakan sebagai *baseline* dalam penelitian ini termasuk metaheuristik *population-based*. ACO meniru perilaku koloni semut dalam menemukan jalur, sedangkan UBK terinspirasi strategi berburu *black-winged kite*. Keduanya menggunakan sekumpulan kandidat solusi untuk mengeksplorasi ruang kebijakan ABAC secara simultan.

Meskipun meningkatkan kemampuan eksplorasi, pendekatan *population-based* membutuhkan penyimpanan banyak kandidat solusi secara eksplisit selama proses optimasi. Akibatnya, kebutuhan memori meningkat seiring ukuran populasi. Karakteristik ini menjadi kendala pada perangkat *edge-IoT* dengan kapasitas memori terbatas dan menjadi dasar kebutuhan terhadap pendekatan optimasi yang lebih *compact*.

II.5 Estimation of Distribution Algorithm

Estimation of Distribution Algorithm (EDA) merupakan kelas algoritma evolusioner yang menghasilkan solusi baru melalui model probabilistik, bukan operator *crossover* dan mutasi seperti pada *Genetic Algorithm*. EDA, yang juga dikenal sebagai *Probabilistic Model-Building Genetic Algorithm* (PMBGA), mempelajari distribusi probabilitas dari individu-individu terbaik, kemudian melakukan *sampling* dari dis-

tribusi tersebut untuk membentuk generasi berikutnya (Larrañaga dan Bielza, 2024 dan Lemtenneche dkk., 2023). Dengan demikian, pencarian solusi dipandu secara eksplisit oleh pola probabilistik yang dipelajari dari solusi berkualitas.

Secara umum, EDA menjalankan tiga tahap berulang, yaitu seleksi, estimasi, dan *sampling* (Larrañaga dan Bielza, 2024). Seleksi memilih individu terbaik berdasarkan nilai *fitness*, estimasi membangun model probabilistik dari individu terpilih, sedangkan *sampling* menghasilkan populasi baru dari model tersebut. Pada persoalan diskret, distribusi dapat direpresentasikan menggunakan *Bayesian network* yang memodelkan dependensi bersyarat antar-variabel secara lebih ringkas dibandingkan tabel probabilitas gabungan penuh. Proses tersebut diulang hingga kriteria penghentian tercapai.

Berdasarkan tingkat dependensi yang dimodelkan, EDA dibedakan menjadi tiga kategori, yaitu univariat, bivariat, dan multivariat. EDA univariat mengasumsikan seluruh variabel independen, EDA bivariat memodelkan dependensi berpasangan, sedangkan EDA multivariat memungkinkan dependensi yang lebih kompleks melalui struktur probabilistik multivariat (Larrañaga dan Bielza, 2024).

EDA telah diterapkan pada berbagai persoalan optimasi kombinatorial, termasuk masalah berbasis permutasi dengan ruang solusi besar dan kendala *mutual exclusivity* antar-elemen (Lemtenneche dkk., 2023). Meskipun ruang kebijakan ABAC bukan masalah permutasi, pembentukan kebijakan tetap bersifat kombinatorial karena setiap kandidat merupakan kombinasi diskret atribut subjek, objek, aksi, dan lingkungan. Oleh karena itu, prinsip EDA dalam mempelajari distribusi konfigurasi yang menjanjikan berpotensi digunakan untuk mengarahkan pencarian aturan ABAC.

Namun, EDA kanonik masih menyimpan populasi berukuran M secara eksplisit pada setiap generasi sehingga belum secara langsung menyelesaikan keterbatasan memori pada perangkat *edge-IoT*. Karena informasi pencarian pada EDA direpresentasikan dalam parameter model probabilistik, populasi eksplisit berpotensi digantikan oleh representasi probabilistik yang diperbarui secara inkremental. Karakteristik ini menjadi dasar pengembangan varian EDA yang lebih *compact* dengan kebutuhan memori lebih rendah.

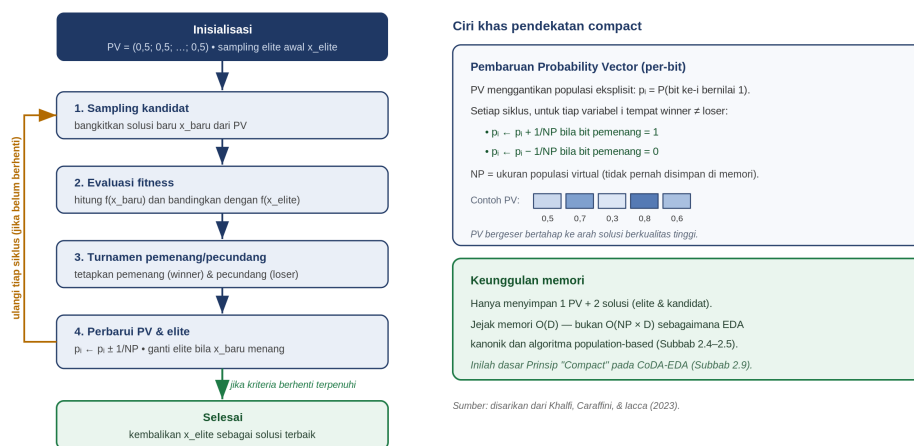
II.6 EDA Ringan/*Compact*

Sebagaimana telah diuraikan, EDA kanonik masih memerlukan penyimpanan populasi penuh pada setiap generasi. Keterbatasan ini dijawab oleh algoritma *compact*. Khalfi dkk. (2023) mendefinisikan algoritma *compact* sebagai bentuk sederhana

EDA yang menggantikan populasi eksplisit dengan satu *Probability Vector* (PV) yang merepresentasikan populasi virtual berukuran NP , tanpa menyimpan seluruh individunya di memori. Relevansi pendekatan ini ditunjukkan melalui contoh pada mikrokontroler ATmega328P dengan RAAM 2 KB, di mana algoritma genetika standar dengan 128 individu ber-*encoding* 32-bit dapat menghabiskan sekitar 80% memori, sehingga hampir tidak menyisakan ruang bagi proses lain (Khalfi dkk., 2023). Kondisi ini menggambarkan secara langsung tantangan memori pada perangkat *edge-IoT*.

Secara umum, algoritma *compact* diawali dengan inisialisasi PV dan *sampling* solusi elit awal. Pada setiap iterasi, satu solusi kandidat baru di-*sampling* dari PV, lalu dibandingkan dengan solusi elit melalui skema turnamen pemenang-pecundang. Selanjutnya, PV diperbarui berdasarkan perbedaan nilai variabel antara pemenang dan pecundang, dengan besar pembaruan yang berbanding terbalik terhadap ukuran populasi virtual (NP). Solusi elit kemudian diganti apabila kandidat baru lebih baik (Khalfi dkk., 2023).

Dengan mekanisme ini, hanya PV serta dua solusi, yaitu elit dan kandidat, yang perlu disimpan pada satu waktu. Akibatnya, kebutuhan memori turun dari $O(NP \times D)$ pada EDA kanonik atau algoritma *population-based* menjadi $O(D)$, dengan D menyatakan jumlah variabel keputusan. Siklus kerja algoritma *compact*, termasuk pembaruan PV dan keunggulan memorinya, diilustrasikan pada Gambar II.2.



Gambar II.2 Siklus Kerja Algoritma *Compact* dan Mekanisme Pembaruan PV

Gambar II.2 menunjukkan bahwa algoritma *compact* menjalankan proses iteratif dengan hanya menyimpan satu PV serta dua solusi, sehingga kebutuhan memorinya

tetap $O(D)$ tanpa menyimpan populasi virtual berukuran NP secara eksplisit. Prinsip pembaruan hemat memori ini diadopsi sebagai komponen *compact* pada solusi yang diusulkan.

Pada ruang pencarian biner, *compact-GA* merepresentasikan PV sebagai probabilitas per-bit yang diperbarui sebesar $\pm 1/NP$ berdasarkan perbedaan bit antara solusi pemenang dan pecundang (Khalfi dkk., 2023). Representasi ini sesuai untuk variabel diskret seperti yang terdapat pada ruang kebijakan ABAC. Pengembangannya mencakup varian elitis serta *Uniform Compact Genetic Algorithm* (UcGA), yang memungkinkan ukuran populasi virtual berbeda pada setiap variabel.

Untuk ruang kontine, *Real-Valued Compact Genetic Algorithm* (rcGA) menggunakan parameter distribusi Gaussian pada setiap dimensi, tetapi tetap menerapkan mekanisme pembaruan pemenang-pecundang. Prinsip *compact* juga diterapkan pada metaheuristik lain, seperti *compact-PSO* yang mengadaptasi mekanisme pencarian PSO ke dalam representasi probabilistik (Khalfi dkk., 2023). Hal ini menunjukkan bahwa penggantian populasi eksplisit dengan model probabilistik ringan dapat diterapkan pada berbagai mekanisme pencarian.

Namun, algoritma *compact* pada umumnya melakukan pencarian tak berkorelasi dan tidak menyimpan korelasi silang antar-variabel keputusan (Khalfi dkk., 2023). Dengan demikian, *compact-GA*, rcGA, dan *compact-PSO* standar pada dasarnya memodelkan setiap variabel secara independen. Meskipun efisien dari sisi memori, asumsi ini membatasi kemampuan algoritma dalam merepresentasikan dependensi antar-atribut pada ruang kebijakan ABAC. Kesenjangan antara efisiensi memori dan kemampuan memodelkan dependensi tersebut menjadi salah satu motivasi utama penelitian ini.

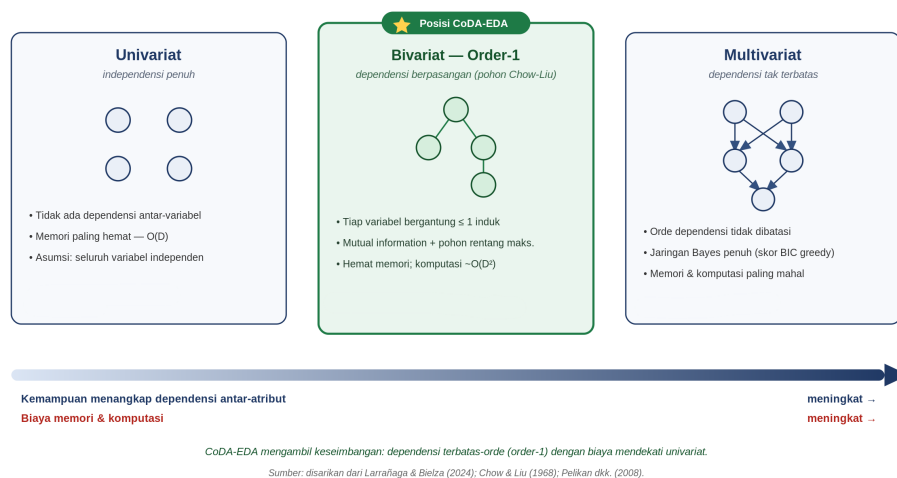
II.7 Pemodelan Dependensi

Kesenjangan antara efisiensi memori dan kemampuan memodelkan dependensi mendorong penggunaan model probabilistik yang tetap mampu menangkap hubungan antar-variabel. *Bayesian Optimization Algorithm* (BOA) merupakan EDA multivariat yang mempelajari *Bayesian network* tanpa membatasi derajat dependensi antar-variabel (Pelikan dkk., 2008). Strukturnya dipelajari secara *greedy* dari graf kosong dengan menambahkan *edge* yang paling meningkatkan skor *Bayesian Information Criterion* (BIC) hingga tidak diperoleh peningkatan lebih lanjut, sedangkan parameter jaringan diestimasi dari populasi solusi terpilih. BOA tetap menjadi salah satu pendekatan EDA multivariat yang digunakan dengan berbagai bentuk model,

termasuk jaringan Bayes diskret, Gaussian, semiparametrik, dan berbasis *kernel density* (Soloviev dkk., 2024).

Solusi baru pada BOA dibangkitkan melalui *logic sampling* berdasarkan struktur jaringan yang telah dipelajari (Pelikan dkk., 2008). Variabel terlebih dahulu diurutkan secara *ancestral* sehingga setiap induk mendahului variabel anaknya, kemudian nilainya di-*sampling* berdasarkan distribusi bersyarat terhadap nilai induk yang telah ditentukan. Mekanisme ini memungkinkan kebutuhan komputasi dan memori seiring kompleksitas jaringan.

Di antara asumsi independensi penuh EDA univariat dan dependensi multivariat pada BOA, terdapat pendekatan bivariat berbasis *dependency tree* Chow dan Liu (1968). Metode ini mengaproksimasi distribusi gabungan multidimensi melalui struktur pohon yang hanya mempertahankan dependensi berpasangan. Struktur tersebut diperoleh dengan menghitung *mutual information* setiap pasangan variabel dan membentuk *maximum spanning tree*. Chow dan Liu (1968) menunjukkan bahwa pohon yang dihasilkan memberikan aproksimasi dengan divergensi Kullback-Leibler minimum terhadap distribusi sebenarnya.



Gambar II.3 Spektrum Pemodelan Dependensi pada EDA dan Posisi CoDA-EDA

Gambar II.3 menunjukkan adanya *trade-off* antara kemampuan memodelkan dependensi dan biaya memori/komputasi. CoDA-EDA ditempatkan di antara pendekatan univariat dan multivariat penuh dengan mempertahankan dependensi antar-atribut yang relevan, tetapi dengan struktur yang dibatasi agar kebutuhan sumber dayanya tetap rendah.

Seperti EDA kanonik, BOA standar masih membutuhkan populasi eksplisit untuk mempelajari struktur dan parameter *Bayesian network*. Pelikan dkk. (2008) mengembangkan *incremental BOA* (iBOA) yang menghilangkan kebutuhan tersebut dengan memperbarui struktur dan parameter jaringan secara inkremental melalui skema turnamen pemenang/pecundang. Dengan demikian, iBOA menunjukkan bahwa pemodelan dependensi multivariat dapat dilakukan tanpa menyimpan populasi secara eksplisit.

Namun, skalabilitas iBOA tetap menjadi tantangan. Penyimpanan tabel probabilitas marginal untuk berbagai kemungkinan struktur dependensi dapat membutuhkan hingga $O(n^k)$ tabel, dengan k sebagai orde dependensi maksimum yang diizinkan (Pelikan dkk., 2008). Tanpa pembatasan struktur, kebutuhan ini dapat mengurangi keunggulan memori pendekatan *compact*. Celan tersebut menjadi dasar rancangan CoDA-EDA, yang menggabungkan pembaruan inkremental tanpa populasi eksplisit dengan struktur dependensi yang dibatasi agar tetap efisien secara memori dan komputasi.

II.8 Penelitian Terdahulu

Pendekatan evolusioner untuk ABAC *policy mining* telah diperkenalkan sejak Medvet dkk. (2015). Mereka mengusulkan optimasi multi-objektif dengan representasi dan operator genetik khusus ABAC, strategi *separate-and-conquer*, inisialisasi populasi kustom, promosi diversitas, dan penghentian dini. Tiga objektif yang dioptimalkan adalah konsistensi terhadap data akses, kompleksitas kebijakan yang rendah, dan penghindaran atribut yang secara unik mengidentifikasi subjek atau objek. Sebelumnya, penerapan teknik evolusioner pada kebijakan keamanan juga telah dieksplorasi secara terbatas, antara lain melalui *genetic programming* untuk kebijakan multi-level (Lim dkk., 2008) dan SPEA2 untuk pembelajaran kebijakan keamanan dinamis (Lim dkk., 2009 dan Medvet dkk., 2015).

Perkembangan berikutnya mengarah pada metaheuristik *population-based* berbasis kecerdasan kawanan. Shang dkk. (2024) menggunakan *Ant Colony Optimization* (ACO), sedangkan Li dkk. (2026) mengusulkan *Upgraded Black-winged Kite* (UBK), dengan evaluasi yang mencakup akurasi dan *Weighted Structural Complexity* (WSC).

Di luar pendekatan evolusioner, Guo dkk. (2025) mengusulkan kerangka *on-the-fly* yang memformulasikan *policy mining* sebagai optimasi aproksimasi submodular dengan pemangkasan ruang pencarian. Pendekatan ini berfokus pada usabilitas kebijakan bagi manusia maupun LLM dan dilaporkan mampu menurunkan redun-

dansi sebesar 93,2% dibandingkan metode pembandingan. Namun, kerangka tersebut bersifat deterministik/*greedy* dan ditujukan untuk kontrol akses secara umum, bukan ABAC secara spesifik.

Meskipun berbeda dalam mekanisme pencarian, Medvet dkk. (2015), Shang dkk. (2024), Li dkk. (2026), dan Guo dkk. (2025) tidak menjadikan keterbatasan memori perangkat *edge-IoT* sebagai pertimbangan utama dalam desain algoritmanya. Kesenjangan ini menjadi salah satu dasar pengembangan pendekatan *policy mining* yang lebih hemat sumber daya pada penelitian ini.

Tabel II.2 Ringkasan Posisi Penelitian Terdahulu

No	Peneliti (Tahun)	Metode	Fokus/Kontribusi	Celah terhadap CoDA-EDA
1	Medvet dkk. (2015)	<i>Genetic programming</i> multi-objektif (representasi fenotipik kustom)	Salah satu pendekatan evolusioner paling awal untuk ABAC policy mining; 3 objektif: konsistensi, kompleksitas rendah, hindari atribut identitas unik	Population-based (menyimpan populasi pohon ekspresi penuh); tidak dirancang untuk perangkat memori terbatas
2	Shang dkk. (2024)	<i>Ant Colony Optimization</i> (ACO)	Baseline population-based penelitian ini; dievaluasi dengan akurasi & WSC	Population-based — jejak memori (matriks feromon) tumbuh dengan ukuran populasi/masalah
3	Li dkk. (2026)	<i>Upgraded Black-winged Kite</i> (UBK)	Baseline population-based penelitian ini; kecerdasan kawanan berbasis populasi	Population-based — kebutuhan memori sama dengan algoritma population-based lain

Lanjut ke halaman berikutnya

Tabel II.2 – lanjutan dari halaman sebelumnya

No	Peneliti (Ta- hun)	Metode	Fokus/Kontribusi	Celah terhadap CoDA-EDA
4	Guo dkk. (2025)	Optimasi submodu- lar/aproksimasi (deterministik, non-evolusioner)	Kontrol akses umum (bukan ABAC spesifik); fokus usabilitas kebijakan bagi manusia/LLM	Bukan pendekat- an population- based/EDA; tidak membahas keter- batasan memori perangkat edge- IoT
5	— (celah pene- litian)	EDA / <i>compact-EDA</i> / <i>dependency- aware</i> EDA	Belum ditemuk- an penelitian yang menerapkan kelas algoritma ini pada ABAC policy mi- ning	Celah inilah yang diisi CoDA-EDA, bersama Premis 1- 4 dan Hipotesis H1-H4

Berdasarkan Tabel II.2, penelitian terdahulu pada *ABAC policy mining* belum menja-
dikan keterbatasan memori perangkat *edge-IoT* sebagai pertimbangan desain algori-
tma. Selain itu, berdasarkan penelusuran literatur yang dilakukan, belum ditemukan
penerapan *Estimation of Distribution Algorithm* (EDA) pada persoalan tersebut.
Kesenjangan ini menjadi dasar kebaruan penelitian yang diuraikan lebih lanjut pada
subbab berikutnya.

II.9 Posisi dan Kebaruan Penelitian

Tabel II.2 menunjukkan dua jalur penelitian yang berkembang secara terpisah. Jalur
pertama mencakup *ABAC policy mining* berbasis evolusioner atau metaheuristik
(Medvet dkk., 2015, Shang dkk., 2024 dan Li dkk., 2026) serta non-evolusioner
(Guo dkk., 2025), yang belum menjadikan keterbatasan memori perangkat *edge-IoT*
sebagai pertimbangan utama desain. Jalur kedua mencakup perkembangan EDA dan
varian hemat memori, seperti *compact-GA* dan *compact-PSO* (Khalfi dkk., 2023),
serta pemodelan dependensi melalui BOA, iBOA, dan pohon Chow-Liu (Pelikan
dkk., 2008 dan Chow dan Liu, 1968). Berdasarkan penelusuran literatur, pendekatan
tersebut belum diterapkan pada *ABAC policy mining*. CoDA-EDA diposisikan pada
titik temu kedua jalur tersebut.

CoDA-EDA dibangun berdasarkan tiga prinsip. Pertama, *compact*, yaitu menggunak-

an pembaruan pemenang-pecundang pada *Probability Vector* (PV) tanpa menyimpan populasi eksplisit (Khalifi dkk., 2023), sehingga sesuai dengan keterbatasan sumber daya perangkat *edge-IoT* (Canavese dkk., 2024). Kedua, *dependency-aware*, yaitu mempertahankan pemodelan dependensi antaratribut dengan struktur yang dibatasi, sebagai kompromi antara asumsi independensi pada algoritma *compact* dan pemodelan multivariat yang lebih kompleks pada BOA/iBOA (Pelikan dkk., 2008). Ketiga, EDA, yaitu menempatkan kedua mekanisme tersebut dalam kerangka seleksi, estimasi, dan *sampling* berbasis distribusi probabilistik.

Dinamika lingkungan IoT (Ragothaman dkk., 2023) juga menuntut kebijakan tetap adaptif terhadap perubahan data. Untuk menghindari biaya pembelajaran ulang struktur pada setiap siklus, CoDA-EDA memisahkan pembaruan parameter dan struktur dependensi. Parameter probabilistik diperbarui setiap siklus, sedangkan struktur hanya dipelajari ulang ketika terdeteksi pergeseran distribusi yang signifikan. Mekanisme ini berbeda dari iBOA yang memperbarui struktur dan parameter secara inkremental dalam satu kerangka pembaruan (Pelikan dkk., 2008), serta menjadi dasar evaluasi efisiensi komputasi dan kemampuan *warm-start* ketika terjadi perubahan data.

Deteksi perubahan tersebut memerlukan ukuran perbedaan distribusi antarjendela. Penelitian ini menggunakan *Jensen–Shannon divergence* (JSD) untuk mengukur besarnya pergeseran distribusi. Untuk dua distribusi probabilitas P dan Q dengan bobot yang sama, JSD dirumuskan sebagai:

$$\text{JSD}(P \parallel Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M), \quad M = \frac{1}{2}(P + Q), \quad (\text{II.1})$$

dengan KL menyatakan *Kullback–Leibler divergence*. JSD dipilih karena memiliki sifat yang sesuai untuk membandingkan distribusi diskret. Berbeda dari *KL divergence*, kelas divergensi berbasis entropi Shannon yang mencakup JSD tidak mensyaratkan kontinuitas absolut, sehingga tetap terdefinisi ketika suatu nilai atribut memiliki probabilitas nol pada salah satu distribusi (Lin, 1991). Dengan logaritma berbasis dua dan bobot yang sama, JSD juga bersifat non-negatif, simetris, dan terbatas pada rentang 0–1; nilai 0 menunjukkan distribusi identik, sedangkan nilai yang lebih besar menunjukkan perbedaan distribusi yang lebih tinggi (Lin, 1991). Sifat terbatas dan simetris tersebut memudahkan penggunaan JSD sebagai dasar penetapan ambang perubahan distribusi. Lin (1991) juga menunjukkan hubungan JSD dengan batas probabilitas kesalahan klasifikasi Bayes, yang memperkuat interp-

retasinya sebagai ukuran keterbedaan informasi antar-distribusi.

Penggunaan JSD untuk mendeteksi perubahan distribusi telah diterapkan pada *concept drift*. Yang dkk. (2022) mengusulkan CDJD (*Concept Drift Detection based on Jensen–Shannon Divergence*) dengan membandingkan distribusi fitur antara jendela lama dan jendela berjalan. *Concept drift* dinyatakan ketika JSD melampaui ambang tertentu, sedangkan kemiripan dengan konsep terdahulu dapat digunakan untuk memanfaatkan kembali model yang tersedia. Hasil eksperimen menunjukkan bahwa pendekatan tersebut mampu mendeteksi perubahan distribusi dan mempertahankan kinerja klasifikasi pada aliran data yang mengalami *concept drift* (Yang dkk., 2022).

CoDA-EDA mengadopsi prinsip tersebut untuk tujuan yang berbeda. JSD digunakan untuk mengukur pergeseran distribusi pasangan atribut–nilai antarjendela log akses. Struktur dependensi Chow–Liu dipelajari ulang hanya ketika JSD melampaui ambang τ , sedangkan parameter probabilistik tetap diperbarui pada setiap siklus. Dengan demikian, JSD berfungsi sebagai pemicu yang membedakan perubahan distribusi yang memerlukan pembaruan struktur dari variasi yang masih dapat ditangani melalui pembaruan parameter. Mekanisme ini mengurangi pembelajaran ulang struktur yang tidak diperlukan pada perangkat *edge* dengan sumber daya terbatas.

Dibandingkan penelitian pada Tabel II.2, CoDA-EDA berbeda dari pendekatan berbasis populasi Medvet dkk. (2015), Shang dkk. (2024), dan Li dkk. (2026) karena menggunakan pencarian probabilistik tanpa menyimpan populasi eksplisit. Perbedaan ini menjadi dasar evaluasi jejak memori terhadap BOA, iBOA, ACO, dan UBK. CoDA-EDA juga berbeda dari Guo dkk. (2025), yang menggunakan optimasi submodular deterministik/*greedy* dengan fokus pada usability kebijakan bagi manusia dan LLM. Sebaliknya, CoDA-EDA dirancang sebagai pencarian probabilistik berbasis model dengan fokus pada efisiensi sumber daya perangkat *edge*.

Diferensiasi CoDA-EDA terhadap EDA terdahulu tidak terletak pada satu komponen, tetapi pada kombinasi beberapa keputusan rancangan. Tabel II.3 membandingkan CoDA-EDA dengan keluarga EDA terdekat berdasarkan empat dimensi, yaitu: (1) model dependensi antaratribut, (2) penyimpanan populasi dan kebutuhan memori terhadap ukuran populasi virtual (NP), (3) mekanisme pembaruan struktur model, dan (4) domain penerapan.

Tabel II.3 Perbedaan CoDA-EDA terhadap Algoritma EDA yang Sudah Ada

Algoritma EDA	Model dependensi antar-atribut	Populasi & memori terhadap NP	Mekanisme pembaruan struktur	Diterapkan pada ABAC?
<i>compact-GA</i> / <i>compact-PSO</i>	Tidak ada — univariat, independensi penuh	Tanpa populasi eksplisit; <i>Probability Vector</i> $O(D)$, memori datar terhadap NP	Tidak ada struktur untuk diperbarui	Belum
BOA	Jaringan Bayes tanpa batasan orde (multivariat penuh)	Populasi eksplisit; memori tumbuh terhadap NP	Membangun ulang jaringan Bayes tiap generasi (mahal secara komputasi)	Belum
iBOA	Pohon dependensi (Chow-Liu), inkremental	Statistik/populasi inkremental; tidak menanggung transien populasi penuh	Struktur dan parameter diperbarui pada skema inkremental yang sama, tiap langkah	Belum
CoDA-EDA (diusulkan)	Pohon dependensi Chow-Liu dibatasi order-1 (terbatas, bukan tanpa batas)	Tanpa populasi eksplisit; <i>Probability Vector</i> + pohon $O(D)$, memori datar terhadap NP	Struktur dipisah dari parameter: struktur jarang (dipicu <i>drift</i>), parameter tiap siklus	Ya — penelitian ini (Raspberry Pi 5)

Tabel II.3 memperjelas posisi CoDA-EDA pada setiap dimensi pembanding. Dari sisi dependensi, CoDA-EDA berada di antara pendekatan *compact* yang mengasumsikan independensi penuh dan BOA yang memodelkan dependensi multivariat tanpa batasan orde, dengan menggunakan pohon Chow–Liu berorde satu. Dari sisi memori, CoDA-EDA mempertahankan kebutuhan yang tidak bergantung pada ukuran populasi virtual (NP) sebagaimana keluarga *compact*, tetapi tetap memodelkan dependensi antarvariabel. Perbedaan utama terhadap iBOA terletak pada mekanisme pembaruan

model: parameter probabilistik diperbarui setiap siklus, sedangkan struktur dependensi hanya dipelajari ulang ketika terdeteksi *drift*. Kombinasi karakteristik tersebut menjadi dasar kontribusi metodologis CoDA-EDA.

Kontribusi tersebut terletak pada penggabungan tiga mekanisme yang sebelumnya digunakan secara terpisah dalam literatur EDA, yaitu pembaruan tanpa populasi eksplisit ala *compact*, pemodelan dependensi berorde terbatas melalui pohon Chow–Liu, dan pemisahan frekuensi pembaruan struktur serta parameter. Berdasarkan sintesis tersebut, penelitian ini mengusulkan tiga pilar kebaruan.

Kebaruan pertama adalah penerapan EDA yang ringan dan peka terhadap dependensi pada *ABAC policy mining*, yang berdasarkan penelusuran literatur belum ditemukan pada domain tersebut. Kebaruan kedua adalah penggabungan pemodelan dependensi orde pertama menggunakan pohon Chow–Liu dengan mekanisme pembaruan tanpa populasi eksplisit. Kebaruan ketiga adalah pemisahan pembaruan parameter yang dilakukan setiap siklus dari pembelajaran ulang struktur yang hanya dipicu ketika terjadi pergeseran distribusi.

Rancangan tersebut selanjutnya dievaluasi pada perangkat *edge-IoT* nyata, yaitu Raspberry Pi 5, untuk menguji apakah efisiensi memori dan komputasi dapat dicapai tanpa mengorbankan kualitas kebijakan secara signifikan dibandingkan metode pembandingan. Evaluasi ini melengkapi studi pada Tabel II.2 yang tidak menempatkan keterbatasan sumber daya perangkat *edge* sebagai fokus utama rancangan algoritma.

II.10 Kerangka Berpikir

Kerangka berpikir penelitian merangkum hubungan antara premis masalah, sintesis kajian pustaka, rancangan solusi, hipotesis, dan aktivitas metodologi. Alur tersebut disajikan pada Gambar II.4.



Gambar II.4 Kerangka Berpikir Penelitian

Gambar II.4 menunjukkan bahwa Premis 1–4 menjadi dasar pembentukan tiga prinsip rancangan CoDA-EDA. Prinsip tersebut kemudian diterjemahkan ke dalam Hipotesis H1–H4 dan diuji secara empiris melalui enam aktivitas DSRM (Pefferers dkk., 2007), dengan luaran berupa algoritma yang tervalidasi pada perangkat *edge-IoT* nyata. Kerangka ini menjadi acuan dalam perancangan algoritma, instrumen evaluasi, dan desain eksperimen.

Bab III Metodologi Penelitian

III.1 Analisis Masalah

Sebelum artefak dirancang, persoalan *ABAC policy mining* pada *edge-IoT* diuraikan menjadi empat *design tension* yang menjadi dasar penurunan kebutuhan dan keputusan rancangan CoDA-EDA.

1. Keterbatasan sumber daya *edge*. Perangkat *edge-IoT* memiliki memori dan daya komputasi terbatas, sedangkan algoritma berbasis populasi seperti BOA, ACO, dan UBK menyimpan banyak kandidat solusi sehingga kebutuhan memorinya meningkat seiring ukuran populasi. Tantangannya adalah memperoleh kebijakan berkualitas tanpa mempertahankan populasi eksplisit.
2. Dependensi versus efisiensi memori. Algoritma *compact* univariat hemat memori, tetapi mengasumsikan independensi antarvariabel. Sebaliknya, BOA/iBOA mampu memodelkan dependensi yang lebih kompleks dengan biaya representasi dan komputasi lebih tinggi. Tantangannya adalah mempertahankan informasi dependensi dengan biaya sumber daya yang tetap rendah.
3. Dinamika data versus biaya pembaruan. Lingkungan IoT yang berubah menyebabkan kebijakan perlu diperbarui secara berkala. Penambahan ulang atau pembelajaran struktur dependensi pada setiap siklus dapat meningkatkan biaya komputasi dan energi. Oleh karena itu, diperlukan mekanisme pembaruan yang adaptif tanpa selalu membangun model dari awal.
4. Kualitas versus efisiensi. Efisiensi sumber daya tidak boleh menyebabkan penurunan kualitas kebijakan yang berlebihan. Algoritma perlu mempertahankan kualitas yang kompetitif sekaligus memenuhi batasan memori dan biaya pembaruan pada perangkat *edge*.

Keempat persoalan tersebut terlihat pada tiga kelompok algoritma pembanding. Keluarga *compact*, seperti *compact-GA* dan *compact-PSO*, memiliki kebutuhan memori $O(D)$ yang tidak bergantung pada ukuran populasi virtual, tetapi menggunakan model univariat sehingga tidak merepresentasikan dependensi antarvariabel. Keterbatasan utamanya adalah kompromi antara efisiensi memori dan kemampuan memodelkan struktur hubungan dalam ruang solusi.

Kelompok kedua adalah EDA yang memodelkan dependensi, yaitu BOA dan iBOA. BOA memerlukan populasi eksplisit serta pembelajaran struktur *Bayesian network* sehingga relatif mahal dari sisi memori dan komputasi. iBOA menghilangkan populasi eksplisit melalui pembaruan statistik secara inkremental dan menjadi pembanding yang lebih dekat dengan CoDA-EDA. Namun, struktur dan parameter model diperbarui dalam proses inkremental yang sama, tanpa mekanisme berbasis *drift* untuk membedakan kapan struktur benar-benar perlu dipelajari ulang.

Kelompok ketiga adalah metaheuristik berbasis populasi seperti ACO dan UBK. Keduanya menyediakan kemampuan eksplorasi yang luas dan menjadi pembanding kualitas kebijakan, tetapi memerlukan representasi populasi atau informasi pencarian yang bertambah seiring konfigurasi algoritma serta tidak dirancang secara khusus untuk pembaruan kebijakan inkremental pada perangkat *edge*.

Dengan demikian, algoritma pembanding memenuhi sebagian kebutuhan, tetapi belum secara bersamaan menyediakan efisiensi memori, pemodelan dependensi, dan pembaruan adaptif untuk lingkungan *edge-IoT*. Kesenjangan tersebut menjadi dasar rancangan CoDA-EDA, yang menggabungkan representasi *compact*, pemodelan dependensi berorde terbatas, dan pembaruan struktur berbasis *drift*. Ringkasan analisis ini disajikan pada Tabel III.1

Tabel III.1 Analisis Masalah pada Algoritma Pembanding terhadap Kebutuhan Inti Edge-IoT

Algoritma (keluarga)	Sadar dependensi	Memori datar thd populasi	Pembaruan inkremental efisien	Masalah utama di edge-IoT
compact-GA (compact)	tidak	ya	tidak	Buta dependensi → kualitas turun pada dimensi tinggi
compact-PSO (compact)	tidak	ya	tidak	Independensi penuh antar-atribut, sama seperti compact-GA
BOA (EDA penuh)	ya	tidak	tidak	Populasi eksplisit + bangun ulang struktur tiap generasi → boros
iBOA (EDA inkremental)	ya	ya	tidak	Relearn struktur kontinu, tak sadar-drift, tanpa warm-start
ACO (berpopulasi)	ya*	tidak	tidak	Populasi/feromon tumbuh; batch/offline, bukan edge
UBK (berpopulasi)	ya*	tidak	tidak	Berpopulasi eksplisit → memori tumbuh; tidak inkremental
CoDA-EDA (usulan)	ya (orde-1)	ya	ya	memenuhi ketiga kebutuhan inti

ya* = keterkaitan atribut ditangani secara implisit melalui jejak feromon/konstruksi solusi, bukan model dependensi eksplisit.

III.2 Analisis Kebutuhan

Berdasarkan analisis masalah, kebutuhan artefak CoDA-EDA dibagi menjadi kebutuhan fungsional dan non-fungsional.

Kebutuhan fungsional (KF):

KF1. Menambang aturan ABAC berbentuk konjungsi kondisi atribut dari log akses untuk merekonstruksi keputusan izin dan tolak.

KF2. Memodelkan dependensi berpasangan antaratribut dalam proses pembangkitan kandidat aturan.

KF3. Memperbarui kebijakan secara inkremental ketika terjadi perubahan distribusi data, termasuk mendeteksi terjadinya *drift*.

Kebutuhan non-fungsional (KNF):

KNF1 (memori) — Jejak memori bersifat $O(D)$ dan tidak tumbuh terhadap ukuran populasi, agar layak pada perangkat *edge*.

KNF2 (efisiensi pembaruan) — Pembaruan berkala hemat waktu; operasi mahal (pembelajaran struktur) hanya dijalankan bila diperlukan.

KNF3 (kualitas) — Kualitas kebijakan non-inferior terhadap algoritma baseline sekelas.

Keterkaitan antara persoalan, kebutuhan, dan keputusan rancangan yang menjawabnya dirangkum pada Tabel III.2. Dengan demikian, setiap komponen rancangan CoDA-EDA bukan merupakan pilihan sembarangan, melainkan jawaban langsung atas kebutuhan yang diturunkan dari analisis masalah.

Tabel III.2 Pemetaan Analisis Masalah — Kebutuhan — Keputusan Rancangan

Persoalan	Kebutuhan	Keputusan rancangan
1. Sumber daya <i>edge</i> terbatas	KNF1 (memori $O(D)$, tanpa populasi)	<i>Probability Vector</i> (PV) tanpa populasi eksplisit; pembaruan kompetitif $\pm 1/NP$
2. Dependensi vs. hemat memori	KF2 + KNF1	Pohon Chow-Liu orde-1 ($O(D)$), bukan jaringan dependensi penuh ($O(D^2)$)
3. Dinamika & biaya pembaruan	KF3 + KNF2	Pemisahan ritme: struktur dipicu- <i>drift</i> ($JSD > \tau$) + parameter tiap siklus + <i>warm-start</i>
4. Kompromi kualitas proporsional	KNF3	Fungsi <i>fitness</i> selaras-F1 (cakupan + presisi) + penambangan <i>separate-and-conquer</i>

III.3 Variabel Penelitian

Penetapan variabel penelitian dilakukan agar aktivitas *Demonstration* dan *Evaluation* dapat dioperasionalkan secara jelas serta pengujian hipotesis berlangsung secara terkontrol. Variabel penelitian dibagi menjadi variabel bebas, terikat, dan kontrol.

Variabel bebas mencakup tiga faktor. Pertama, jenis algoritma yang terdiri atas tujuh kondisi, yaitu CoDA-EDA, *compact-GA*, *compact-PSO*, BOA, iBOA, *Ant Colony Optimization* (ACO), dan *Upgraded Black-winged Kite* (UBK). Kedua, tingkat korelasi antaratribut pada dataset pengujian, yang dikendalikan pada kategori rendah, sedang, dan tinggi melalui perluasan generator ABAC Lab. Ketiga, skenario *drift* data akses, yaitu besarnya perubahan antarsiklus yang memengaruhi pemicu pembaruan struktur dependensi pada CoDA-EDA.

Variabel terikat mencakup empat kelompok metrik: (1) kualitas kebijakan, meliputi akurasi/*F1-score*, *Weighted Structural Complexity* (WSC), dan *coverage*; (2) jejak memori, yang diukur melalui konsumsi memori puncak; (3) waktu komputasi, mencakup waktu pembelajaran awal dan satu siklus pembaruan inkremental; serta (4) kinerja re-optimasi pada skenario *drift*, yang membandingkan waktu dan kualitas kebijakan antara strategi *warm-start* dan *re-mining* penuh.

Untuk menjaga keadilan perbandingan, seluruh algoritma dijalankan pada perangkat keras yang sama, yaitu Raspberry Pi 5, menggunakan dataset identik untuk setiap skenario. Kriteria pemberhentian disetarakan antaralgoritma, sedangkan setiap kombinasi algoritma dan skenario dieksekusi berulang dengan *random seed* yang berbeda.

Definisi operasional seluruh variabel beserta keterkaitannya dengan hipotesis dirangkum pada III.3.

Tabel III.3 Definisi Operasional Variabel Penelitian

No	Variabel	Jenis	Definisi Operasional	Terkait Hipotesis
1	Algoritma yang dibandingkan	Bebas	Tujuh kondisi: CoDA-EDA (diusulkan) dan enam algoritma pembanding (<i>compact-GA</i> , <i>compact-PSO</i> , BOA, iBOA, ACO, UBK). Dari keenam pembanding, tiga berpopulasi eksplisit (BOA, ACO, UBK), sedangkan <i>compact-GA/PSO</i> dan iBOA berjejak memori datar terhadap NP.	H1 (vs <i>compact-GA/PSO</i>); H2 (vs BOA, ACO, UBK); H3 (vs iBOA)
2	Tingkat korelasi atribut	Bebas	Tiga tingkat terkontrol (rendah, sedang, tinggi) yang disuntikkan pada generator ABAC Lab yang diperluas.	H1
2b	Dimensi ruang solusi (D)	Bebas	Banyaknya pasangan atribut-nilai yang membentuk vektor keputusan: D = 436 pada <i>workforce</i> dan D = 1.907 pada <i>e-document</i> . Ditetapkan sebagai variabel bebas berdasarkan studi pendahuluan yang menunjukkan dimensi soal sebagai faktor penentu keunggulan pemodelan dependensi.	H1
3	Skenario <i>drift</i> data akses	Bebas	Besaran perubahan (delta) pada log akses antar-siklus pembaruan yang memicu pembaruan struktur dependensi (tidak ada/rendah/sedang).	H3, H4

Lanjut ke halaman berikutnya

Tabel III.3 – lanjutan dari halaman sebelumnya

No	Variabel	Jenis	Definisi Operasional	Terkait Hipotesis
4	Kualitas kebijakan	Terikat	Akurasi/ <i>F-score</i> , <i>Weighted Structural Complexity</i> (WSC), dan cakupan (<i>coverage</i>) aturan terhadap log akses; formula rinci pada bagian metode evaluasi.	H1, H3, H4
5	Jejak memori	Terikat	Konsumsi memori puncak (<i>peak resident set size</i>) selama algoritma berjalan pada perangkat target; prosedur pengukuran pada bagian metode evaluasi.	H2
6	Waktu komputasi	Terikat	Waktu pembelajaran awal (<i>initial training</i>) dan waktu satu siklus pembaruan inkremental; prosedur pengukuran pada bagian metode evaluasi.	H3
7	Efisiensi re-optimalisasi pada <i>drift</i>	Terikat	Perbandingan waktu dan kualitas kebijakan antara strategi <i>warm-start</i> dan <i>re-mining</i> penuh dari awal pasca- <i>drift</i> .	H4

III.4 Rancangan Algoritma CoDA-EDA

Berdasarkan tiga prinsip rancangan sebelumnya, subbab ini menjabarkan CoDA-EDA secara konkret, meliputi representasi solusi dan model probabilistik, inisialisasi dan *sampling*, pembaruan parameter, pembaruan struktur dependensi, algoritma keseluruhan, serta kompleksitas memori dan waktu. Rancangan ini merupakan hasil aktivitas *Design and Development* yang dapat disempurnakan berdasarkan hasil *Demonstration* dan *Evaluation* sesuai sifat iteratif DSRM.

Ketiga unsur utama CoDA-EDA diwujudkan secara langsung dalam rancangan algoritma. Unsur *compact* diterapkan melalui pembaruan *Probability Vector* (PV)

tanpa populasi eksplisit, yang membedakannya dari BOA. Unsur *dependency-aware* diwujudkan melalui pohon Chow–Liu orde satu yang dibangun menggunakan algoritma Prim agar kebutuhan memorinya tetap $O(D)$, berbeda dari asumsi independensi *compact-GA/PSO* maupun jaringan Bayes multivariat BOA. Unsur ketiga adalah pemisahan pembaruan parameter yang dilakukan setiap siklus dari pembaruan struktur yang hanya dipicu ketika *drift* melampaui ambang τ , yang membedakannya dari iBOA.

Setiap kandidat aturan ABAC direpresentasikan sebagai vektor biner $x = (x_1, x_2, \dots, x_D)$, dengan D menyatakan jumlah pasangan atribut–nilai dan aksi yang mungkin. Nilai $x_i = 1$ menunjukkan bahwa pasangan ke- i digunakan dalam aturan, sedangkan $x_i = 0$ menunjukkan sebaliknya. Kebijakan multi-aturan dibentuk menggunakan strategi *separate-and-conquer*: algoritma mencari satu aturan, mengeluarkan *permit* yang telah tercakup, kemudian mengulangi proses hingga anggaran evaluasi habis atau tidak diperoleh tambahan cakupan. Semantik aturan mengikuti ABAC Lab, yaitu nilai dalam satu atribut bersifat disjungtif, antaratribut bersifat konjungtif, dan atribut tanpa nilai terpilih dianggap tidak dibatasi (Bui dkk., 2025). WSC suatu aturan sama dengan jumlah pasangan atribut–nilai aktif dalam vektor tersebut.

Kualitas kandidat dievaluasi melalui fungsi *fitness* $f(x) = \frac{TP^2}{TP + w_{fp}FP} - w_{wsc}WSC(x)$, dengan TP menyatakan jumlah *permit* baru yang dicakup, FP jumlah *deny* yang keliru diizinkan, dan $WSC(x)$ kompleksitas aturan. Suku pertama ekuivalen dengan TP dikalikan presisi sehingga secara simultan menghargai cakupan dan ketepatan aturan. Kandidat dengan $TP = 0$ diberi nilai yang sangat rendah agar tidak terpilih. Formulasi ini menggantikan penalti linier $TP - w_{fp}FP$, yang pada pengujian awal cenderung menghasilkan aturan sangat spesifik dengan cakupan rendah. Keputusan ini sejalan dengan pentingnya ukuran kualitas aturan dalam mengarahkan proses *separate-and-conquer* (Gudyś dkk., 2024).

Inisialisasi PV dibuat sadar-dimensi karena aturan ABAC yang baik umumnya relatif ringkas, sedangkan inisialisasi $p_i = 0,5$ menghasilkan sekitar separuh dari D variabel aktif dan cenderung membentuk aturan terlalu spesifik. Oleh karena itu, setiap elemen PV diinisialisasi dengan $p_i = p_0 = \frac{k}{D}$, dengan $k \approx 3$, sehingga jumlah atribut–nilai aktif yang diharapkan tetap relatif konstan ketika dimensi berubah. Pilihan ini konsisten dengan penggunaan kesederhanaan struktural sebagai salah satu ukuran kualitas kebijakan ABAC (Perez-Haro dan Diaz-Perez, 2024 dan Quan dkk., 2023). Pencarian juga dibenihkan dari satu contoh *permit* yang belum tercakup untuk menghindari wilayah *fitness* datar akibat rendahnya peluang aturan

acak mencocokkan data, sesuai prinsip pembelajaran berbasis contoh positif pada *sequential covering* (Gudyś dkk., 2024).

Model probabilistik CoDA-EDA terdiri atas PV dan struktur dependensi T . PV, $PV = (p_1, p_2, \dots, p_D)$, merepresentasikan probabilitas marginal setiap variabel serta populasi virtual berukuran NP , mengikuti prinsip algoritma *compact* (Khalfi dkk., 2023). Sementara itu, T merupakan pohon Chow–Liu yang mempertahankan dependensi berpasangan dengan *mutual information* tertinggi (Chow dan Liu, 1968). Struktur pohon membatasi setiap variabel nonakar pada satu induk, sehingga lebih sederhana dibandingkan *Bayesian network* multivariat pada BOA/iBOA (Pelikan dkk., 2008).

Pada awal pencarian, PV ditetapkan sebesar $p_i = k/D$ dan struktur T belum memuat dependensi. Solusi elit awal dibentuk dari contoh *permit* yang belum tercakup. Setelah struktur tersedia, kandidat baru di-*sampling* menurut urutan ancestral T : variabel akar dibangkitkan berdasarkan probabilitas marginal, sedangkan variabel anak menggunakan probabilitas bersyarat terhadap nilai induknya.

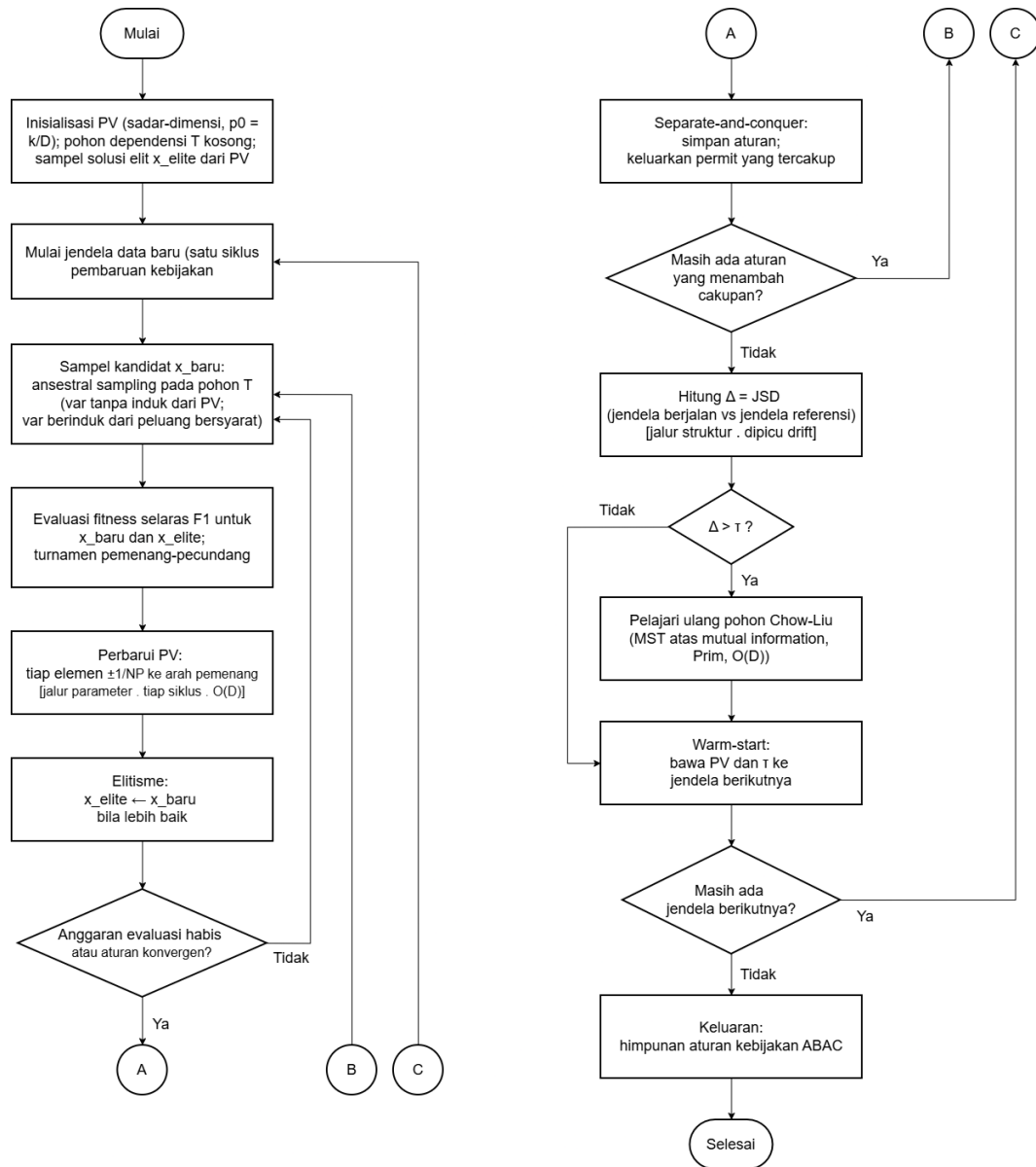
Parameter probabilistik diperbarui pada setiap iterasi menggunakan kompetisi pemenang–pecundang seperti pada *compact-GA* dan iBOA. Kandidat dan elit dibandingkan berdasarkan *fitness*, kemudian probabilitas variabel diperbarui sebesar $\pm 1/NP$ sesuai perbedaan nilai antara pemenang dan pecundang. Kandidat menggantikan elit apabila memiliki *fitness* lebih baik. Karena tidak ada populasi eksplisit yang disimpan, mekanisme ini mempertahankan kebutuhan memori linear terhadap D .

Berbeda dengan parameter, struktur dependensi T hanya diperbarui ketika terjadi perubahan distribusi yang signifikan. Pergeseran distribusi pasangan atribut–nilai antara jendela berjalan dan jendela referensi diukur sebagai Δ . Jika $\Delta > \tau$, struktur Chow–Liu dipelajari ulang melalui *mutual information* berpasangan dan *maximum spanning tree*, sedangkan pembaruan parameter tetap berlangsung pada setiap siklus.

Pembelajaran struktur dirancang agar tidak menghilangkan sifat *compact*. *Maximum spanning tree* dibangun menggunakan algoritma Prim (Cormen dkk., 2022) dengan menyimpan hanya larik sepanjang D , sementara *mutual information* dihitung sesuai kebutuhan tanpa menyimpan matriks $D \times D$. Dengan demikian, kebutuhan memori tambahan tetap $O(D)$. Implementasi awal yang menyimpan seluruh matriks *mutual information* membutuhkan sekitar 29 MB pada $D = 1.907$, sehingga tidak digunakan. Perhitungan *mutual information* juga menggunakan presisi ganda untuk menghindari hilangnya presisi numerik ketika probabilitas mendekati batas 0 atau 1.

Pemisahan pembaruan parameter dan struktur merupakan mekanisme utama CoDA-

EDA: parameter beradaptasi pada setiap siklus, sedangkan struktur hanya dipelajari ulang ketika perubahan distribusi cukup besar. Mekanisme ini menjadi dasar pengujian H3 dan H4. Arsitektur keseluruhan, yang mencakup model probabilistik, jalur pembaruan parameter, dan jalur pembaruan struktur berbasis *drift*, ditunjukkan pada Gambar III.1, sedangkan langkah algoritmiknya dirinci pada Algoritma III.1.



Gambar III.1 Diagram Alir Arsitektur CoDA-EDA

Algoritma III.1 merangkum keseluruhan mekanisme CoDA-EDA. Baris 1–2 memuat inisialisasi, baris 4–12 menjalankan *sampling* dan pembaruan parameter pada setiap iterasi, sedangkan baris 13–17 menangani deteksi *drift* dan pembaruan struktur secara bersyarat.

Algoritma III.1 CoDA-EDA (Compact Dependency-Aware EDA)

Input: NP (ukuran populasi virtual), τ (ambang delta pemicu pembaruan struktur), kriteria pemberhentian

Output: kebijakan ABAC elite ($\mathbf{x}_{\text{elite}}$)

```
1: Inisialisasi  $\mathbf{PV} \leftarrow (0,5, 0,5, \dots, 0,5)$ ;  $T \leftarrow \emptyset$  (struktur kosong)
2:  $\mathbf{x}_{\text{elite}} \leftarrow \text{SAMPLE}(\mathbf{PV})$ 
3: while kriteria pemberhentian belum terpenuhi do
4:    $\mathbf{x}_{\text{baru}} \leftarrow \text{SAMPLE}(\mathbf{PV}, T)$   $\triangleright$  urutan ancestral mengikuti  $T$ 
5:   evaluasi  $f(\mathbf{x}_{\text{baru}})$  dan  $f(\mathbf{x}_{\text{elite}})$ 
6:   (winner, loser)  $\leftarrow \text{TURNAMEN}(\mathbf{x}_{\text{baru}}, \mathbf{x}_{\text{elite}})$ 
7:   for  $i \leftarrow 1$  to  $D$  do
8:      $\mathbf{PV}[i] \leftarrow \mathbf{PV}[i] \pm \frac{1}{NP}$  berdasarkan winner[i], loser[i]
9:   end for
10:  if  $f(\mathbf{x}_{\text{baru}})$  lebih baik dari  $f(\mathbf{x}_{\text{elite}})$  then
11:     $\mathbf{x}_{\text{elite}} \leftarrow \mathbf{x}_{\text{baru}}$ 
12:  end if
13:   $\Delta \leftarrow \text{UKUR\_DRIFT}(\text{jendela akses berjalan}, \text{jendela referensi})$ 
14:  if  $\Delta > \tau$  then
15:     $T \leftarrow \text{CHOW-LIU}(\text{mutual information berpasangan}, \text{pohon rentang maksimum})$ 
16:    perbarui jendela referensi
17:  end if
18: end while
19: return  $\mathbf{x}_{\text{elite}}$ 
```

Analisis kompleksitas berikut diturunkan berdasarkan rancangan algoritma (*by construction*) dan selanjutnya divalidasi secara empiris melalui eksperimen. Penelitian ini tidak mencakup pembuktian konvergensi formal secara mendalam.

Kompleksitas memori CoDA-EDA adalah $O(D)$. PV membutuhkan $O(D)$, sedangkan struktur pohon T juga membutuhkan $O(D)$ karena setiap variabel hanya menyimpan satu referensi induk dan tabel probabilitas bersyarat berukuran konstan. Dengan demikian, kebutuhan memori tetap linear terhadap dimensi dan tidak bergantung pada ukuran populasi virtual (NP). Karakteristik ini sebanding dengan algoritma *compact* standar, tetapi lebih terbatas dibandingkan iBOA yang dapat memerlukan hingga $O(n^k)$ tabel probabilitas ketika orde dependensi k tidak dibatasi.

Untuk analisis waktu, D menyatakan dimensi ruang solusi, n jumlah entri *log* per jendela, E anggaran evaluasi *fitness* per jendela, W jumlah jendela, δ proporsi jendela yang memicu pembaruan struktur, dan k rata-rata jumlah kondisi aktif dalam aturan. Karena aturan ABAC bersifat jarang, k diharapkan jauh lebih kecil daripada D .

Satu evaluasi kandidat terdiri atas tiga tahap. Pertama, kandidat dibangkitkan melalui *ancestral sampling* pada pohon T dengan biaya $O(D)$. Kedua, *fitness* dihitung dengan memindai n entri *log* untuk memperoleh TP dan FP . Pada implementasi umum, biaya terburuknya adalah $O(n \cdot D)$, sedangkan dengan hanya memeriksa k kondisi aktif dapat mendekati $O(n \cdot k)$. Ketiga, PV diperbarui dalam $O(D)$. Dengan demikian, kompleksitas terburuk satu evaluasi adalah $O(n \cdot D)$, sehingga E evaluasi membutuhkan $O(E \cdot n \cdot D)$ per jendela.

Jalur pembaruan struktur diawali dengan deteksi *drift*. Distribusi pasangan atribut–nilai pada jendela berjalan dihitung dan dibandingkan dengan jendela referensi menggunakan JSD, dengan biaya terburuk $O(n \cdot D)$. Jika $\Delta > \tau$, struktur Chow–Liu dipelajari ulang. Perhitungan *mutual information* untuk seluruh pasangan variabel membutuhkan $O(n \cdot D^2)$, sedangkan pembentukan *maximum spanning tree* dengan algoritma Prim pada graf lengkap membutuhkan $O(D^2)$. Oleh karena itu, biaya pembelajaran ulang struktur didominasi oleh $O(n \cdot D^2)$.

Karena pembelajaran struktur hanya terjadi pada proporsi δ dari seluruh jendela, biaya total untuk W jendela dapat dinyatakan secara ringkas sebagai $O(W \cdot E \cdot n \cdot D + W \cdot n \cdot D + \delta W \cdot n \cdot D^2)$, atau, dengan mengabaikan suku yang terdominasi, $O(W \cdot E \cdot n \cdot D + \delta W \cdot n \cdot D^2)$.

Dengan demikian, jalur parameter tetap linear terhadap D untuk pembaruan model dan didominasi oleh biaya evaluasi *fitness*, sedangkan biaya kuadratik terhadap

dimensi hanya muncul ketika struktur dependensi perlu dipelajari ulang. Pemisahan ini merupakan tujuan utama mekanisme berbasis *drift*: menghindari biaya $O(n \cdot D^2)$ pada jendela yang distribusinya masih stabil.

Tabel III.4 Perbandingan kompleksitas waktu dan memori per siklus/generasi

Algoritma	Sampling 1 kandidat	Evaluasi <i>fitness</i>	Pembaruan model per siklus	Pembaruan struktur	Memori
compact-GA / compact- PSO	$O(D)$	$O(n \cdot D)$	$O(D)$ (PV)	— (tanpa struktur)	$O(D)$
CoDA-EDA	$O(D)$ (ancestral pd pohon)	$O(n \cdot D)$	$O(D)$ (PV)	$O(n \cdot D^2)$ — hanya saat <i>drift</i>	$O(D)$
iBOA	$O(D)$	$O(n \cdot D)$	$O(D)$	$O(D^2)$ tiap langkah (kontinu)	$O(D)$
BOA	$O(NP \cdot D)$	$O(NP \cdot n \cdot D)$	—	$O(NP \cdot D^2)$ bangun ulang tiap generasi	$O(NP \cdot D)$
ACO / UBK	$O(NP \cdot D)$ (konstruk- si)	$O(NP \cdot n \cdot D)$	$O(NP \cdot D)$ (feromon)	— (tak inkremental)	$O(NP \cdot D)$

Terbaca bahwa jalur per siklus CoDA-EDA satu orde dengan keluarga *compact* $O(n \cdot D)$, sedangkan operasi struktur $O(n \cdot D^2)$ -nya hanya dibayar saat *drift*. Bandingkan dengan iBOA yang membayar biaya struktur $O(D^2)$ pada setiap langkah (kontinu) dan dengan BOA/ACO/UBK yang seluruh biayanya diperbesar faktor populasi NP. Perbedaan inilah yang membuat CoDA-EDA lebih efisien pada pembaruan berkala sekaligus tetap hemat memori.

III.5 Konstruksi Dataset Penelitian

Dataset ABAC dengan tingkat dependensi antaratribut yang dapat dikendalikan belum tersedia sebagai sumber daya siap pakai untuk kebutuhan penelitian ini. ABAC Lab menyediakan lima dataset *ground-truth*, yaitu *healthcare*, *project-management*, *university*, *workforce*, dan *e-document*, beserta *generator log* sintetis yang mendukung pengaturan ukuran *log*, rasio *permit/deny*, dan injeksi *noise*. Namun, *generator*

tersebut belum menyediakan mekanisme untuk mengendalikan tingkat dependensi antaratribut. Oleh karena itu, konstruksi dataset dilakukan sebagai bagian dari aktivitas *Design and Development*.

Generator ABAC Lab diperluas dengan mekanisme injeksi dependensi atribut secara terkontrol. Alih-alih men-*sampling* seluruh atribut secara independen, pasangan atribut yang relevan pada setiap domain *disampling* dari distribusi bersama yang dikalibrasi terhadap target *Normalized Mutual Information* (NMI). Tiga tingkat dependensi ditetapkan sebagai variabel bebas, yaitu rendah, sedang, dan tinggi, dengan target NMI masing-masing 0,05, 0,35, dan 0,65. Nilai terendah ditetapkan 0,05 karena proses pembentukan *log* berdasarkan kebijakan *ground-truth* dapat menghasilkan dependensi dasar antaratribut sehingga kondisi independensi sempurna tidak selalu dapat dipertahankan.

Sebagai ilustrasi, digunakan atribut *position* dan *associatedContract* pada domain *workforce*. Misalkan $P(\text{position} = p) = 0,30$ dan $P(\text{associatedContract} = c) = 0,20$. Jika keduanya independen, probabilitas kemunculan bersama adalah $0,30 \times 0,20 = 0,06$, sehingga $P(\text{associatedContract} = c \mid \text{position} = p) = 0,20$. Injeksi dependensi dilakukan dengan menggeser sebagian massa probabilitas ke kombinasi nilai tertentu tanpa mengubah distribusi marginal. Sebagai contoh, peningkatan probabilitas gabungan dari 0,060 menjadi 0,151 akan meningkatkan probabilitas bersyarat menjadi sekitar 0,503. Dengan demikian, frekuensi marginal setiap nilai tetap dipertahankan, tetapi pola kemunculan bersama menjadi lebih terstruktur. Perubahan distribusi bersama inilah yang digunakan untuk mengendalikan tingkat dependensi antaratribut pada dataset pengujian.

Tabel III.5 Ilustrasi distribusi bersama dua atribut sebelum injeksi korelasi

	<i>associatedContract</i> = <i>c</i>	<i>associatedContract</i> ≠ <i>c</i>	Total
<i>position</i> = <i>p</i>	0,060	0,240	0,300
<i>position</i> ≠ <i>p</i>	0,140	0,560	0,700
Total	0,200	0,800	1,000

Tabel III.6 Ilustrasi distribusi bersama dua atribut setelah injeksi korelasi

	<i>associatedContract</i> = <i>c</i>	<i>associatedContract</i> ≠ <i>c</i>	Total
<i>position</i> = <i>p</i>	0,151	0,149	0,300
<i>position</i> ≠ <i>p</i>	0,049	0,651	0,700
Total	0,200	0,800	1,000

Tabel III.5 dan Tabel III.6 menunjukkan bahwa injeksi dependensi tidak meng-

ubah distribusi marginal kedua atribut. Proporsi $position = p$ tetap 0,30 dan $associatedContract = c$ tetap 0,20, sedangkan perubahan hanya terjadi pada distribusi bersama. Probabilitas gabungan $P(position = p, associatedContract = c)$ meningkat dari 0,060 menjadi 0,151 sehingga probabilitas bersyarat berubah dari

$$P(associatedContract = c | position = p) = \frac{0,060}{0,300} = 0,20 \quad (III.1)$$

menjadi

$$P(associatedContract = c | position = p) = \frac{0,151}{0,300} \approx 0,503. \quad (III.2)$$

Dengan demikian, nilai $position$ menjadi informatif terhadap $associatedContract$ meskipun distribusi marginal keduanya tetap. Dependensi ini diukur menggunakan *Normalized Mutual Information* (NMI) dan dapat dimanfaatkan oleh model *dependency-aware*. Berbeda dengan *compact-GA* dan *compact-PSO* yang hanya mempertahankan probabilitas marginal, CoDA-EDA menggunakan *mutual information* untuk memilih dependensi yang direpresentasikan pada pohon Chow–Liu. Nilai di atas hanya bersifat ilustratif; pada eksperimen, tingkat dependensi dikalibrasi secara *closed-loop* hingga NMI empiris mendekati target 0,05, 0,35, dan 0,65.

Kalibrasi dilakukan melalui pencarian biner terhadap parameter penggandeng distribusi bersama berdasarkan NMI pada log yang benar-benar dihasilkan. Ukuran log kalibrasi disamakan dengan log final karena estimasi NMI dipengaruhi ukuran sampel dan banyaknya sel kontingensi. Pada pengujian awal, penggunaan 800 entri untuk mengkalibrasi log final 10.000 entri menghasilkan galat NMI hingga $-0,11$ pada *e-document* yang memiliki 379 sel kontingensi. Ketika ukuran log kalibrasi disamakan dengan log final, galat tersebut turun menjadi sekitar $-0,005$.

Pengaruh ukuran sampel juga ditemukan pada pengukuran *drift* menggunakan *Jensen–Shannon divergence* (JSD). Dalam kondisi stasioner, JSD tetap dapat menghasilkan nilai positif sebagai lantai derau yang berbeda antardataset. Pada jendela awal berukuran kecil, lantai derau tercatat 0,017 pada *workforce* dengan 45 sel dan 0,132–0,145 pada *e-document* dengan 379 sel. Setelah ukuran jendela diperbesar, nilai pada *e-document* turun menjadi sekitar 0,014–0,0333, tetapi masih dapat melampaui ambang $\tau = 0,01$. Oleh karena itu, besaran *drift* dilaporkan sebagai

JSD bersih, yaitu JSD terukur dikurangi lantai derau dari kondisi stasioner. Dengan koreksi tersebut, galat terhadap target *drift* berada pada kisaran -0,006 hingga -0,022. Kondisi stasioner karenanya digunakan sebagai bagian dari prosedur kalibrasi, bukan hanya sebagai kondisi pembanding.

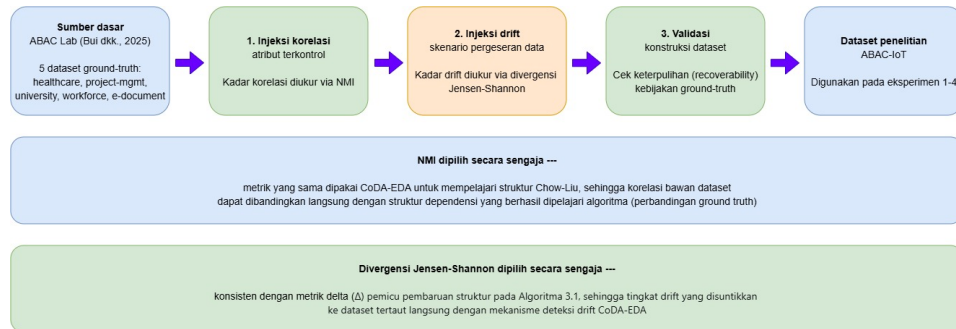
Untuk menguji H3 dan H4, *log* akses dibagi menjadi jendela berurutan yang merepresentasikan siklus pembaruan kebijakan. Distribusi antarjendela kemudian diubah secara terkontrol dan besarnya pergeseran, Δ , diukur menggunakan JSD terhadap jendela referensi. Sebelum digunakan pada aktivitas *Demonstration*, setiap varian dataset diverifikasi berdasarkan NMI aktual, tingkat *drift*, serta proporsi *permit/deny*. Alur konstruksi dan validasi dataset dirangkum pada Gambar III.2.

Pelaksanaan awal konstruksi dataset menghasilkan tiga temuan yang memengaruhi rancangan eksperimen. Pertama, enumerasi ruang subjek–objek–aksi menunjukkan bahwa proporsi *permit* pada dataset ABAC Lab relatif rendah, yaitu 1,99% pada *workforce*, 2,50% pada *university*, 3,32% pada *project-management*, 4,27% pada *healthcare*, dan 5,34% pada *e-document*. Oleh karena itu, *log* tidak dibangkitkan melalui *sampling* seragam, tetapi menggunakan stratifikasi *permit/deny* dengan himpunan *permit* yang diprakomputasi.

Kedua, seluruh dataset merupakan data sintetis yang dihasilkan dari *generator* ABAC Lab yang diperluas, sehingga karakteristik yang dilaporkan merupakan properti hasil konstruksi, bukan data operasional nyata. Setelah dibatasi pada entitas dengan pasangan atribut yang dapat dikorelasikan, jumlah *permit* unik adalah 14 pada *healthcare*, 17 pada *project-management*, 90 pada *university*, 14.513 pada *workforce*, dan 32.961 pada *e-document*. Tiga dataset pertama menghasilkan duplikasi sekitar 48–93% ketika *log* diperbesar sehingga keragaman pola menjadi terbatas. Karena itu, *workforce* dan *e-document* dijadikan dataset utama untuk H1, sedangkan tiga dataset lainnya digunakan sebagai konteks pelengkap. Untuk H2–H4, seluruh dataset tetap digunakan sesuai karakteristik masing-masing. Dimensi ruang solusi masing-masing adalah ($D=46$), 51, 54, 436, dan 1.907 untuk *healthcare*, *project-management*, *university*, *workforce*, dan *e-document*.

Ketiga, tingkat duplikasi meningkat seiring target dependensi. Pada *workforce*, duplikasi meningkat dari 17% menjadi 37% dan 51%, sedangkan pada *e-document* meningkat dari 4% menjadi 14% dan 30% untuk tingkat rendah, sedang, dan tinggi. Peningkatan ini merupakan konsekuensi dari pemusatan massa probabilitas pada sejumlah kombinasi atribut ketika dependensi diperkuat. Karena permintaan berulang juga dapat muncul pada *log* akses, duplikasi tidak dihilangkan, tetapi tingkatnya dilaporkan pada setiap konfigurasi untuk menunjukkan keragaman efektif data yang

digunakan.



Gambar III.2 *Pipeline* Konstruksi Dataset Penelitian

Gambar III.2 menegaskan bahwa metrik konstruksi dataset diselaraskan dengan mekanisme internal CoDA-EDA. Dependensi antaratribut dikendalikan menggunakan NMI, yang berbasis *mutual information* sebagaimana pembelajaran struktur Chow–Liu, sedangkan *drift* diukur menggunakan JSD yang juga digunakan sebagai pemicu pembaruan struktur. Dengan demikian, tingkat dependensi dan *drift* pada dataset dapat dibandingkan dengan respons algoritma sehingga pengujian H1, H3, dan H4 memiliki acuan *ground-truth* yang terukur.

Selain dataset hasil konstruksi, penelitian ini menggunakan *Smart Building IoT* dari Díaz-Rodríguez dan Díaz-Pérez (2025) sebagai validasi eksternal untuk menilai apakah temuan tetap berlaku pada data yang dihasilkan melalui *generator*, kebijakan, dan distribusi berbeda. Dataset digunakan tanpa injeksi dependensi maupun *drift* tambahan. Dari sekitar 5,7 juta permintaan akses dengan rasio *permit* 0,75 dan dimensi awal sekitar 1.164, data disubsampel secara terstratifikasi dengan mempertahankan rasio *permit*, sedangkan nilai atribut dengan *support* rendah digabungkan sehingga dimensi efektif menjadi sekitar 428. Dataset ini juga bersifat sintesis; nilai validasinya terletak pada sumber dan proses pembangkitan yang independen dari konstruksi dataset penelitian, bukan pada representasinya sebagai data operasional nyata.

Tabel III.7 Skema Dataset Penelitian

No	Dataset Dasar	Dimensi D	Permit Unik	Perluasan yang Diterapkan	Peran dalam Penelitian
1	<i>Workforce</i> (ABAC Lab)	436	14.513	3 tingkat korelasi (NMI rendah/sedang/tinggi) $\times$ 3 skenario drift (stasioner/rendah/sedang)	EKSPERIMEN UTAMA — pengujian H1 (korelasi) dan H3-H4 (drift)
2	<i>E-document</i> (ABAC Lab)	1.907	32.961	3 tingkat korelasi $\times$ 3 skenario drift	EKSPERIMEN UTAMA — pengujian H1 dan H3-H4 pada dimensi lebih besar
3	<i>University</i> (ABAC Lab)	54	90	3 tingkat korelasi $\times$ 3 skenario drift	KONTEKS PELENGKAP — H2/H3/H4 sah; vonis H1 terbatas (duplikasi 27-57% pada log memadai)
4	<i>Project-management</i> (ABAC Lab)	51	17	3 tingkat korelasi $\times$ 3 skenario drift	KONTEKS PELENGKAP — H2/H3/H4 sah; vonis H1 terbatas (duplikasi 48-69%)
5	<i>Healthcare</i> (ABAC Lab)	46	14	3 tingkat korelasi $\times$ 3 skenario drift	KONTEKS PELENGKAP — H2/H3/H4 sah; vonis H1 terbatas (duplikasi 58-93%)

Lanjut ke halaman berikutnya

Tabel III.7 – lanjutan dari halaman sebelumnya

No	Dataset Dasar	Dimensi D	Permit Unik	Perluasan yang Diterapkan	Peran dalam Penelitian
6	<i>Smart Building IoT</i> (Díaz-Rodríguez dan Díaz-Pérez, 2025)	428*	—	Digunakan apa adanya; subsampel terstratifikasi dari 5,7 juta baris, tanpa injeksi korelasi/drift	VALIDASI EKSTERNAL — generalisasi H1-H4 pada dataset berdomain IoT yang dibangkitkan pihak lain

Dataset-dataset pada Tabel III.7 menjadi lingkungan pengujian bersama bagi CoDA-EDA dan seluruh algoritma pembanding. Tanda (*) pada dataset *Smart Building IoT* menunjukkan dimensi efektif setelah subsampel dan penyatuan nilai *bersupport* rendah yang mana berkas asalnya memuat 5,7 juta permintaan akses dengan rasio *permit* 0,75.

III.6 Algoritma Pembanding

Sebagaimana ditetapkan pada variabel bebas jenis algoritma, CoDA-EDA dibandingkan dengan enam *baseline*, yaitu *compact-GA*, *compact-PSO*, BOA, iBOA, ACO, dan UBK. Seluruh algoritma diimplementasikan dan dikonfigurasi dengan prinsip perbandingan yang setara agar hasil *Evaluation* tidak dipengaruhi perbedaan representasi, anggaran komputasi, maupun kualitas implementasi.

Compact-GA dan *compact-PSO* mengikuti *template* algoritma *compact* dengan PV univariat, kompetisi pemenang–pecundang, dan ukuran populasi virtual (NP) sebagai parameter utama. Keduanya menggunakan representasi biner yang sama dengan CoDA-EDA agar perbedaan hasil terutama mencerminkan mekanisme pencarian dan pemodelan dependensi. *Compact-PSO*, yang berasal dari ruang kontinu, diadaptasi ke representasi biner menggunakan pendekatan probabilistik yang sejalan dengan praktik pembineran metaheuristik kontinu (Pan dkk., 2023).

BOA dan iBOA diimplementasikan menggunakan pembelajaran struktur berbasis BIC dan estimasi parameter kemungkinan maksimum tanpa pembatasan orde dependensi. BOA mempertahankan populasi eksplisit dan mempelajari ulang struktur

pada setiap generasi, sedangkan iBOA memperbarui struktur dan parameter secara inkremental melalui skema turnamen. Karena karakteristik tersebut, iBOA menjadi pembanding terdekat untuk mengevaluasi mekanisme pembaruan CoDA-EDA pada H3.

ACO dan UBK digunakan sebagai *baseline* metaheuristik berbasis populasi. ACO menggunakan mekanisme feromon dengan parameter evaporasi serta bobot heuristik (α, β) , sedangkan UBK mengikuti mekanisme pencarian berbasis populasi sebagaimana publikasi aslinya. Parameter awal mengikuti rekomendasi sumber asli dan dapat dikalibrasi pada dataset penelitian agar kinerja *baseline* tetap representatif.

Kesetaraan perbandingan dijaga melalui empat kontrol. Pertama, seluruh algoritma menggunakan representasi solusi dan fungsi *fitness* yang sama. Kedua, anggaran komputasi disetarakan berdasarkan jumlah maksimum evaluasi *fitness*, bukan jumlah iterasi, karena definisi iterasi berbeda antaralgoritma (Rajwar dkk., 2023 dan Omran dkk., 2022). Anggaran ditetapkan sebagai fungsi dimensi D , mengingat dua dataset utama memiliki dimensi berbeda, yaitu 436 pada *workforce* dan 1.907 pada *e-document*. Ketiga, anggaran dihitung untuk keseluruhan proses *separate-and-conquer*, bukan per aturan. Keempat, seluruh algoritma dijalankan pada perangkat keras dan dataset yang sama untuk setiap skenario, dengan replikasi menggunakan *random seed* berbeda untuk mengakomodasi variabilitas stokastik.

Tabel III.8 Algoritma Pembanding dan Perlakuan Kesetaraan Implementasi

No	Algoritma	Sumber	Kelas	Parameter Kunci	Catatan Kesetaraan
1	<i>compact-GA</i>	(Khalfi dkk., 2023)	<i>Compact</i> , univariat (diskret)	Ukuran populasi virtual (NP)	Representasi biner identik dengan CoDA-EDA; tanpa pemodelan dependensi
2	<i>compact-PSO</i>	(Khalfi dkk., 2023)	<i>Compact</i> , univariat (kontinu, diadaptasi)	NP; parameter kecepatan-posisi	Representasi diadaptasi ke ruang diskret agar sebanding dengan <i>compact-GA</i>

Lanjut ke halaman berikutnya

Tabel III.8 – lanjutan dari halaman sebelumnya

No	Algoritma	Sumber	Kelas	Parameter Kunci	Catatan Kesetaraan
3	BOA	(Pelikan dkk., 2008)	<i>Population-based</i> , dependensi tak terbatas	Ukuran populasi; ambang BIC	Struktur dipelajari ulang penuh tiap generasi; representasi biner sama
4	iBOA	(Pelikan dkk., 2008)	Inkremental, dependensi tak terbatas	Ambang/ kriteria koneksi struktur	Skema turnamen sama dengan CoDA-EDA; pembanding langsung untuk H3
5	ACO	(Shang dkk., 2024)	<i>Population-based</i> , <i>swarm</i>	Jumlah semut; evaporasi feromon; α , β	Parameter mengikuti publikasi asli, dikalibrasi bila perlu
6	UBK	(Li dkk., 2026)	<i>Population-based</i> , <i>swarm</i>	Ukuran populasi; parameter strategi berburu	Parameter mengikuti publikasi asli, dikalibrasi bila perlu

Selain kontrol eksperimen, kesetaraan juga dijaga pada operator bantu yang digunakan lintas algoritma. Operator *repair* untuk memastikan aturan valid, yaitu memiliki sekurang-kurangnya satu aksi dan satu kondisi atribut aktif, serta prosedur pembenihan dari contoh *permit* diterapkan identik pada seluruh tujuh kondisi algoritma. Seluruh solusi juga menggunakan tipe struktur data yang sama agar perbedaan konsumsi memori merefleksikan karakteristik algoritma, bukan *overhead* implementasi. Kontrol ini penting terutama untuk menjaga kesahihan pengujian H2.

Sebelum eksperimen utama, setiap implementasi menjalani uji monotonisitas terhadap anggaran evaluasi. Peningkatan anggaran tidak seharusnya menghasilkan penurunan kualitas kebijakan secara sistematis. Uji ini digunakan untuk mendeteksi ketidakselarasan antara fungsi *fitness* dan metrik evaluasi yang mungkin tidak terlihat pada satu tingkat anggaran. Implementasi yang menunjukkan pola penurunan konsisten akan diperiksa dan diperbaiki sebelum digunakan dalam pengujian hipotesis.

III.7 Perangkat dan Lingkungan Eksperimen

Seluruh algoritma dijalankan pada perangkat dan lingkungan eksperimen yang identik sesuai variabel kontrol. Subbab ini menjelaskan perangkat keras dan perangkat lunak yang digunakan, serta prosedur pengukuran memori dan waktu komputasi pada aktivitas *Demonstration* dan *Evaluation*.

Perangkat target adalah Raspberry Pi 5 varian RAM 8 GB LPDDR4X-4267 dengan *System-on-Chip Broadcom BCM2717*, yang memiliki *CPU quad-core ARM Cortex-A76 2,4 GHz*, *cache L2 512 KB per inti*, dan *cache L3 bersama 2 MB* (Raspberry Pi Ltd, 2026). Perangkat menggunakan penyimpanan microSD dan Raspberry Pi OS 64-bit berbasis Debian. Konfigurasi ini digunakan untuk merepresentasikan perangkat *edge-IoT* dengan sumber daya komputasi terbatas dibandingkan *server* konvensional.

CoDA-EDA dan seluruh *baseline* diimplementasikan menggunakan Python. Pemilihan ini konsisten dengan ekosistem implementasi EDA yang tersedia; BOA dan berbagai EDA multivariat, misalnya, telah tersedia dalam paket Python seperti EDAspy (Soloviev dkk., 2024), sehingga implementasi *baseline* dapat mengacu pada implementasi referensi yang tersedia.

Jejak memori diukur menggunakan dua instrumen yang saling melengkapi. Metrik utama, $MemPeak_{alloc}$, merupakan puncak alokasi memori struktur data algoritma yang diukur menggunakan *tracemalloc*. Metrik kedua, $MemPeak_{sys}$, diperoleh dari VmHWM pada `/proc/[pid]/status` untuk memberikan konteks penggunaan memori pada tingkat sistem operasi. Pemisahan ini diperlukan karena VmHWM mencakup *overhead* interpreter dan pustaka; pada perangkat target, nilai awalnya sekitar 31 MB, sedangkan perbedaan struktur data antaralgoritma dapat berada pada skala *kilobyte* hingga beberapa *megabyte*. Dengan demikian, penggunaan VmHWM saja dapat menyamarkan perbedaan memori algoritmik.

Kedua metrik dicatat sebagai nilai puncak karena kelayakan eksekusi pada perangkat *edge* ditentukan oleh kebutuhan memori maksimum selama algoritma berjalan. Setiap kombinasi algoritma dan dataset dieksekusi dalam proses sistem operasi yang terpisah karena VmHWM bersifat *high-water mark* dan tidak menurun selama umur proses. Pendekatan ini juga mencegah pengukuran sebelumnya memengaruhi konfigurasi berikutnya. Eksekusi dilakukan satu per satu tanpa algoritma lain yang berjalan secara bersamaan untuk mengurangi interferensi penggunaan sumber daya.

Waktu komputasi diukur menggunakan *wall-clock time* pada perangkat target dan

dibedakan menjadi waktu pembelajaran awal (*initial training*) serta waktu satu siklus pembaruan inkremental. Untuk mengurangi variasi akibat *dynamic frequency scaling* dan *thermal throttling*, *CPU governor* ditetapkan pada mode *performance* selama pengujian dan suhu perangkat dipantau agar tetap berada dalam rentang operasi yang ditetapkan produsen.

Tabel III.9 Spesifikasi Perangkat dan Lingkungan Eksperimen

Komponen	Spesifikasi/Prosedur
Perangkat	Raspberry Pi 5 (Raspberry Pi Ltd., 2026)
SoC	Broadcom BCM2712, quad-core ARM Cortex-A76 @ 2,4GHz, cache L2 512KB/core, cache L3 bersama 2MB
RAM	8GB LPDDR4X-4267 SDRAM
Penyimpanan	microSD (mode SDR104); PCIe 2.0 x1 tersedia untuk SSD via M.2 HAT terpisah (tidak dipakai sebagai konfigurasi baku)
Suhu Operasi	0°C–70°C (spesifikasi produsen)
Sistem Operasi	Raspberry Pi OS 64-bit (berbasis Debian)
Bahasa Pemrograman	Python
Pengukuran Memori	Dual: puncak alokasi <code>tracemalloc</code> (metrik utama) + <code>VmHWM</code> pada <code>/proc/[pid]/status</code> (konteks sistem); satu proses terpisah per konfigurasi
Pengukuran Waktu	Wall-clock time langsung pada perangkat target; CPU governor mode <i>performance</i> ; suhu dipantau terhadap rentang operasi resmi

III.8 Desain Eksperimen

Berdasarkan variabel penelitian, dataset, algoritma pembandingan, dan prosedur pengukuran yang telah ditetapkan, dirancang empat eksperimen yang masing-masing dipetakan terhadap hipotesis penelitian sebagai operasionalisasi aktivitas *Demonstration* dan *Evaluation*.

Eksperimen 1 menguji H1 melalui dua faktor, yaitu tingkat dependensi antaratribut dan dimensi ruang solusi (D). Ketujuh algoritma dijalankan pada lima dataset ABAC Lab dalam kondisi stasioner pada tiga tingkat dependensi, yaitu rendah, sedang, dan

tinggi. Metrik yang diukur meliputi *F1-score*, *WSC*, dan *coverage*. Dataset *workforce* ($D = 436$) dan *e-document* ($D = 1.907$) menjadi dataset utama karena memiliki keragaman pola yang memadai, sedangkan *healthcare*, *project-management*, dan *university* ($D = 46\text{--}54$) digunakan sebagai konteks pelengkap karena tingkat duplikasinya lebih tinggi. H1 menduga manfaat pemodelan dependensi CoDA-EDA semakin terlihat ketika tingkat dependensi antaratribut meningkat.

Eksperimen 2a menguji H2 melalui pengukuran jejak memori pada eksekusi yang sama dengan Eksperimen 1. Metrik utama yang digunakan adalah $MemPeak_{alloc}$, sedangkan $MemPeak_{sys}$ digunakan sebagai konteks penggunaan memori pada tingkat sistem. Pengujian ini membandingkan kebutuhan memori absolut antaralgoritma, khususnya pada dataset berdimensi besar.

Eksperimen 2b melengkapi pengujian H2 dengan mengevaluasi bentuk penskalaan memori terhadap ukuran populasi atau populasi virtual (NP), yang divariasikan pada $NP \in 25, 50, 100, 200, 400$. Representasi solusi, fungsi *fitness*, dataset, dimensi (D), dan prosedur pengukuran dipertahankan identik. Setiap konfigurasi dijalankan dalam proses terpisah agar pengukuran puncak memori tidak dipengaruhi eksekusi sebelumnya.

Pengaruh NP terhadap memori bergantung pada mekanisme algoritma. Pada BOA, ACO, dan UBK, peningkatan NP menambah jumlah kandidat yang disimpan sehingga komponen memorinya secara konseptual tumbuh terhadap $O(NP \times D)$. Sebaliknya, pada CoDA-EDA dan keluarga *compact*, NP hanya mengatur granularitas pembaruan probabilitas sebesar $\pm 1/NP$ tanpa mematerialisasi NP individu. iBOA juga menggunakan statistik inkremental tanpa menyimpan populasi penuh. Oleh karena itu, sensitivitas memori terhadap NP diringkas menggunakan

$$R_{NP} = \frac{MemPeak_{alloc}(NP = 400)}{MemPeak_{alloc}(NP = 25)}. \quad (III.3)$$

Nilai R_{NP} mendekati 1 menunjukkan jejak memori relatif tidak sensitif terhadap NP, sedangkan nilai yang lebih besar menunjukkan pertumbuhan memori. H2 mengharapkan kurva CoDA-EDA, algoritma *compact*, dan iBOA relatif datar, sedangkan BOA, ACO, dan UBK meningkat seiring NP. Dengan demikian, Eksperimen 2b memberikan bukti empiris terhadap perbedaan perilaku memori $O(D)$ dan $O(NP \times D)$.

Eksperimen 3 menguji H3 dengan membandingkan CoDA-EDA dan iBOA pada

lima dataset ABAC Lab dalam skenario *drift* rendah dan sedang. Waktu komputasi setiap siklus pembaruan dicatat secara terpisah untuk siklus yang hanya memperbarui parameter dan siklus yang juga memperbarui struktur. H3 menduga CoDA-EDA membutuhkan waktu pembaruan rata-rata lebih rendah karena pembelajaran struktur dilakukan secara bersyarat ketika *drift* melampaui ambang.

Eksperimen 4 menguji H4 melalui dua strategi pembaruan CoDA-EDA, yaitu *warm-start* dengan melanjutkan model sebelum *drift* dan *re-mining* penuh dengan menginisialisasi ulang proses setelah *drift* terdeteksi. Waktu re-optimasi dan kualitas kebijakan pasca-*drift* dibandingkan untuk kedua strategi. H4 menduga *warm-start* dapat mencapai kualitas yang sebanding dengan *re-mining* penuh dengan waktu pembaruan lebih rendah, terutama pada *drift* berintensitas rendah.

Pada eksperimen utama, setiap kombinasi algoritma, dataset, dan skenario dijalankan menggunakan enam *seed* pembangkitan dataset dan lima replikasi algoritma, sehingga menghasilkan 30 replikasi per kombinasi. Validasi eksternal menggunakan lima *seed*. Replikasi tersebut digunakan untuk mengakomodasi variabilitas stokastik dan menyediakan data yang memadai bagi analisis statistik.

Tabel III.10 Ringkasan Desain Eksperimen

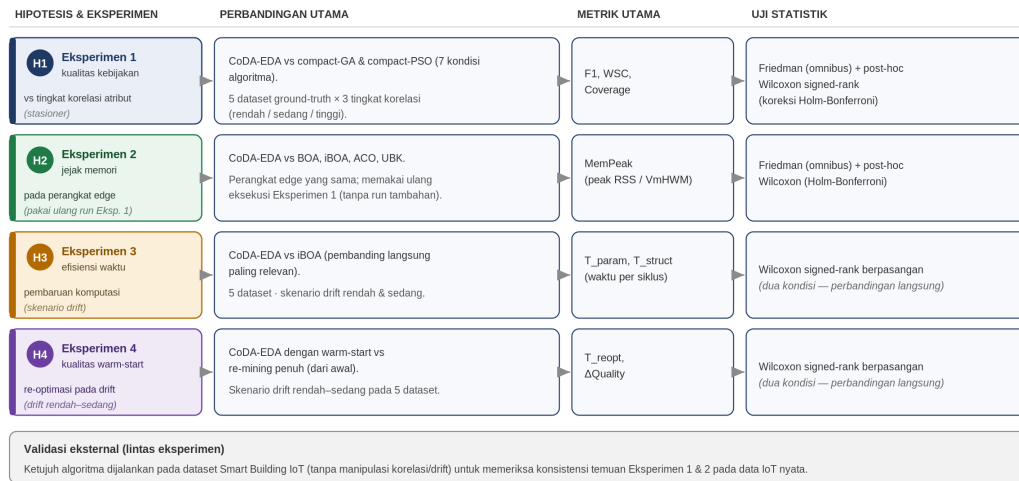
No	Eksperimen	Hipotesis	Algoritma Dilibatkan	Dataset	Variabel Diukur
1	Kualitas kebijakan vs. korelasi dan dimensi	H1	7 kondisi (CoDA-EDA + 6 baseline)	Baris 1–5 (lima dataset; dua taraf D utama 436 dan 1.907 + tiga dataset kecil D=46–54 sebagai konteks), kondisi stasioner, 3 tingkat korelasi	Akurasi/F-score, WSC, coverage
2a	Jejak memori antar-algoritma	H2	CoDA-EDA vs BOA, iBOA, ACO, UBK	Sama dengan Eksperimen 1 (reuse eksekusi)	MemPeak_alloc (utama), MemPeak_sys

Lanjut ke halaman berikutnya

Tabel III.10 – lanjutan dari halaman sebelumnya

No	Eksperimen	Hipotesis	Algoritma Dilibatkan	Dataset	Variabel Diukur
2b	Kurva penskalaan memori	H2	CoDA-EDA vs BOA, iBOA, ACO, UBK	Baris 1–2; NP divariasikan (25–400) pada dua nilai D	MemPeak_alloc sebagai fungsi NP dan D
3	Efisiensi waktu pembaruan	H3	CoDA-EDA vs iBOA	Baris 1–5, skenario drift rendah-sedang	Waktu komputasi per siklus pembaruan
4	Kualitas & efisiensi warm-start	H4	CoDA-EDA (warm-start vs re-mining penuh)	Baris 1–5, skenario drift rendah-sedang	Waktu re-optimasi, kualitas kebijakan pasca-drift
5	Validasi eksternal	H1-H4	7 kondisi (CoDA-EDA + 6 baseline)	Baris 6 (Smart Building IoT)	F-score relatif garis sepele, presisi, recall, coverage, memori, warm-start

Keterkaitan antara keempat hipotesis, eksperimen yang mengujinya, perbandingan yang dilakukan, metrik yang diukur, dan uji statistik yang digunakan divisualisasikan sebagai peta pada Gambar III.3.



Setiap kombinasi kondisi dijalankan dalam 30 replikasi dengan seed acak berbeda untuk memungkinkan uji statistik

Gambar III.3 Peta Eksperimen: Pemetaan Hipotesis H1-H4 terhadap Perbandingan, Metrik, dan Uji Statistik

Gambar III.3 menegaskan bahwa setiap hipotesis diuji oleh tepat satu eksperimen dengan perbandingan, metrik, dan uji statistik yang spesifik. Eksperimen 1 dan 2 membandingkan tujuh kondisi algoritma sekaligus sehingga memakai uji omnibus Friedman dengan post-hoc Wilcoxon berkoreksi Holm-Bonferroni, sedangkan Eksperimen 3 dan 4 hanya membandingkan dua kondisi sehingga cukup memakai Wilcoxon signed-rank berpasangan langsung. Peta ini juga memperlihatkan efisiensi rancangan pada Eksperimen 2 yang memakai ulang Eksperimen 1.

III.9 Metrik dan Instrumen Evaluasi

Definisi konseptual variabel terikat telah dijelaskan sebelumnya. Bagian ini menetapkan definisi operasional, formula, dan instrumen pengukuran setiap metrik yang digunakan dalam eksperimen.

Akurasi kebijakan diukur menggunakan *F1-score*, yaitu rata-rata harmonik antara presisi dan *recall* keputusan akses yang dihasilkan kebijakan hasil *mining* terhadap *log* akses uji atau *holdout*. F1 didefinisikan sebagai berikut:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}},$$

dengan $\text{Precision} = \frac{TP}{TP + FP}$ dan $\text{Recall} = \frac{TP}{TP + FN}$.

Dengan *permit* sebagai kelas positif, TP (*true positive*) adalah permintaan yang seharusnya diizinkan dan diprediksi *permit*, FP (*false positive*) adalah permintaan yang seharusnya ditolak tetapi diprediksi *permit*, sedangkan FN (*false negative*) adalah permintaan yang seharusnya diizinkan tetapi diprediksi *deny*.

Kompleksitas struktural kebijakan diukur menggunakan *Weighted Structural Complexity* (WSC), yaitu jumlah pasangan atribut–nilai yang digunakan pada seluruh aturan kebijakan. WSC dirumuskan sebagai berikut:

$$WSC(\pi) = \sum_{r \in \pi} |attr(r)|,$$

dengan π menyatakan kebijakan yang dihasilkan dan $|attr(r)|$ jumlah pasangan atribut–nilai pada aturan r . Secara *default*, seluruh pasangan atribut–nilai diberi bobot seragam sebesar 1, meskipun pembobotan berbeda untuk atribut subjek, objek, atau lingkungan dapat diterapkan bila diperlukan.

Coverage mengukur proporsi permintaan akses pada *log* uji yang dicakup oleh sedikitnya satu aturan dalam kebijakan hasil *mining*, sehingga tidak bergantung pada keputusan *default* seperti *default-deny*. Metrik ini digunakan untuk menilai kelengkapan cakupan kebijakan:

$$Coverage = \frac{\text{jumlah permintaan akses yang cocok dengan } \geq 1 \text{ aturan}}{\text{total permintaan akses pada log uji}}.$$

F1, WSC, dan *coverage* secara bersama merepresentasikan kualitas kebijakan yang digunakan pada Eksperimen 1.

Jejak memori diukur sebagai konsumsi memori puncak selama eksekusi dan dilaporkan dalam dua bentuk. $MemPeak_{alloc}$ adalah puncak alokasi struktur data algoritma yang diukur menggunakan *tracemalloc* dan digunakan sebagai metrik utama, sedangkan $MemPeak_{sys}$ adalah nilai VmHWM pada tingkat sistem operasi yang merepresentasikan puncak RSS proses. Selain nilai absolut, memori dianalisis terhadap ukuran populasi virtual (NP) dan dimensi (D). Bentuk kurva penskalaan ini digunakan pada Eksperimen 2 sebagai bukti empiris pendukung H2, yaitu bahwa kebutuhan memori CoDA-EDA tidak sensitif terhadap NP dan bertumbuh secara linear terhadap D.

Waktu komputasi dibedakan menjadi T_{init} , yaitu waktu pembelajaran awal; T_{param} , yaitu waktu siklus yang hanya memperbarui parameter *Probability Vector*; dan

T_{struct} , yaitu waktu siklus yang juga melakukan pembelajaran ulang struktur setelah *drift* terdeteksi. Ketiganya diukur menggunakan *wall-clock time* pada perangkat target dan digunakan pada Eksperimen 3 untuk membandingkan efisiensi pembaruan CoDA-EDA dan iBOA.

Pada skenario *drift*, efektivitas strategi *warm-start* diukur melalui dua metrik tambahan. T_{reopt} menyatakan waktu sejak *drift* terdeteksi hingga kebijakan memenuhi kriteria konvergensi, sedangkan $\Delta Quality$ menyatakan perbedaan F1 dan WSC antara *warm-start* dan *re-mining* penuh pada titik evaluasi yang setara. Kedua metrik tersebut digunakan untuk menilai apakah *warm-start* dapat mempertahankan kualitas kebijakan dengan waktu re-optimasi yang lebih rendah.

Tabel III.11 Definisi Operasional Metrik dan Instrumen Evaluasi

No	Metrik	Simbol	Definisi/Formula	Instrumen	Hipotesis
1	Akurasi	F1	Rata-rata harmonik presisi dan recall keputusan akses terhadap log uji	Skrip evaluasi Python terhadap log uji holdout	H1
2	Kompleksitas struktural	WSC	Jumlah pasangan atribut-nilai pada seluruh aturan kebijakan	Skrip evaluasi Python	H1
3	Cakupan	Coverage	Proporsi permintaan akses yang tercakup ≥ 1 aturan	Skrip evaluasi Python	H1
4	Jejak memori	MemPeak_alloc (utama); MemPeak_sys (konteks)	Puncak alokasi struktur data algoritma; dan puncak resident set size pada tataran sistem. Dilaporkan pula sebagai kurva penskalaan terhadap NP dan D	tracemalloc (utama) + VmH-WM pada /proc/[pid]/status, satu proses terpisah per konfigurasi (Subbab 3.7)	H2

Lanjut ke halaman berikutnya

Tabel III.11 – lanjutan dari halaman sebelumnya

No	Metrik	Simbol	Definisi/Formula	Instrumen	Hipotesis
5	Waktu inisialisasi	T_init	Waktu hingga elite pertama terbentuk	Wall-clock timer pada perangkat target	H3
6	Waktu siklus parameter	T_param	Waktu satu siklus pembaruan Probability Vector saja	Wall-clock timer pada perangkat target	H3
7	Waktu siklus struktur	T_struct	Waktu satu siklus dengan pembaruan struktur dependensi	Wall-clock timer pada perangkat target	H3
8	Waktu re-optimasi	T_reopt	Waktu sejak drift terdeteksi hingga konvergensi tercapai	Wall-clock timer pada perangkat target	H4
9	Selisih kualitas warm-start	Δ Quality	Selisih F1/WSC antara warm-start dan re-mining penuh	Skrip evaluasi Python	H4

Definisi operasional dan formula pada Tabel III.11 menjadi landasan pengolahan data dari pelaksanaan demonstrasi, sekaligus menjadi rujukan bagi teknik analisis statistik yang akan dilakukan.

III.10 Teknik Analisis Data

Data sembilan metrik pada Tabel III.11 yang diperoleh dari 30 replikasi setiap kombinasi algoritma–dataset–skenario dianalisis secara statistik untuk menguji H1–H4. Uji Shapiro–Wilk digunakan sebagai pemeriksaan awal distribusi data. Namun, karena kinerja metaheuristik bersifat stokastik dan tidak selalu berdistribusi normal, analisis utama menggunakan prosedur non-parametrik, sesuai praktik evaluasi metaheuristik (Omran dkk., 2022).

Untuk Eksperimen 1 dan 2 yang membandingkan tujuh algoritma, digunakan uji Friedman sebagai uji omnibus. Jika diperoleh perbedaan signifikan pada $\alpha = 0,05$, analisis dilanjutkan dengan uji Wilcoxon *signed-rank* antara CoDA-EDA dan setiap *baseline*, disertai koreksi Holm–Bonferroni. Perbandingan dilakukan pada blok

yang sama berdasarkan konfigurasi dataset, tingkat dependensi, dan replikasi yang berpasangan. Koreksi diterapkan per keluarga pengujian dalam satu dataset, yaitu enam perbandingan CoDA-EDA terhadap enam *baseline*, dan tidak digabungkan lintas dataset.

Jumlah replikasi juga menentukan resolusi uji statistik. Pada uji Wilcoxon *signed-rank* dua sisi tanpa *tie* atau selisih nol, nilai- p minimum untuk n pasangan adalah $2/2^n$. Dengan lima pasangan, nilai minimum tersebut adalah 0,0625 sehingga tidak dapat mencapai $\alpha = 0,05$, sedangkan enam pasangan secara prinsip sudah memungkinkan. Oleh karena itu, 30 replikasi digunakan untuk menyediakan jumlah pengamatan yang memadai sekaligus menangkap variabilitas stokastik. *Seed* pembangkitan dataset dan *seed* algoritma dipisahkan agar variasi data tidak tercampur dengan variasi proses optimasi.

Eksperimen 3 dan 4 masing-masing membandingkan dua kondisi, yaitu CoDA-EDA terhadap iBOA serta *warm-start* terhadap *re-mining* penuh. Karena hanya terdapat dua kondisi, keduanya dianalisis langsung menggunakan uji Wilcoxon *signed-rank* berpasangan tanpa uji omnibus.

Taraf signifikansi ditetapkan sebesar $\alpha = 0,05$. Selain nilai- p , hasil dilengkapi ukuran efek non-parametrik, yaitu *rank-biserial correlation* untuk uji Wilcoxon (Peres, 2026). Seluruh analisis dilakukan menggunakan pustaka SciPy pada Python.

Kualitas kebijakan dihitung terhadap *log* yang digunakan dalam proses *mining* sehingga F1, WSC, dan *coverage* diinterpretasikan sebagai fidelitas rekonstruksi kebijakan, bukan kemampuan generalisasi prediktif terhadap permintaan yang belum diamati. Interpretasi ini sesuai dengan tujuan *ABAC policy mining*, yaitu memperoleh kebijakan yang merepresentasikan pola keputusan pada *log* akses.

Untuk hipotesis yang menyatakan kualitas sebanding atau non-inferior, pengujian tidak hanya didasarkan pada ketidakberhasilan menolak hipotesis nol. Margin non-inferioritas ditetapkan sebelum analisis, yaitu $\delta = 0,02$ untuk F1 serta margin praktis yang ditetapkan untuk WSC. Non-inferioritas dievaluasi menggunakan selang kepercayaan 95% terhadap selisih antarkondisi. Untuk F1, CoDA-EDA dinyatakan non-inferior apabila batas bawah selisih $F1_{\text{CoDA}} - F1_{\text{baseline}}$ berada di atas $-\delta$. Untuk WSC, karena nilai yang lebih rendah lebih baik, arah kriteria disesuaikan dengan definisi selisih yang digunakan.

Rancangan analisis statistik H1–H4 dirangkum pada Tabel III.12 berdasarkan eksperimen sumber data, kondisi yang dibandingkan, prosedur pengujian, dan ukuran efek yang digunakan.

Tabel III.12 Rancangan Uji Statistik per Hipotesis

Hipotesis	Eksperimen	Kondisi yang dibandingkan	Prosedur uji	Ukuran efek
H1 — Kualitas kebijakan	Eksperimen 1	Tujuh kondisi algoritma sekaligus, per dataset $\times$ tingkat korelasi	Friedman (omnibus) $\rightarrow$ post-hoc Wilcoxon signed-rank CoDA-EDA vs tiap baseline, koreksi Holm-Bonferroni	Rank-biserial
H2 — Jejak memori	Eksperimen 2	Tujuh kondisi algoritma sekaligus, per dataset	Friedman (omnibus) $\rightarrow$ post-hoc Wilcoxon signed-rank berkoreksi Holm-Bonferroni	Rank-biserial
H3 — Waktu pembaruan	Eksperimen 3	Dua kondisi (CoDA-EDA warm vs iBOA; serta warm vs always)	Wilcoxon signed-rank berpasangan langsung (tanpa omnibus)	Rank-biserial
H4 — Warm-start	Eksperimen 4	Dua kondisi (warm-start vs re-mining penuh)	Wilcoxon signed-rank berpasangan langsung (tanpa omnibus)	Rank-biserial

Tabel III.12 menjadi acuan pelaksanaan uji statistik atas hasil *Demonstration*. Dengan tersusunnya rancangan analisis ini, seluruh instrumen metodologi telah lengkap.

III.11 Ketertelusuran Rancangan Penelitian

Setelah variabel, rancangan algoritma, konstruksi dataset, desain eksperimen, metrik, dan teknik analisis ditetapkan, Tabel III.13 merangkum ketelusuran dari akar masalah, kebutuhan artefak, keputusan rancangan, hingga bukti evaluasi. Matriks ini memastikan setiap komponen CoDA-EDA memiliki dasar permasalahan yang jelas dan setiap klaim penelitian memiliki mekanisme pengujian yang sesuai.

Tabel III.13 Desain artefak dan pemetaan kebutuhan

ID	Masalah / akar penyebab	Kebutuhan artefak	Keputusan desain	Bukti evaluasi
M1	Populasi eksplisit membebani memori	Memori tidak bergantung pada NP	PV tanpa populasi eksplisit	Kurva memori vs NP ; analisis & regresi vs D (Gambar IV.x)
M2	Model univariat mengabaikan dependensi	Menangkap dependensi dengan biaya terbatas	Pohon Chow-Liu orde satu ($O(D)$ via Prim)	Perbandingan terhadap baseline compact univariat (compact-GA/PSO); kualitas per dimensi/korelasi (H1)
M3	Pembaruan struktur berulang mahal	Struktur diperbarui hanya saat diperlukan	Pemisahan pembaruan parameter–struktur + deteksi drift (JSD, τ)	Ablasi mekanisme pembaruan (warm vs always); trigger rate; overhead deteksi (H3)
M4	Kebijakan berubah setelah drift	Re-optimasi tanpa mining penuh	Warm-start dari model sebelumnya	Ablasi warm-start (warm vs cold); waktu menuju target kualitas + uji non-inferioritas (H4)
M5	Benchmark berpotensi tidak adil	Representasi, fitness, anggaran, operator setara	Kontrol eksperimen bersama (kesetaraan implementasi)	Tabel konfigurasi, sanity check, kontrol kesetaraan

Dengan demikian, alur argumentasi dari Bab I hingga Bab IV dapat ditelusuri secara konsisten. Setiap masalah pada konteks *edge-IoT* dihubungkan dengan mekanisme rancangan yang menjawabnya serta eksperimen yang digunakan untuk mengevaluasinya.

III.12 Studi Pendahuluan

Studi pendahuluan diawali dengan analisis karakteristik dataset sebelum eksperimen utama dilakukan. Analisis mencakup dimensi ruang solusi D , jumlah *permit* unik sebagai indikator potensi duplikasi, rasio *permit*, serta rantai derau *Jensen–Shannon divergence* pada kondisi stasioner untuk menilai kelayakan mekanisme pemicu *drift*. Dengan demikian, pemilihan dan peran setiap dataset ditetapkan berdasarkan karakteristik empirisnya. Lima dataset ABAC Lab dan dataset validasi eksternal *Smart Building IoT* diprofilkan menggunakan prosedur yang sama, dengan hasil dirangkum pada Tabel III.14.

Tabel III.14 Karakteristik Dataset dan Kecocokannya dengan Hipotesis (Analisis Pra-Eksperimen)

Dataset	Dimensi D	Permit unik (risiko duplikasi)	Rasio permit	Lantai derau JSD (H3)	Kecocokan hipotesis
workforce	436	14.513 (rendah)	0,30	0,0066 (layak)	H1 baik; H2/H4 baik
e-document	1.907	32.961 (rendah)	0,30	0,0325 (tak layak)	H1 baik; H3 terbatas (rantai derau $> \tau$); H2/H4 baik
healthcare	46	14 (TINGGI)	0,30	0,0022 (layak)	H1 tumpul (duplikasi); H2/H3/H4 baik
project-management	51	17 (TINGGI)	0,30	0,0010 (layak)	H1 tumpul (duplikasi); H2/H3/H4 baik
university	54	90 (TINGGI)	0,30	0,0026 (layak)	H1 tumpul (duplikasi); H2/H3/H4 baik
Smart Building IoT*	428*	5.729 (rendah)	0,75	0,0065 (layak)	berat-permit (sepele $F1=0,857$); H1 vs garis sepele; H2/H3/H4 baik

Tanda (*) pada Tabel III.14 menunjukkan dimensi efektif *Smart Building IoT* setelah subsampel terstratifikasi sebanyak 8.000 entri dan penggabungan nilai dengan *support* rendah. Dataset asal memuat sekitar 5,7 juta permintaan, terdiri atas 4.273.814 *permit* dan 1.429.088 *deny*. Rasio *permit* sebesar 0,30 pada dataset ABAC Lab merupakan parameter konstruksi, sedangkan rasio sekitar 0,75 pada *Smart Building IoT* merupakan karakteristik bawaan dataset.

Tabel III.14 menunjukkan bahwa keenam dataset memiliki karakteristik berbeda dari sisi dimensi D , jumlah *permit* unik, tingkat duplikasi, rasio *permit*, dan rantai derau *Jensen-Shannon divergence* (JSD) pada kondisi stasioner. Perbedaan tersebut digunakan untuk menentukan peran setiap dataset dalam pengujian hipotesis sehingga interpretasi hasil mempertimbangkan karakteristik data, bukan hanya metrik kinerja algoritma.

Healthcare, *project-management*, dan *university* merupakan dataset berdimensi rendah dengan $D=46$, 51, dan 54 serta masing-masing hanya memiliki 14, 17, dan 90 *permit* unik. Tingginya pengulangan pola membuat ketiganya kurang informatif sebagai bukti utama H1 karena dapat memperkecil perbedaan kualitas antaralgoritma. Namun, dataset tersebut tetap relevan untuk H2 serta H3–H4. Rantai derau JSD stasionernya rendah, yaitu 0,0022, 0,0010, dan 0,0026, sehingga berada di bawah ambang $\tau = 0,01$.

Workforce memiliki dimensi menengah, $D = 436$, dengan 14.513 *permit* unik sehingga menyediakan keragaman pola yang lebih memadai untuk evaluasi kualitas kebijakan. Rantai derau JSD sebesar 0,0066 juga berada di bawah $\tau = 0,01$, sehingga kondisi stasioner dapat dibedakan dari perubahan distribusi yang cukup besar untuk memicu pembaruan struktur. Karena itu, *workforce* digunakan sebagai dataset utama untuk H1 serta pengujian H3 dan H4.

E-document merupakan dataset dengan dimensi tertinggi, $D = 1.907$, dan memiliki 32.961 *permit* unik. Karakteristik ini menjadikannya kasus utama untuk menguji manfaat pemodelan dependensi pada ruang pencarian besar. Pada studi pendahuluan, CoDA-EDA memperoleh *F1-score* 0,452, sedangkan *compact-GA* dan *compact-PSO* masing-masing memperoleh 0,329 dan 0,260. Sebaliknya, pada *workforce*, nilainya masing-masing 0,889, 0,900, dan 0,893. Pola awal ini menunjukkan bahwa manfaat pemodelan dependensi lebih terlihat pada dimensi tinggi, sedangkan pada dimensi menengah perbedaannya relatif kecil.

Dimensi tinggi *e-document* juga meningkatkan lantai derau JSD. Pada jendela 800 entri, nilainya mencapai 0,0325, sedangkan pada 2.000 entri turun menjadi sekitar 0,013–0,016, tetapi masih melampaui $\tau = 0,01$. Akibatnya, pemicu pembaruan struktur dapat aktif pada kondisi yang sebenarnya mendekati stasioner. Karena itu, *e-document* tetap kuat untuk H1, H2, dan H4, sedangkan hasil H3 pada dataset ini perlu ditafsirkan dengan mempertimbangkan keterbatasan penggunaan ambang JSD tetap.

Smart Building IoT digunakan sebagai validasi eksternal. Setelah subsampel dan penggabungan nilai dengan *support* rendah, dataset memiliki dimensi efektif sekitar $D = 428,5.729$ *permit* unik, dan rasio *permit* sekitar 0,75. Berbeda dari ABAC Lab, dependensi dan *drift* pada dataset ini tidak dikendalikan oleh *generator* penelitian. Nilai validasinya terletak pada proses pembangkitan yang independen. Lantai derau JSD sebesar 0,0065 juga berada di bawah τ , sehingga dataset tetap dapat digunakan untuk mengamati perilaku mekanisme pembaruan struktur.

Secara keseluruhan, profil dataset menunjukkan bahwa kekuatan bukti H1 meningkat pada dataset dengan ruang solusi dan keragaman pola yang lebih besar. Sebaliknya, kelayakan H3 dipengaruhi oleh hubungan antara lantai derau JSD dan ambang τ . Dataset dengan lantai derau di bawah ambang memungkinkan mekanisme pembaruan struktur tetap tidak aktif pada kondisi stasioner, sedangkan *e-document* menunjukkan keterbatasan pendekatan ambang tetap ketika jumlah sel kontingensi relatif besar terhadap ukuran jendela.

Analisis karakteristik dataset dilakukan menggunakan instrumen terpisah dari algoritma yang diuji sehingga tidak memengaruhi anggaran evaluasi maupun kesetaraan eksperimen. Hasilnya digunakan sebagai penyaring kecocokan pra-eksperimen sebelum studi pendahuluan algoritmik dilakukan.

Studi pendahuluan kemudian dijalankan pada perangkat target untuk memverifikasi implementasi, mengkalibrasi parameter, dan mengidentifikasi kebutuhan penyempurnaan hipotesis. Tahap ini bersifat eksploratori, sedangkan eksperimen final pada Bab IV menggunakan *seed* pembangkitan dataset dan *seed* algoritma yang berbeda agar data eksplorasi tidak digunakan kembali untuk pengujian hipotesis.

Studi pendahuluan membandingkan CoDA-EDA dengan *compact-GA* dan *compact-PSO* pada *workforce* dan *e-document* menggunakan representasi biner, fungsi *fitness*, operator *repair*, pembenihan, serta anggaran evaluasi yang sama. Empat *seed* dataset masing-masing dijalankan dengan sepuluh replikasi algoritma sehingga menghasilkan 40 replikasi per kondisi. Anggaran evaluasi disesuaikan terhadap dimensi, yaitu

12.001 evaluasi untuk *workforce* dan 52.001 untuk *e-document*. Analisis menggunakan uji Friedman sebagai uji omnibus, dilanjutkan Wilcoxon *signed-rank* dengan koreksi Holm–Bonferroni jika signifikan, serta pelaporan *rank-biserial correlation* sebagai ukuran efek.

Tabel III.15 Hasil Studi Pendahuluan: Kualitas Kebijakan (F-score) pada Dua Dataset Utama

Dataset	Korelasi	CoDA-EDA	compact-GA	compact-PSO	Friedman	Post-hoc CoDA-EDA vs compact-GA
Workforce ($D = 436$)	Rendah	0,8354	0,8409	0,8123	$p = 0,025$	tidak signifikan ($p = 0,545$)
Workforce ($D = 436$)	Sedang	0,8644	0,8486	0,8471	$p = 0,592$	tidak dilanjutkan (omnibus tidak signifikan)
Workforce ($D = 436$)	Tinggi	0,9017	0,8852	0,8769	$p = 0,161$	tidak dilanjutkan (omnibus tidak signifikan)
E-document ($D = 1.907$)	Rendah	0,3754	0,2071	0,2395	$p = 8,5 \times 10^{-7}$	signifikan; $p_{\text{adj}} = 1,1 \times 10^{-4}$; $r = +0,751$
E-document ($D = 1.907$)	Sedang	0,3534	0,2308	0,2685	$p = 1,9 \times 10^{-4}$	signifikan; $p_{\text{adj}} = 4,9 \times 10^{-4}$; $r = +0,685$
E-document ($D = 1.907$)	Tinggi	0,4160	0,2944	0,3067	$p = 1,8 \times 10^{-3}$	signifikan; $p_{\text{adj}} = 7,9 \times 10^{-4}$; $r = +0,663$

Hasil pada Tabel III.15 menunjukkan bahwa rumusan awal H1 tidak didukung. Pada *workforce*, keunggulan CoDA-EDA atas *compact-GA* hanya sekitar +0,016 pada korelasi sedang dan tinggi serta tidak signifikan ($p = 0,197$ dan $p = 0,111$). Tren peningkatan keunggulan seiring korelasi juga tidak signifikan ($p = 0,073$). Sebaliknya, pada *e-document*, CoDA-EDA unggul signifikan pada seluruh tingkat korelasi dengan selisih *F1-score* +0,12 hingga +0,17 dan ukuran efek besar $r =$

0,66–0,75, tetapi keunggulan tersebut tidak meningkat seiring korelasi ($p = 0,506$). Dengan demikian, dugaan bahwa keunggulan CoDA-EDA terutama muncul pada korelasi tinggi tidak memperoleh dukungan empiris.

Perbedaan yang lebih konsisten terlihat pada dimensi ruang solusi. Pada *workforce* ($D = 436$), *compact-GA* telah mencapai *F1-score* 0,885 pada korelasi tinggi sehingga ruang peningkatannya terbatas. Pada *e-document* ($D = 1.907$), *F1-score compact-GA* turun menjadi 0,294, sedangkan CoDA-EDA tetap mencapai 0,416. Pola ini mengindikasikan bahwa manfaat pemodelan dependensi lebih menonjol ketika dimensi ruang pencarian meningkat. Berdasarkan temuan tersebut, H1 direvisi dengan memasukkan D sebagai faktor penelitian.

Studi pendahuluan juga memberikan dua temuan pendukung. Pertama, pada *workforce*, peningkatan *F1-score* dari korelasi rendah ke tinggi mencapai +0,066 pada CoDA-EDA ($p = 1,6 \times 10^{-6}$; $r = 0,873$), dibandingkan +0,044 pada *compact-GA* ($p = 1,0 \times 10^{-3}$; $r = 0,598$). Hal ini menunjukkan bahwa CoDA-EDA tetap lebih responsif terhadap struktur dependensi meskipun rumusan H1 awal tidak terdukung. Kedua, CoDA-EDA menghasilkan kebijakan lebih ringkas, dengan WSC rata-rata 23,8 dibandingkan 28,9 pada *compact-GA* dan 38,2 pada *compact-PSO* pada *workforce*.

Tahap pendahuluan berikutnya memperluas pengujian ke tujuh algoritma, pengukuran memori, simulasi pembaruan lintas jendela, eksplorasi cakupan, dan validasi eksternal. Hasilnya mendorong tiga penyesuaian utama: H1 difokuskan pada pengaruh dimensi dan perbandingan terhadap keluarga *compact* univariat, H3 diberi kondisi terkait tingkat *drift*, dan H4 dirumuskan sebagai klaim non-inferioritas kualitas.

Studi pendahuluan juga menghasilkan beberapa ketentuan teknis untuk eksperimen final. Ambang deteksi *drift* ditetapkan pada $\tau = 0,01$ berdasarkan pemisahan antara sinyal JSD dan lantai derau; bobot w_{fp} pada fungsi *fitness* dikalibrasi melalui uji monotonisitas; serta perhitungan *mutual information* dioptimalkan menggunakan representasi *bit-packed*, yang menurunkan puncak alokasi memori CoDA-EDA dari 2.195 KB menjadi 450 KB tanpa mengubah struktur dependensi yang dihasilkan maupun waktu komputasinya.

Bab IV Hasil dan Pembahasan

Bab ini melaporkan hasil aktivitas *Demonstration* dan *Evaluation* melalui pengujian empiris Hipotesis H1–H4 pada lima dataset utama serta validasi eksternal menggunakan *Smart Building IoT*. Seluruh eksperimen dijalankan pada Raspberry Pi 5. Pengujian H1 menggunakan tiga *seed* dataset dengan lima replikasi algoritma untuk setiap kombinasi dataset dan tingkat dependensi, sedangkan anggaran evaluasi disesuaikan dengan dimensi ruang solusi. Hasil disajikan berdasarkan hipotesis, meliputi kualitas kebijakan, jejak memori, efisiensi pembaruan dan *warm-start*, validasi eksternal, serta sintesis temuan.

IV.1 Kualitas Kebijakan

Hipotesis H1 menyatakan bahwa CoDA-EDA menghasilkan kualitas kebijakan yang lebih tinggi daripada keluarga *compact* univariat, dengan manfaat yang meningkat pada ruang solusi berdimensi besar, serta tetap kompetitif terhadap EDA yang memodelkan dependensi. Kualitas dievaluasi menggunakan *F1-score*, *coverage*, dan WSC melalui uji Friedman, dilanjutkan Wilcoxon *signed-rank* dengan koreksi Holm–Bonferroni. Rerata *F1-score* seluruh dataset disajikan pada Tabel IV.1.

Tabel IV.1 Rerata *F-score* Ketujuh Algoritma pada Lima Dataset ABAC Lab

Algoritma	workforce (436)	e- document (1.907)	healthcare (46)	project- mgmt (51)	university (54)
CoDA-EDA	0,889	0,452	0,779	0,898	0,830
compact-GA	0,900	0,329	0,765	0,883	0,822
compact-PSO	0,893	0,260	0,801	0,902	0,829
iBOA	0,888	0,452	0,802	0,909	0,824
BOA	0,832	0,633	0,835	0,927	0,831
ACO	0,870	0,661	0,761	0,886	0,835
UBK	0,853	0,630	0,782	0,853	0,869

Pada dua dataset utama, uji Friedman menunjukkan perbedaan signifikan (*workforce*: $p = 2,0 \times 10^{-14}$; *e-document*: $p = 1,6 \times 10^{-42}$). Pada *workforce* ($D = 436$), CoDA-EDA memperoleh *F1-score* 0,889 dan secara signifikan mengungguli BOA

($r = +0,56$) serta UBK ($r = +0,33$), tetapi tidak berbeda signifikan dari *compact-GA*, *compact-PSO*, iBOA, dan ACO. Dengan demikian, CoDA-EDA berada pada kelompok berkinerja tinggi tanpa menunjukkan dominasi.

Pada *e-document* ($D = 1.907$), CoDA-EDA memperoleh *F1-score* 0,452, setara dengan iBOA (0,452), serta mengungguli *compact-GA* dan *compact-PSO* dengan ukuran efek masing-masing ($r = +0,42$) dan ($r = +0,60$). Namun, CoDA-EDA masih tertinggal signifikan dari BOA, ACO, dan UBK dengan ukuran efek besar ($r = -0,87$ hingga $-0,91$). Tiga dataset berdimensi rendah memberikan pola yang lebih datar akibat jumlah permit unik yang terbatas dan tingkat duplikasi tinggi. CoDA-EDA umumnya setara dengan *baseline*, kecuali mengungguli UBK sebesar 0,045 pada *project-management* dan tertinggal 0,039 pada *university*. Karena itu, dataset kecil digunakan sebagai konteks pelengkap, bukan bukti utama H1.

Tabel IV.2 Rincian Kualitas H1 pada *e-document* ($D = 1.907$; rerata lintas korelasi; post-hoc vs CoDA-EDA)

Algoritma	F-score	ΔF vs CoDA	Cakupan	Δ cakupan vs CoDA	WSC	Posisi terhadap CoDA-EDA (post-hoc Holm)
ACO	0,661	+0,209	0,572	+0,324	16,9	Unggul signifikan ($r = -0,91$)
BOA	0,633	+0,181	0,624	+0,376	11,2	Unggul signifikan ($r = -0,87$)
UBK	0,630	+0,178	0,468	+0,220	6,2	Unggul signifikan ($r = -0,91$)
iBOA	0,452	0,000	0,246	-0,002	13,8	Setara (n.s.)
CoDA-EDA	0,452	—	0,248	—	11,8	— (acuan)
compact-GA	0,329	-0,123	0,156	-0,092	10,2	CoDA-EDA unggul ($r = +0,42$)
compact-PSO	0,260	-0,192	0,116	-0,132	11,0	CoDA-EDA unggul ($r = +0,60$)

Perincian *e-document* pada Tabel IV.2 menunjukkan bahwa kelemahan CoDA-EDA terutama berasal dari *coverage*, bukan presisi. Dengan batas empat aturan, BOA,

ACO, dan UBK mampu mencakup sekitar 47–62% permintaan, sedangkan CoDA-EDA dan iBOA menghasilkan aturan berpresisi tinggi tetapi hanya mencakup sekitar 25%. Pengujian bobot *false positive* (w_{fp}) hampir tidak mengubah *coverage* CoDA-EDA, yaitu dari 0,186 menjadi 0,187, sehingga keterbatasan tersebut bukan terutama disebabkan oleh pembobotan presisi.

Sebaliknya, peningkatan anggaran jumlah aturan memperbaiki kualitas secara nyata. Ketika jumlah maksimum aturan dinaikkan dari empat menjadi delapan dan dua belas, *F1-score* CoDA-EDA meningkat dari 0,362 menjadi 0,571 dan jaraknya terhadap ACO menyempit dari $-0,301$ menjadi $-0,100$ (Tabel IV.3). Pada konfigurasi dua belas aturan, CoDA-EDA tetap setara dengan iBOA ($p_{adj} = 0,52$) dan kesenjangan terhadap ACO serta UBK berkurang. Perbaikan tersebut disertai kenaikan waktu komputasi dari 97 menjadi 136 detik dan WSC dari 9,6 menjadi 26,8, sehingga menunjukkan adanya pertukaran antara kualitas, waktu, dan kompleksitas kebijakan.

Tabel IV.3 Efek Anggaran Jumlah Aturan terhadap Kualitas CoDA-EDA pada *e-document*

Jumlah aturan	F-score	Cakupan	Δ cakupan vs 4-aturan	WSC	Waktu	Posisi terhadap baseline (post-hoc Holm)
4 (baku Tabel IV.2)	0,362	0,186	—	9,6	97 s	Kalah dari ACO, UBK, BOA; setara iBOA
8	0,571	0,391	+0,205	18,8	117 s	Kalah dari ACO, UBK; setara BOA & iBOA
12	0,568	0,428	+0,242	26,8	136 s	Kalah dari ACO, UBK; setara BOA & iBOA

Hasil utama H1 menunjukkan bahwa manfaat CoDA-EDA terhadap keluarga *compact* univariat lebih berkaitan dengan dimensi ruang solusi daripada tingkat dependensi yang disuntikkan. Pada *workforce*, CoDA-EDA (0,889) secara praktis setara dengan *compact-GA* (0,900) dan *compact-PSO* (0,893), sehingga selisih terhadap rerata keduanya sekitar $(-0,008)$. Sebaliknya, pada *e-document*, CoDA-EDA (0,452) berada di atas *compact-GA* (0,329) dan *compact-PSO* (0,260), dengan selisih terhadap rerata keduanya sekitar $(+0,158)$.

Analisis berdasarkan tingkat dependensi menunjukkan pola serupa. Pada *e-document*, keunggulan CoDA-EDA terhadap rerata *compact-GA* dan *compact-PSO* berada sekitar $+0,15$ hingga $+0,17$ pada tingkat rendah, sedang, maupun tinggi. Dengan demikian, peningkatan dependensi tidak secara konsisten memperbesar keunggulan CoDA-EDA. Interpretasi yang lebih sesuai adalah bahwa pada ruang berdimensi tinggi, model univariat harus membangkitkan kombinasi berdasarkan probabilitas marginal secara independen, sedangkan CoDA-EDA memperoleh informasi tambahan dari dependensi orde satu yang dipelajari melalui *mutual information* dan direpresentasikan dalam pohon Chow–Liu.

Temuan tersebut tidak menunjukkan bahwa CoDA-EDA merupakan algoritma dengan kualitas penambangan statis tertinggi pada semua kondisi. Pada *e-document*, CoDA-EDA tetap tertinggal dari beberapa algoritma berbasis populasi, terutama karena menghasilkan aturan yang lebih sempit. Oleh karena itu, H1 didukung secara bersyarat: keunggulan CoDA-EDA terutama terlihat terhadap keluarga *compact* univariat pada ruang solusi berdimensi tinggi, bukan sebagai dominasi terhadap seluruh algoritma pembandingan.

Validasi eksternal mendukung batas interpretasi tersebut. Pada *Smart Building IoT* dengan dimensi efektif sekitar $D = 428$, CoDA-EDA memperoleh *F1-score* 0,938, mendekati iBOA (0,943), tetapi sedikit di bawah *compact-GA* (0,961) dan *compact-PSO* (0,964). Karena dimensinya sebanding dengan *workforce*, hasil ini konsisten dengan temuan bahwa manfaat pemodelan dependensi CoDA-EDA belum terlihat jelas pada ruang solusi berdimensi menengah.

Secara keseluruhan, struktur Chow–Liu memberikan informasi dependensi yang tidak tersedia pada model *compact* univariat dan memberikan manfaat nyata ketika dimensi ruang pencarian meningkat. Namun, representasi tanpa populasi eksplisit membatasi keragaman eksplorasi dibandingkan beberapa algoritma berbasis populasi. Posisi CoDA-EDA karena itu bukan sebagai penambang kebijakan statis yang selalu menghasilkan *F1-score* tertinggi, melainkan sebagai algoritma *dependency-aware* yang mempertahankan kualitas kompetitif sambil menyediakan efisiensi memori dan mekanisme pembaruan yang menjadi fokus utama penelitian.

Sebagai ilustrasi keluaran konkret, Tabel IV.4 menyajikan contoh aturan CoDA-EDA pada *workforce* dan *Smart Building IoT* beserta aturan ACO sebagai pembandingan. CoDA-EDA cenderung menghasilkan aturan berpresisi tinggi, misalnya hanya tiga *FP* dari 405 kasus pada aturan *modify workforce* dan tanpa *FP* pada contoh *Smart Building IoT*. Sebaliknya, ACO menghasilkan aturan dengan cakupan lebih luas tetapi dapat disertai *FP* yang lebih tinggi, hingga 1.297 pada contoh yang ditampilkan.

Pola ini konsisten dengan pertukaran presisi–cakupan pada Tabel IV.2. Pada *Smart Building IoT*, beberapa aturan CoDA-EDA juga memiliki WSC tinggi, dengan total mencapai 276, karena enumerasi panjang nilai atribut pengenalan seperti *uname* dan *rname*.

Tabel IV.4 Contoh Kebijakan Hasil Mining (cuplikan aturan terbaca)

Sumber / algoritma	Metrik	Contoh aturan (cuplikan)	Cakupan aturan
workforce / CoDA-EDA	F1 0,885; WSC 20; 5 aturan	IF contractStatus=active AND action=view THEN permit	TP=1.291; FP=339
workforce / CoDA-EDA	(aturan berpresisi)	IF provider in {externalHelpdeskSupplier, telco} AND tenant=telco AND associatedContract in {contract001,003,005} AND action=modify THEN permit	TP=402; FP=3
workforce / ACO	F1 0,820; WSC 19; 5 aturan	IF tenantType=primary AND action in {modify,view} THEN permit	TP=2.879; FP=1.297 (luas, presisi rendah)
Smart Building IoT / CoDA-EDA	F1 0,998; WSC 276; 5 aturan	IF type in {TV, Security Cameras, Smart locks} AND action in {arm, control} THEN permit	TP=3.417; FP=0

IV.2 Jejak Memori

Hipotesis H2 menyatakan bahwa jejak memori CoDA-EDA tidak bertumbuh terhadap ukuran populasi virtual (NP), berbeda dari algoritma yang menyimpan populasi eksplisit. Pengujian dilakukan pada $NP \in 25, 50, 100, 200, 400$ menggunakan $MemPeak_{alloc}$ dari *tracemalloc*, dengan setiap konfigurasi dijalankan dalam proses terpisah. Hasil rerata lima dataset disajikan pada Tabel IV.5.

Tabel IV.5 Penskalaan Jejak Memori terhadap Ukuran Populasi NP (rerata lintas lima dataset)

Algoritma	alloc NP=25 (KB)	alloc NP=400 (KB)	Rasio 400/25	Keterangan
CoDA-EDA	492	492	$1,00\times$	DATAR — memori tak bergantung NP ($O(D)$)
compact-GA	169	103	$0,77\times$	Datar (compact univariat)
compact-PSO	172	106	$0,76\times$	Datar (compact univariat)
iBOA	190	134	$0,82\times$	Datar (statistik inkremental)
ACO	212	2.036	$6,22\times$	Tumbuh — populasi/feromon
BOA	349	4.028	$7,68\times$	Tumbuh — populasi + jaringan Bayes
UBK	387	5.169	$9,20\times$	Tumbuh — populasi swarm

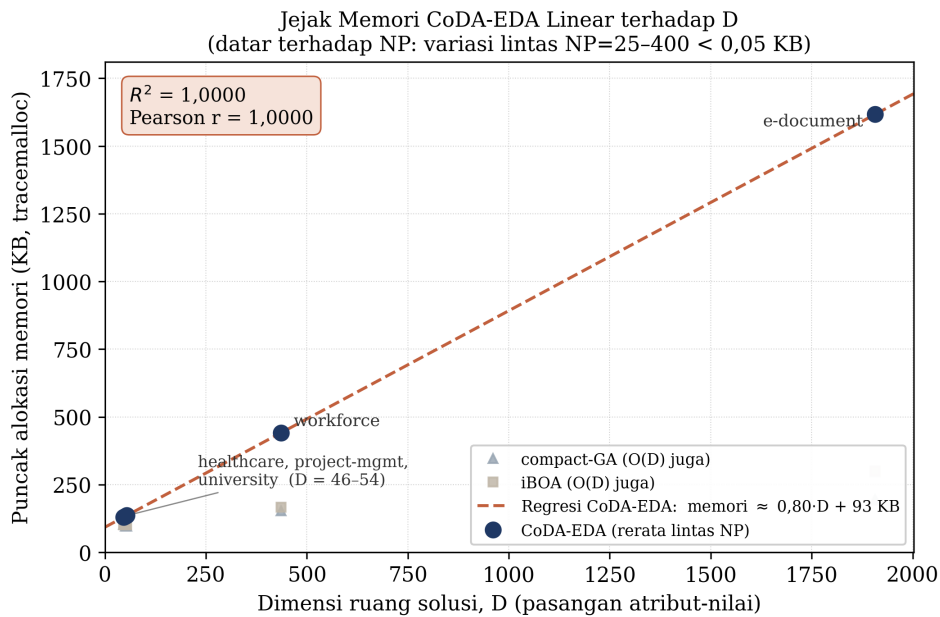
Pola yang diperoleh konsisten dengan mekanisme penyimpanan masing-masing algoritma. Ketika NP dinaikkan enam belas kali dari 25 menjadi 400, puncak alokasi CoDA-EDA tetap sekitar 492 KB dengan rasio $R_{NP} = 1,00$. Hal ini terjadi karena NP hanya menentukan granularitas pembaruan probabilistik, bukan jumlah kandidat yang disimpan. Sebaliknya, ACO, BOA, dan UBK menyimpan struktur yang berkaitan dengan populasi eksplisit sehingga puncak alokasinya meningkat masing-masing 6, 22, 7, 68, dan 9, 20 kali.

Keluarga *compact* dan iBOA menunjukkan pola serupa dengan CoDA-EDA, dengan R_{NP} masing-masing sekitar 0,76–0,77 dan 0,82. Nilai di bawah 1 mencerminkan variasi pengukuran, tetapi secara keseluruhan menunjukkan tidak adanya pertumbuhan sistematis terhadap NP . Dengan demikian, hasil empiris mendukung perbedaan antara algoritma tanpa populasi eksplisit dan algoritma yang kebutuhan memorinya bertambah ketika ukuran populasi diperbesar.

Sebagai bukti pelengkap terhadap ketergantungan memori pada dimensi, Gambar IV.1 memetakan $MemPeak_{alloc}$ CoDA-EDA terhadap D pada lima dataset. Regresi menghasilkan hubungan sekitar $MemPeak_{alloc} \approx 0,80D + 93$ KB, dengan $R^2 =$

1,000 dan korelasi Pearson $r = 1,000$, sedangkan variasi memori terhadap NP berada di bawah 0,05 KB. Hasil ini konsisten dengan analisis *by-construction* bahwa kebutuhan memori CoDA-EDA bertumbuh linear terhadap D dan tidak bergantung pada NP . Namun, karena lima titik D berasal dari dataset berbeda dan bukan dari *sweep* dimensi terkontrol pada dataset yang sama, hasil tersebut diperlakukan sebagai validasi empiris pendukung, bukan pembuktian asimtotik tersendiri.

Secara absolut, CoDA-EDA menggunakan memori lebih besar daripada *compact-GA/compact-PSO* (103-172 KB) dan *iBOA* (134–190 KB), dengan puncak sekitar 492 KB akibat tambahan struktur dependensi. Namun, fokus H2 adalah perilaku penskalaan, bukan penggunaan memori terendah pada satu konfigurasi. CoDA-EDA mempertahankan pemodelan dependensi tanpa mengalami pertumbuhan memori terhadap NP , sedangkan algoritma berpopulasi eksplisit menunjukkan peningkatan 6, 22–9, 20 kali pada rentang pengujian.



Gambar IV.1 Jejak Memori CoDA-EDA Linear Terhadap Dimensi Ruang Solusi D

Dengan demikian, H2 didukung pada konfigurasi yang diuji. CoDA-EDA mempertahankan jejak memori yang stabil ketika NP dinaikkan, sekaligus menunjukkan pola pertumbuhan terhadap D yang konsisten dengan kompleksitas memori $O(D)$. Hasil ini menunjukkan bahwa pemodelan dependensi berorde terbatas dapat ditambahkan ke mekanisme *compact* tanpa menghilangkan sifat utamanya, yaitu tidak menyimpan populasi virtual secara eksplisit.

IV.3 Efisiensi Pembaruan Kebijakan

H3 dan H4 diuji bersama karena keduanya berkaitan dengan siklus pembaruan kebijakan. H3 menguji apakah pemisahan pembaruan struktur dan parameter dapat mengurangi waktu komputasi tanpa menurunkan kualitas secara signifikan, dengan efek yang bergantung pada tingkat *drift*. H4 menguji apakah strategi *warm-start* dapat mencapai kualitas yang sekurang-kurangnya setara dengan *re-mining* penuh dalam waktu lebih singkat.

Pengujian dilakukan dengan membagi *log* menjadi lima jendela berurutan. Struktur Chow–Liu dipelajari ulang hanya ketika *Jensen–Shannon divergence* melampaui ambang $\tau = 0,01$. Empat kondisi dibandingkan, yaitu *warm* (pembaruan struktur berbasis *drift*), *always* (struktur diperbarui pada setiap jendela), *cold* (*re-mining* penuh), dan iBOA *cold*. Ringkasan hasil disajikan pada Tabel IV.6.

Tabel IV.6 Hasil Pengujian H3 dan H4 pada Lima Dataset (rerata; warm/always/cold)

Dataset / skenario	Pemicu relearn	warm (s)	always (s)	cold (s)	Vonis
workforce / stasioner	0 dari 4	3,46	4,58	9,05	H3 TERDUKUNG ($p = 6 \times 10^{-5}$); H4 2,6 $\times$, F1 +0,095
workforce / drift sedang	4 dari 4	4,52	4,53	8,95	H3 tak berlaku (struktur memang perlu); H4 2,0 $\times$, F1 +0,061
e-document / stasioner	4 dari 4	22,3	22,3	28,2	H3 tak dapat ditunjukkan (lantai derau $> \tau$); H4 1,27 $\times$, F1 +0,102
project- management / stasioner	0 dari 4	0,17	0,21	4,45	H4 25,9 $\times$, F1 −0,020 (kecil, jenuh)
university / stasioner	0 dari 4	0,08	0,11	3,76	H4 49 $\times$, F1 +0,007

H3 didukung secara bersyarat. Pada *workforce* stasioner, pemicu *drift* tidak aktif selama empat siklus, sedangkan kondisi *always* tetap mempelajari ulang struktur pada setiap siklus. Akibatnya, *warm-start* lebih cepat daripada *always* (3,46 dibanding

4,58 detik; $p = 6,1 \times 10^{-5}$) tanpa penurunan kualitas. Pada *drift* rendah dan sedang, pemicu aktif pada setiap jendela sehingga *warm* berperilaku sama dengan *always*, sesuai rancangan mekanisme. Pada *e-document*, rantai derau JSD melebihi τ sehingga pemicu terus aktif dan penghematan H3 tidak dapat ditunjukkan. Dengan demikian, manfaat pemisahan pembaruan struktur bergantung pada intensitas *drift* dan rantai derau distribusi dataset.

H4 memperoleh dukungan kuat. *Warm-start* mempercepat re-optimasi dibandingkan *re-mining* penuh pada seluruh dataset, yaitu sekitar 2,0–2,6 kali pada *workforce*, 1,27 kali pada *e-document*, dan 21–52 kali pada tiga dataset kecil. Pada dua dataset utama, *warm-start* juga menghasilkan F1 lebih tinggi sebesar 0,06–0,17 karena proses pembaruan mempertahankan informasi kebijakan sebelumnya dan berfokus pada pola yang belum tercakup. Pada tiga dataset kecil, perbedaan kualitas relatif kecil dan tidak konsisten ($-0,057$ hingga $+0,031$), sehingga bukti utama kualitas H4 berasal dari *workforce* dan *e-document*.

Hasil H3 dan H4 menunjukkan manfaat CoDA-EDA pada pembaruan kebijakan berkala di *edge*. Pemicu berbasis *drift* dapat menghindari pembelajaran ulang struktur ketika distribusi stabil, sedangkan *warm-start* mengurangi waktu re-optimasi dibandingkan penambangan ulang dari awal. Dibandingkan iBOA, yang memiliki kualitas kompetitif pada penambangan statis, keunggulan CoDA-EDA lebih terlihat pada skenario pembaruan berkala. Bersama hasil H2, temuan ini menunjukkan bahwa kontribusi utama CoDA-EDA terletak pada efisiensi memori dan pembaruan kebijakan, bukan pada dominasi kualitas penambangan statis.

IV.4 Validasi Eksternal

Seluruh pengujian utama menggunakan dataset hasil perluasan generator ABAC Lab. Validasi eksternal dilakukan menggunakan *Smart Building IoT* dari Díaz-Rodríguez dan Díaz-Pérez (2025) untuk menilai apakah temuan tetap konsisten pada data yang dibangkitkan dengan generator, kebijakan, dan distribusi yang independen dari penelitian ini. Dataset tersebut memiliki rasio *permit* sekitar 0,75, sehingga kebijakan trivial yang mengizinkan seluruh permintaan sudah menghasilkan *F1-score* 0,858 dan digunakan sebagai *baseline* minimum yang relevan. Karena dataset asli berukuran sekitar 5,7 juta permintaan, data disubsampel secara terstratifikasi menjadi 8.000 entri dengan dimensi efektif 428. Hasil pengujian kualitas H1 disajikan pada Tabel IV.7.

Tabel IV.7 Validasi Eksternal Smart Building IoT — Kualitas ($D = 428$; garis sepele $F1=0,858$; 5 seed)

Algoritma	F-score	Presisi	Cakupan	>sepele	Keterangan
BOA	0,998	1,000	0,747	5/5 seed	Kebijakan hampir dipulihkan sempurna
ACO	0,994	1,000	0,742	5/5 seed	Setara BOA
compact-PSO	0,964	1,000	0,704	4/5 seed	Di atas garis sepele
compact-GA	0,961	1,000	0,697	5/5 seed	Di atas garis sepele
iBOA	0,943	1,000	0,674	4/5 seed	Setara CoDA-EDA
CoDA-EDA	0,938	1,000	0,665	5/5 seed	Melampaui sepele di seluruh seed
UBK	0,834	1,000	0,539	2/5 seed	Satu-satunya yang tak konsisten

Validasi eksternal H1 menunjukkan bahwa CoDA-EDA melampaui *baseline* trivial pada seluruh lima *seed*, dengan *F1-score* rata-rata 0,938 dibandingkan 0,858. Lima algoritma lain menunjukkan pola serupa, sedangkan UBK hanya melampaui *baseline* pada dua dari lima *seed*. Uji Friedman menunjukkan perbedaan signifikan antarkondisi ($p = 1,6 \times 10^{-3}$). Seluruh algoritma mencapai presisi 1,000 sehingga perbedaan kualitas terutama ditentukan oleh *coverage*. CoDA-EDA berada pada tingkat yang sebanding dengan iBOA dan sedikit di bawah keluarga *compact*, BOA, serta ACO, konsisten dengan pola yang sebelumnya ditemukan pada dataset berdimensi menengah.

H2–H4 juga diuji pada dataset ini. Untuk H2, pola jejak memori antaralgoritma konsisten dengan hasil ABAC Lab. Untuk H3 dan H4, lima jendela diuji pada kondisi stasioner dan kondisi *drift*. Pada kondisi stasioner, rantai derau JSD sebesar 0,0065 berada di bawah $\tau = 0,01$, sehingga pemicu pembaruan struktur hanya aktif pada satu dari lima jendela. Akibatnya, *warm-start* lebih cepat daripada strategi *always* (6,7 dibanding 7,9 detik) dengan kualitas yang serupa ($F1$ 0,977 dibanding 0,973). Hasil ini memberikan dukungan tambahan terhadap H3, yang sebelumnya tidak dapat ditunjukkan pada *e-document* akibat rantai derau yang lebih tinggi.

Pada kondisi *drift* yang dibentuk melalui pengurutan berdasarkan atribut lokasi, pemicu struktur aktif pada seluruh lima jendela sebagaimana diharapkan. H4 juga konsisten pada kedua kondisi: *warm-start* sekitar 2,1–2,5 kali lebih cepat daripada

re-mining penuh (6,7 dibanding 16,9 detik pada salah satu konfigurasi) serta menghasilkan F1 lebih tinggi, yaitu 0,977 dibanding 0,914.

Secara keseluruhan, validasi eksternal menunjukkan bahwa pola H1–H4 tetap muncul pada dataset sintesis yang dibangkitkan oleh *generator* independen. Hasil ini mendukung ketahanan lintas-*generator*, tetapi tidak dapat ditafsirkan sebagai bukti generalisasi terhadap data operasional *edge-IoT*. Khusus H3, keberhasilan mekanisme pada *Smart Building IoT* dengan rantai derau rendah menunjukkan bahwa keterbatasan yang ditemukan pada *e-document* lebih berkaitan dengan karakteristik estimasi distribusi dataset daripada kegagalan mekanisme pembaruan struktur itu sendiri.

IV.5 Sintesis, Keterbatasan, dan Jawaban Rumusan Masalah

Subbab ini menyintesis hasil keempat hipotesis lintas seluruh dataset, memosisikannya terhadap kebaruan yang diklaim, menjawab rumusan masalah, dan menyatakan keterbatasan. Status ringkas H1-H4 disajikan pada Tabel IV.8.

Tabel IV.8 Sintesis Status Hipotesis H1-H4 Lintas Dataset

Hipotesis	Dua dataset utama ($D = 436$ & 1.907)	Tiga dataset kecil ($D = 46-54$)	Smart Building IoT (eksternal)
H1 — Kualitas	Unggul keluarga compact; setara iBOA; kalah moderat ACO/UBK pada D besar	Setara mayoritas (tumpul; duplikasi tinggi)	Melampaui sepele 5/5; setara iBOA
H2 — Memori	Terkonfirmasi (datar $1,00\times$ vs $6,2-9,2\times$)	Konsisten (properti algoritma)	Konsisten
H3 — Pembaruan	Terdukung (workforce stasioner); tak tampil di e-document (rantai derau)	Terbatas (kejenuhan)	Terdukung (mode sepadan; relearn 1/5)

Lanjut ke halaman berikutnya

Tabel IV.8 – lanjutan dari halaman sebelumnya

Hipotesis	Dua dataset utama ($D = 436$ & 1.907)	Tiga dataset kecil ($D = 46-54$)	Smart Building IoT (eksternal)
H4 — Warm-start	Terdukung penuh (2,0–2,6×, kualitas naik)	Terdukung (21–52×; kualitas campur)	Terdukung (2,1–2,5×, kualitas naik)

Dua temuan utama penelitian adalah efisiensi memori dan pembaruan kebijakan. H2 menunjukkan bahwa memori CoDA-EDA tetap stabil terhadap NP , sedangkan BOA, ACO, dan UBK meningkat 6, 2–9, 2 kali. H4 juga menunjukkan bahwa *warm-start* mempercepat pembaruan kebijakan dengan kualitas yang tetap kompetitif. Sementara itu, H1 bergantung pada dimensi dan anggaran aturan, sedangkan H3 dipengaruhi tingkat *drift* dan rantai derau JSD.

Hasil tersebut mendukung tiga unsur utama CoDA-EDA, yaitu pembaruan tanpa populasi eksplisit, pemodelan dependensi orde satu melalui Chow–Liu, dan pemisahan pembaruan parameter dari struktur. CoDA-EDA mampu mempertahankan pemodelan dependensi tanpa kehilangan sifat *compact* serta mengurangi pembelajaran ulang struktur ketika distribusi stabil.

Dengan demikian, kontribusi utama CoDA-EDA terletak pada efisiensi memori dan pembaruan inkremental, bukan pada dominasi kualitas penambangan statis. Pengujian langsung pada Raspberry Pi 5 dan validasi lintas-*generator* mendukung temuan tersebut, meskipun generalisasi masih terbatas oleh penggunaan satu dataset eksternal, data sintetis, serta beberapa hasil dengan jumlah replikasi terbatas.

Bab V Kesimpulan dan Saran

V.1 Kesimpulan

Penelitian ini merancang dan menguji CoDA-EDA, yaitu EDA *compact* yang mempertahankan dependensi antaratribut untuk *ABAC policy mining* pada perangkat *edge-IoT*.

Pertama, CoDA-EDA menggabungkan *Probability Vector*, struktur dependensi Chow–Liu orde satu, serta pemisahan pembaruan parameter dan struktur. Rancangan ini memungkinkan pemodelan dependensi tanpa menyimpan populasi eksplisit.

Kedua, H1 didukung secara bersyarat. CoDA-EDA memberikan kualitas yang kompetitif dan menunjukkan keunggulan terhadap keluarga *compact* univariat terutama pada ruang solusi berdimensi tinggi. Pada anggaran aturan yang memadai, kualitasnya sebanding dengan iBOA/BOA, tetapi masih dapat tertinggal dari ACO dan UBK pada penambahan statis.

Ketiga, H2 didukung kuat. Jejak memori CoDA-EDA tetap stabil terhadap ukuran populasi virtual dengan rasio 1,00, sedangkan BOA, ACO, dan UBK meningkat 6, 2–9, 2 kali ketika ukuran populasi dinaikkan enam belas kali.

Keempat, H3 dan H4 menunjukkan manfaat pada pembaruan kebijakan. Pembaruan struktur berbasis *drift* mengurangi waktu komputasi ketika distribusi relatif stabil, meskipun manfaat H3 bergantung pada tingkat *drift* dan lantai derau dataset. Sementara itu, *warm-start* mempercepat re-optimalisasi sekitar 2,0–2,6 kali dibandingkan *re-mining* penuh dengan kualitas yang sekurang-kurangnya sebanding.

Kelima, validasi pada *Smart Building IoT* menunjukkan pola hasil yang konsisten dengan pengujian utama, sehingga mendukung ketahanan temuan pada *generator* sintetis yang berbeda, meskipun belum membuktikan generalisasi terhadap data operasional *edge-IoT*.

Secara keseluruhan, kontribusi utama CoDA-EDA terletak pada efisiensi memori dan pembaruan kebijakan secara inkremental, bukan pada dominasi kualitas penambahan statis. Karakteristik tersebut menjadikannya sesuai untuk perangkat *edge* dengan sumber daya terbatas yang memerlukan pembaruan kebijakan secara berkala.

V.2 Saran

Saran penelitian dibagi menjadi penerapan praktis dan pengembangan lanjutan.

Untuk penerapan praktis, CoDA-EDA sesuai digunakan ketika prioritas utama adalah efisiensi memori dan pembaruan kebijakan berkala pada perangkat *edge*. Jika prioritasnya adalah kualitas penambangan statis pada dataset berdimensi besar, algoritma berpopulasi seperti ACO dan UBK dapat menjadi alternatif. Jumlah aturan pada strategi *separate-and-conquer* juga perlu disesuaikan karena peningkatan anggaran aturan dapat memperluas *coverage* dengan konsekuensi waktu komputasi dan kompleksitas kebijakan yang lebih tinggi.

Untuk pengembangan lanjutan, ambang *drift* (τ) dapat dibuat adaptif terhadap karakteristik distribusi agar tetap efektif pada dataset berdimensi besar. Selain itu, fungsi *fitness* atau mekanisme pencarian dapat dikembangkan agar lebih sadar *coverage*, sehingga kualitas tidak terlalu bergantung pada jumlah aturan. Pemodelan dependensi orde lebih tinggi secara selektif juga layak dieksplorasi dengan tetap mempertahankan efisiensi memori. Terakhir, validasi perlu diperluas ke lebih banyak dataset independen, data operasional IoT nyata, beragam perangkat *edge*, serta integrasi dalam *pipeline* pengelolaan kebijakan untuk menilai kelayakan penerapan secara lebih menyeluruh.

DAFTAR PUSTAKA

- Abu Jabal, A., Bertino, E., Lobo, J., Law, M., Russo, A., Calo, S., dan Verma, D. (2020): Polisma - A Framework for Learning Attribute-Based Access Control Policies (Lecture Notes in Computer Science), 523–544 *dalam* L. Chen, N. Li, K. Liang, dan S. Schneider, ed., *Computer Security – ESORICS 2020*, Springer International Publishing, Cham. https://doi.org/10.1007/978-3-030-58951-6_26.
- Ahsan, M. S., dan Pathan, A.-S. K. (2025) : A Comprehensive Survey on the Requirements, Applications, and Future Challenges for Access Control Models in IoT: The State of the Art, *IoT*, **6**, 9.
- Bamberger, A., dan Fernández, M. (2024) : Automated Generation and Update of Structured ABAC Policies, *Proceedings of the 2024 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems*, 31–40.
- Batra, G., Atluri, V., Vaidya, J., dan Sural, S. (2021) : Incremental Maintenance of ABAC Policies, *Proceedings of the Eleventh ACM Conference on Data and Application Security and Privacy*, 185–196.
- Bui, T., Matricia, A., Contreras, E., Mauvais, R., Medina, L., dan Serrano, I. (2025) : ABAC Lab: An Interactive Platform for Attribute-based Access Control Policy Analysis, Tools, and Datasets [Dataset/Tool Paper], *Proceedings of the 30th ACM Symposium on Access Control Models and Technologies*, 111–116.
- Canavese, D., Mannella, L., Regano, L., dan Basile, C. (2024) : Security at the Edge for Resource-Limited IoT Devices, *Sensors*, **24**, 590.
- Chow, C., dan Liu, C. (1968) : Approximating discrete probability distributions with dependence trees, *IEEE Transactions on Information Theory*, **14**, 462–467.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., dan Stein, C. (2022). *Introduction to algorithms*. Fourth edition. Cambridge, Massachusetts London, England: The MIT Press. 1 p.
- Cotrini Jimenez, C., Weghorn, T., dan Basin, D. (2018) : Mining ABAC Rules from Sparse Logs, 31–46.
- Dauda, A., Flauzac, O., dan Nolot, F. (2024) : A Survey on IoT Application Architectures, *Sensors*, **24**, 5320.
- Díaz-Rodríguez, H., dan Díaz-Pérez, A. (2025) : ABAC Policy Mining Using Complex Network Analysis Techniques, *Applied Sciences*, **15**, 12571.
- Dritsas, E., dan Trigka, M. (2025) : A Survey on Cybersecurity in IoT, *Future Internet*, **17**, 30.

- Gudyś, A., Sikora, M., dan Wróbel, Ł. (2024) : Separate and conquer heuristic allows robust mining of contrast sets in classification, regression, and survival data, *Expert Systems with Applications*, **248**, 123376.
- Guo, Y., Wang, X., Yu, M., Li, F., Pang, Z., dan Fang, L. (2025) : An on-the-fly framework for usable access control policy mining, *Computers & Security*, **150**, 104211.
- Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., dan Scarfone, K. (2014): *Guide to Attribute Based Access Control (ABAC) Definition and Considerations* (NIST SP 800-162), National Institute of Standards dan Technology, diperoleh 11 Mei 2026 melalui situs internet: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-162.pdf>.
- Iyer, P., dan Masoumzadeh, A. (2018) : Mining Positive and Negative Attribute-Based Access Control Policy Rules, *Proceedings of the 23rd ACM on Symposium on Access Control Models and Technologies*, 161–172.
- Jakšić, Z., Devi, S., Jakšić, O., dan Guha, K. (2023) : A Comprehensive Review of Bio-Inspired Optimization Algorithms Including Applications in Microelectronics and Nanophotonics, *Biomimetics*, **8**, 278.
- Khalifi, S., Caraffini, F., dan Iacca, G. (2023) : Metaheuristics in the Balance: A Survey on Memory-Saving Approaches for Platforms with Seriously Limited Resources, *International Journal of Intelligent Systems*, **2023**, 5708085.
- Khosrowshahi, H. N., Aghdasi, H. S., dan Salehpour, P. (2025) : A refined Greylag Goose optimization method for effective IoT service allocation in edge computing systems, *Scientific Reports*, **15**, 15729.
- Larrañaga, P., dan Bielza, C. (2024) : Estimation of Distribution Algorithms in Machine Learning: A Survey, *IEEE Transactions on Evolutionary Computation*, **28**, 1301–1321.
- Lemtenneche, S., Bensayah, A., dan Cheriet, A. (2023) : An Estimation of Distribution Algorithm for Permutation Flow-Shop Scheduling Problem, *Systems*, **11**, 389.
- Li, S., Liu, W., Wu, Y., Wu, X., dan Li, L. (2026) : Attribute based access control of geographic spatial data sharing using blockchain and smart contracts, *Scientific Reports*, **16**, 9132.
- Lim, Y. T., Cheng, P. C., Clark, J. A., dan Rohatgi, P. (2008): Policy Evolution with Grammatical Evolution, 71–80 *dalam* X. Li, M. Kirley, M. Zhang, D. Green, V. Ciesielski, H. Abbass, Z. Michalewicz, T. Hendtlass, K. Deb, K. C. Tan, J. Branke, dan Y. Shi, ed., *Simulated Evolution and Learning*, Springer Berlin Heidelberg, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-89694-4_8.

- Lim, Y. T., Cheng, P.-C., Rohatgi, P., dan Clark, J. A. (2009) : Dynamic security policy learning, *Proceedings of the first ACM workshop on Information security governance*, 39–48.
- Lin, J. (1991) : Divergence Measures Based on the Shannon Entropy, *IEEE Transactions on Information Theory*, **37**, 145–151.
- Medvet, E., Bartoli, A., Carminati, B., dan Ferrari, E. (2015): Evolutionary Inference of Attribute-Based Access Control Policies, 351–365 *dalam* A. Gaspar-Cunha, C. Henggeler Antunes, dan C. C. Coello, ed., *Evolutionary Multi-Criterion Optimization*, Springer International Publishing, Cham. https://doi.org/10.1007/978-3-319-15934-8_24.
- Molloy, M., dan Reed, B. (1995) : A critical point for random graphs with a given degree sequence, *Random Structures & Algorithms*, **6**, 161–180.
- Omran, M. G. H., Clerc, M., Ghaddar, F., Aldabagh, A., dan Tawfik, O. (2022) : Permutation Tests for Metaheuristic Algorithms, *Mathematics*, **10**, 2219.
- Pan, J.-S., Hu, P., Snášel, V., dan Chu, S.-C. (2023) : A survey on binary metaheuristic algorithms and their engineering applications, *Artificial Intelligence Review*, **56**, 6101–6167.
- Peffer, K., Tuunanen, T., Rothenberger, M. A., dan Chatterjee, S. (2007) : A Design Science Research Methodology for Information Systems Research, *Journal of Management Information Systems*, **24**, 45–77.
- Pelikan, M., Sastry, K., dan Goldberg, D. E. (2008) : iBOA: the incremental bayesian optimization algorithm, *Proceedings of the 10th annual conference on Genetic and evolutionary computation*, 455–462.
- Peres, F. F. (2026) : Effect sizes for nonparametric tests, *Biochemia medica*, **36**, 5–16.
- Perez-Haro, A., dan Diaz-Perez, A. (2024) : ABAC Policy Mining through Affiliation Networks and Biclique Analysis, *Information*, **15**, 45.
- Quan, S., Zhao, Y., dan Helil, N. (2023) : Attribute-Based Access Control Policy Generation Approach from Access Logs Based on the CatBoost, *Computing and Informatics*, **42**, 615–650.
- Ragothaman, K., Wang, Y., Rimal, B., dan Lawrence, M. (2023) : Access Control for IoT: A Survey of Existing Research, Dynamic Policies and Future Directions, *Sensors*, **23**, 1805.
- Rajwar, K., Deep, K., dan Das, S. (2023) : An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges, *Artificial Intelligence Review*, **56**, 13187–13257.

- Raspberry Pi Ltd (2026): *Raspberry Pi 5 Product Brief* (Raspberry Pi Technical Documentation RP-008348-DS-6), Raspberry Pi Ltd, Cambridge, UK, diperoleh 7 Agustus 2026 melalui situs internet: <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>.
- Shang, S., Wang, X., dan Liu, A. (2024) : ABAC policy mining method based on hierarchical clustering and relationship extraction, *Computers & Security*, **139**, 103717.
- Siyuan, S., Aodi, L., Xuehui, D., Xiaohan, W., dan Ming, T. (2025) : ABAC policy mining method for heterogeneous access control system, *The Journal of Supercomputing*, **81**, 1065.
- Soloviev, V. P., Larrañaga, P., dan Bielza, C. (2024) : EDAspy: An extensible python package for estimation of distribution algorithms, *Neurocomputing*, **598**, 128043.
- Xu, Z., dan Stoller, S. D. (2015) : Mining Attribute-Based Access Control Policies, *IEEE Transactions on Dependable and Secure Computing*, **12**, 533–545.
- Yang, W., Su, R., Cheng, Y., dan Guo, J. (2022) : A Concept Drift Detection Approach Based on Jensen–Shannon Divergence for Network Traffic Classification, *Proceedings of the 2022 5th International Conference on Artificial Intelligence and Pattern Recognition*, 982–987.

LAMPIRAN

Lampiran A Surat Pernyataan Penggunaan AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya:

Nama/NIM : Ziadan Qowi / 23524019
Fakultas/Sekolah : STEI
Kode Mata Kuliah : IF6099
Nama Mata Kuliah : Sidang Magister
Judul Tugas/Proposal : CoDA-EDA: *Compact Dependency-Aware Estimation of Distribution Algorithm* untuk *Incremental Policy Mining Attribute-Based Access Control* (ABAC) pada *Edge-IoT*

Menyatakan bahwa saya menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools ChatGPT.	Ya	3
Pembuatan Teks: menggunakan tools seperti ChatGPT, dan DeepSeek.	Ya	3
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools Claude.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti ChatGPT dan Claude.	Ya	2

Saya juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 September 2026

(Ziadan Qowi)
NIM. 23524019

Lampiran B Contoh Kebijakan Hasil Mining Lengkap

B.1 CoDA-EDA pada Dataset *Workforce* ($D = 436$)

Sumber data: ABAC Lab / *workforce* (*corr* = sedang, *drift* = stasioner)

Ukuran log: $n = 10000$ permintaan, $D = 436$ pasangan atribut-nilai

Metrik kebijakan: $F1 = 0,885$; $precision = 0,862$; $recall = 0,910$; $coverage = 0,317$;

$WSC = 20$; jumlah aturan = 5

- **Aturan 1:** JIKA $provider \in \{eWorkforce, externalWorkforceSupplier\}$ DAN $position \in \{workforceManager, technician\}$ DAN $action = view$ MAKA izinkan (permit)
Cakupan: TP=1651 FP=113 WSC=4
- **Aturan 2:** JIKA $provider = eWorkforce$ DAN $assignedTenant = powerProtection$ DAN $associatedContract = contract006$ DAN $action = complete$ MAKA izinkan (permit)
Cakupan: TP=28 FP=36 WSC=3
- **Aturan 3:** JIKA $provider \in \{externalHelpdeskSupplier, telco\}$ DAN $tenant = telco$ DAN $tenantType = primary$ DAN $associatedContract \in \{contract001, contract003, contract005\}$ DAN $isAppointment = False$ DAN $action = modify$ MAKA izinkan (permit)
Cakupan: TP=402 FP=3 WSC=8
- **Aturan 4:** JIKA $contractStatus = active$ DAN $action = view$ MAKA izinkan (permit)
Cakupan: TP=1291 FP=339 WSC=1
- **Aturan 5:** JIKA $provider = externalHelpdeskSupplier$ DAN $tenant = powerProtection$ DAN $recurrence \in \{True, False\}$ DAN $action = modify$ MAKA izinkan (permit)
Cakupan: TP=82 FP=2 WSC=4

B.2 ACO pada Dataset *Workforce* (Pembanding)

Sumber data: ABAC Lab / *workforce* (*corr* = sedang, *drift* = stasioner)

Ukuran log: $n = 10000$ permintaan, $D = 436$ pasangan atribut-nilai

Metrik kebijakan: $F1 = 0,820$; $precision = 0,697$; $recall = 0,995$; $coverage = 0,428$;
WSC = 19; jumlah aturan = 5

- **Aturan 1:** JIKA `tenantType = primary` DAN `action ∈ {modify, view}` MAKA izinkan (permit)
Cakupan: TP=2879 FP=1297 WSC=1
- **Aturan 2:** JIKA `provider = eWorkforce` DAN `position = helpdeskOperator` DAN `action = delete` MAKA izinkan (permit)
Cakupan: TP=57 FP=0 WSC=2
- **Aturan 3:** JIKA `assignedRegion = north` DAN `certifications = powerProtectionSpecialist` DAN `action = complete` MAKA izinkan (permit)
Cakupan: TP=19 FP=0 WSC=2
- **Aturan 4:** JIKA `managedStaff ∈ {tech009, tech011, tech014, tech025, tech036, tech046, tech060, tech062, tech065}` DAN `action = complete` MAKA izinkan (permit)
Cakupan: TP=21 FP=2 WSC=9
- **Aturan 5:** JIKA `department = workforce` DAN `assignedRegion = north` DAN `assignedTechnician ∈ {tech005, tech014, tech046}` DAN `action = complete` MAKA izinkan (permit)
Cakupan: TP=12 FP=0 WSC=5

B.3 CoDA-EDA pada Dataset *Smart Building IoT* ($D = 428$)

Sumber data: *Smart Building IoT*

Ukuran log: $n = 8000$ permintaan, $D = 428$ pasangan atribut-nilai

Metrik kebijakan: $F1 = 0,998$; $precision = 1,000$; $recall = 0,996$; $coverage = 0,747$;
WSC = 276; jumlah aturan = 5

- **Aturan 1:** JIKA `area ∈ {backyard, bedroom}` DAN `action = arm` MAKA izinkan (permit)
Cakupan: TP=1770 FP=0 WSC=2
- **Aturan 2:** JIKA `uname ∈ {116, 104, 110, 108, 118, 119, 107, 84, 77, 114, 38, 97, 117, 109, 87, 106, 54, 88, 115, 99, 93, 36, 2, 113, 111} ... (+34 nilai lain)` DAN `action ∈ {arm, access, control}` MAKA

izinkan (permit)

Cakupan: TP=4111 FP=0 WSC=59

- **Aturan 3:** JIKA $rname \in \{1100, 1109, 1068, 1106, \text{__rare__}, 1091, 1108, 629, 1067, 1052, 1105, 254, 961, 1097, 1038, 1031, 1090, 1098, 1102, 1069, 1041, 1033, 1008, 1107, 1077\} \dots$ (+87 nilai lain) DAN $action \in \{arm, access, control\}$ MAKA izinkan (permit)

Cakupan: TP=4794 FP=0 WSC=112

- **Aturan 4:** JIKA $rname \in \{1100, 1092, 1065, 1043, 1097, 1096, 1032, 1073, 711, 1078, 1060, 1087, 1041, 1051, 1008, 1071, 843, 1057, 837, 980, 899, 849, 1076, 1086, 1055\} \dots$ (+75 nilai lain) DAN $action \in \{access, control\}$ MAKA izinkan (permit)

Cakupan: TP=854 FP=0 WSC=100

- **Aturan 5:** JIKA $type \in \{TV, Security\ Cameras, Smart\ locks\}$ DAN $action \in \{arm, control\}$ MAKA izinkan (permit)

Cakupan: TP=3417 FP=0 WSC=3

Lampiran C Tabel Hasil Eksperimen Lengkap

C.1 Kualitas Kebijakan (H1) – Rerata Per Dataset dan Tingkat Korelasi

Tabel C.1 F-score, cakupan, dan WSC pada workforce (D=436)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,860	0,890	0,918	0,337	33,3
compact- GA	0,872	0,902	0,925	0,325	41,7
compact- PSO	0,864	0,897	0,918	0,320	51,7
iBOA	0,862	0,883	0,919	0,341	29,2
BOA	0,823	0,831	0,840	0,418	9,0
ACO	0,861	0,862	0,887	0,388	13,9
UBK	0,837	0,861	0,860	0,376	6,4

Tabel C.2 F-score, cakupan, dan WSC pada e-document (D=1.907)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,453	0,442	0,459	0,248	11,8
compact- GA	0,355	0,277	0,356	0,156	10,2
compact- PSO	0,220	0,287	0,273	0,116	11,0
iBOA	0,413	0,460	0,483	0,246	13,8
BOA	0,605	0,643	0,652	0,624	11,2
ACO	0,648	0,663	0,671	0,572	16,9
UBK	0,627	0,617	0,647	0,468	6,2

Tabel C.3 F-score, cakupan, dan WSC pada healthcare (D=46)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,731	0,802	0,804	0,479	2,0
compact- GA	0,747	0,771	0,778	0,490	2,1
compact- PSO	0,747	0,818	0,838	0,459	3,0
iBOA	0,760	0,823	0,824	0,455	2,5
BOA	0,792	0,833	0,881	0,433	2,6
ACO	0,731	0,774	0,778	0,494	1,4
UBK	0,739	0,793	0,812	0,478	2,0

Tabel C.4 F-score, cakupan, dan WSC pada project-management (D=51)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,852	0,918	0,923	0,372	7,0
compact- GA	0,832	0,900	0,918	0,384	6,4
compact- PSO	0,854	0,920	0,930	0,370	6,7
iBOA	0,857	0,922	0,947	0,366	6,7
BOA	0,918	0,927	0,935	0,349	6,2
ACO	0,834	0,901	0,924	0,382	5,5
UBK	0,815	0,855	0,889	0,404	7,7

Tabel C.5 F-score, cakupan, dan WSC pada university (D=54)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,824	0,834	0,832	0,425	6,6
compact- GA	0,786	0,833	0,848	0,432	6,4

Tabel C.5 F-score, cakupan, dan WSC pada university (D=54) (lanjutan)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
compact-PSO	0,803	0,829	0,854	0,426	9,7
iBOA	0,779	0,829	0,863	0,431	7,6
BOA	0,809	0,826	0,856	0,424	6,9
ACO	0,791	0,832	0,884	0,420	7,9
UBK	0,860	0,870	0,876	0,385	6,6

C.2 Jejak Memori (H2) – Rasio Penskalaan

$\text{alloc}(NP = 400)/\text{alloc}(NP = 25)$ Per Dataset

Tabel C.6 Rasio penskalaan jejak memori terhadap ukuran populasi (per dataset dan rata-rata)

Algoritma	workforce	e-document	healthcare	project-mgmt	university	Rata-rata
CoDA-EDA	1,00	1,00	1,00	1,00	1,00	1,00x
compact-GA	0,47	0,36	1,00	1,00	1,00	0,77x
compact-PSO	0,48	0,39	1,00	1,00	0,95	0,76x
iBOA	0,53	0,58	1,00	1,00	1,00	0,82x
BOA	12,09	13,80	3,94	4,21	4,37	7,68x
ACO	9,47	13,24	2,63	2,82	2,94	6,22x
UBK	12,08	15,64	5,64	6,17	6,49	9,20x

Catatan: rasio mendekati 1,00 menandakan memori datar terhadap ukuran populasi. Nilai per-dataset pada algoritma berpopulasi lebih tinggi pada dua dataset besar karena skala populasi absolutnya lebih besar.

C.3 Efisiensi Pembaruan (H3 dan H4) – Waktu Per Siklus dan Kualitas

Tabel C.7 Waktu total pembaruan (warm/always/cold) dan F-score akhir per dataset dan skenario drift

Dataset / skenario	Relearn (dari 4)	Waktu pembaruan			F-score akhir	
		warm (s)	always (s)	cold (s)	F1 warm	F1 cold
workforce / stasioner	0,0	3,46	4,58	9,05	0,901	0,805
workforce / rendah	4,0	4,50	4,49	9,12	0,891	0,812
workforce / sedang	4,0	4,52	4,53	8,95	0,877	0,816
edocument / stasioner	4,0	22,26	22,27	28,24	0,399	0,297
edocument / rendah	4,0	22,38	22,39	28,36	0,465	0,341
edocument / sedang	4,0	22,37	22,37	28,31	0,433	0,267
healthcare / stasioner	0,0	0,00	0,00	2,00	0,814	0,807
healthcare / rendah	3,3	0,00	0,00	2,81	0,789	0,838
healthcare / sedang	4,0	0,00	0,00	2,67	0,832	0,890
project-management / stasioner	0,0	0,17	0,21	4,45	0,909	0,928
project-management / rendah	3,0	0,16	0,20	3,98	0,928	0,896
project-management / sedang	3,3	0,20	0,24	4,13	0,936	0,928

Tabel C.7 Waktu total pembaruan (warm/always/cold) dan F-score akhir per dataset dan skenario drift (lanjutan)

Dataset / skenario	Relearn (dari 4)	Waktu pembaruan			F-score akhir	
		warm (s)	always (s)	cold (s)	F1 warm	F1 cold
university / stasioner	0,0	0,08	0,11	3,76	0,810	0,803
university / rendah	4,0	0,07	0,07	3,88	0,808	0,819
university / sedang	3,7	0,08	0,08	3,99	0,798	0,850

H_3 (*warm* vs *always*) tampak ketika $Relearn < 4$ (pemicu drift diam); H_4 (*warm* vs *cold*) tampak dari waktu *warm* yang jauh lebih kecil dari *cold* dengan $F1_{warm} \geq F1_{cold}$.

C.4 Validasi Eksternal *Smart Building IoT* (H1-H4)

Tabel C.8 Tabel Kualitas pada Smart Building IoT (D=428; garis sepele F1=0,858; 5 seed)

Algoritma	F-score	Presisi	Cakupan	Memori (KB)	>sepele
BOA	0,998	1,000	0,747	960	5/5
ACO	0,994	1,000	0,742	513	5/5
compact-PSO	0,964	1,000	0,704	989	4/5
compact-GA	0,961	1,000	0,697	729	5/5
iBOA	0,943	1,000	0,674	505	4/5
CoDA-EDA	0,938	1,000	0,665	949	5/5
UBK	0,834	1,000	0,539	1122	2/5

Tabel C.9 Siklus pembaruan pada Smart Building IoT (total waktu, F1 akhir, jumlah relearn)

Mode jendela	Algoritma	Kondisi	Total waktu (s)	F1 akhir	Relearn
acak (sepadan)	CoDA-EDA	always	nan	0,973	5,0
acak (sepadan)	CoDA-EDA	cold	nan	0,914	5,0
acak (sepadan)	CoDA-EDA	warm	nan	0,977	1,0
acak (sepadan)	iBOA	cold	nan	0,934	5,0
attr (drift)	CoDA-EDA	always	nan	0,958	5,0
attr (drift)	CoDA-EDA	cold	nan	0,839	5,0
attr (drift)	CoDA-EDA	warm	nan	0,958	5,0
attr (drift)	iBOA	cold	nan	0,952	5,0

Lampiran D Hasil Uji Statistik Lengkap

D.1 Uji H1 (Kualitas Kebijakan) Per Dataset

Tabel D.1 Post-hoc CoDA-EDA vs baseline pada workforce (D=436) (Friedman $\chi^2 = 50,5$; $p = 3,8 \times 10^{-9}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	-0,011	0,306	-0,29	setara
compact-PSO	-0,004	1,000	-0,11	setara
iBOA	+0,001	1,000	-0,16	setara
BOA	+0,058	$8,6 \times 10^{-5}$	+0,56	CoDA-EDA unggul
ACO	+0,019	0,266	+0,24	setara
UBK	+0,036	0,011	+0,33	CoDA-EDA unggul

Tabel D.2 Post-hoc CoDA-EDA vs baseline pada e-document (D=1.907) (Friedman $\chi^2 = 186,8$; $p = 1,2 \times 10^{-37}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,122	0,003	+0,42	CoDA-EDA unggul
compact-PSO	+0,192	$2,2 \times 10^{-7}$	+0,60	CoDA-EDA unggul
iBOA	-0,001	0,763	-0,16	setara
BOA	-0,181	$9,5 \times 10^{-8}$	-0,87	CoDA-EDA kalah
ACO	-0,209	$7,1 \times 10^{-12}$	-0,91	CoDA-EDA kalah
UBK	-0,178	$6,5 \times 10^{-12}$	-0,91	CoDA-EDA kalah

Tabel D.3 Post-hoc CoDA-EDA vs baseline pada healthcare (D=46) (Friedman $\chi^2 = 21,6$; $p = 0,001$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,014	1,000	+0,00	setara
compact-PSO	-0,022	0,679	-0,41	setara
iBOA	-0,023	1,000	-0,45	setara
BOA	-0,056	0,122	-0,56	setara
ACO	+0,018	1,000	+0,43	setara
UBK	-0,002	1,000	+0,11	setara

Tabel D.4 Post-hoc CoDA-EDA vs baseline pada project-management (D=51) (Friedman $\chi^2 = 47,6$; $p = 1,4 \times 10^{-8}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,014	0,241	+0,41	setara
compact-PSO	-0,004	1,000	+0,06	setara
iBOA	-0,011	1,000	-0,03	setara
BOA	-0,029	0,143	-0,24	setara
ACO	+0,011	1,000	+0,10	setara
UBK	+0,045	0,002	+0,49	CoDA-EDA unggul

Tabel D.5 Post-hoc CoDA-EDA vs baseline pada university (D=54) (Friedman $\chi^2 = 41,5$; $p = 2,3 \times 10^{-7}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,007	1,000	+0,11	setara
compact-PSO	+0,001	1,000	+0,21	setara
iBOA	+0,006	1,000	+0,05	setara
BOA	-0,001	1,000	-0,09	setara
ACO	-0,005	1,000	-0,12	setara

Tabel D.5 Post-hoc CoDA-EDA vs baseline pada university (D=54) (lanjutan)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
UBK	-0,039	$2,3 \times 10^{-4}$	-0,73	CoDA-EDA kalah

D.2 Uji H3 dan H4 (Efisiensi Pembaruan) Per Dataset dan Skenario

Tabel D.6 Uji Wilcoxon berpasangan waktu pembaruan — H3 (warm vs always) dan H4 (warm vs cold)

Dataset / skenario	H3: p (warm vs always)	r	H4: p (warm vs cold)	r
workforce / stasioner	$6,1 \times 10^{-5}$	+1,00	$6,1 \times 10^{-5}$	+1,00
workforce / rendah	0,389	+0,20	$6,1 \times 10^{-5}$	+1,00
workforce / sedang	0,053	+0,60	$6,1 \times 10^{-5}$	+1,00
edocument / stasioner	0,489	−0,07	$6,1 \times 10^{-5}$	+1,00
edocument / rendah	0,389	+0,33	$6,1 \times 10^{-5}$	+1,00
edocument / sedang	0,762	−0,07	$6,1 \times 10^{-5}$	+1,00
healthcare / stasioner	—	—	$6,5 \times 10^{-4}$	+1,00
healthcare / rendah	—	—	$6,1 \times 10^{-5}$	+1,00
healthcare / sedang	—	—	$6,1 \times 10^{-5}$	+1,00
project-management / stasioner	0,715	+0,00	$6,1 \times 10^{-5}$	+1,00
project-management / rendah	0,068	+1,00	$6,1 \times 10^{-5}$	+1,00
project-management / sedang	0,144	+0,50	$6,1 \times 10^{-5}$	+1,00
university / stasioner	0,225	+0,60	$6,1 \times 10^{-5}$	+1,00
university / rendah	0,285	−0,33	$6,1 \times 10^{-5}$	+1,00

Tabel D.6 Uji Wilcoxon berpasangan waktu pembaruan — H3 (warm vs always) dan H4 (warm vs cold) (lanjutan)

Dataset / skenario	H3: p (warm vs always)	r	H4: p (warm vs cold)	r
university / sedang	0,655	+0,00	$6,1 \times 10^{-5}$	+1,00

Keterangan: $p = \text{“—”}$ berarti kedua kondisi identik (mis. warm = always ketika pemicu drift menyala di setiap jendela) atau jumlah pasangan terlalu kecil untuk uji. H3 didukung pada baris ber- p signifikan dengan r positif; H4 konsisten didukung (warm lebih cepat dari cold) di seluruh dataset.

D.3 Uji Validasi Eksternal *Smart Building IoT* (H1)

Tabel D.7 Post-hoc CoDA-EDA vs baseline pada Smart Building IoT (Friedman $\chi^2 = 21,4$; $p = 0,002$; $n = 5$ seed)

Pembandingan	$\Delta F1$	p (Wilcoxon)	r (rank-biserial)
compact-GA	-0,023	0,438	-0,60
compact-PSO	-0,026	0,625	-0,60
iBOA	-0,005	1,000	-0,20
BOA	-0,057	0,062	-1,00
ACO	-0,056	0,062	-1,00
UBK	+0,104	0,062	+1,00

Catatan: Dengan hanya lima seed, nilai p terkecil yang mungkin pada uji Wilcoxon dua sisi adalah $2/2^5 = 0,0625$, sehingga taraf 0,05 tidak dapat dicapai berapa pun besar perbedaannya. Hasil eksternal karena itu ditafsirkan secara deskriptif (arah r dan posisi relatif), bukan sebagai vonis signifikansi; uji Friedman omnibus tetap signifikan ($p = 0,002$).

Lampiran E Analisis Karakteristik Dataset (Pra-Eksperimen)

Lampiran ini memuat profil karakteristik seluruh dataset yang dijalankan sebelum eksperimen utama, sebagai dasar penentuan kecocokan tiap dataset dengan tiap hipotesis. Kolom skala menyatakan jumlah *user/resource/aksi*; *permit* unik menandai risiko duplikasi (makin sedikit makin berisiko); rantai derau JSD stasioner menentukan kelayakan mekanisme H3 (layak bila di bawah ambang $\tau = 0,01$).

Tabel E.1 Profil karakteristik dataset

Dataset	Skala (U/R/A)	D	Permit unik	Duplikasi	Rasio permit	Lantai derau JSD	H3
workforce	353/250/9	436	14.513	rendah	0,30	0,0066	layak
e-document	500/300/4	1907	32.961	rendah	0,30	0,0325	tak layak
healthcare	21/16/3	46	14	TINGGI	0,30	0,0022	layak
project-management	19/40/4	51	17	TINGGI	0,30	0,0010	layak
university	22/34/9	54	90	TINGGI	0,30	0,0026	layak
Smart Building IoT	—/—/4	428	5.729	rendah	0,75	0,0065	layak

Tabel E.2 Matriks kecocokan dataset $\times$ hipotesis

Dataset	H1 (kualitas)	H2 (memori)	H3 (pembaruan)	H4 (warm-start)
workforce	Baik	Baik	Layak ($JSD < \tau$)	Baik
e-document	Baik	Baik	Tak layak ($JSD > \tau$)	Baik

Tabel E.2 Matriks kecocokan dataset $\times$ hipotesis (lanjutan)

Dataset	H1 (kualitas)	H2 (memori)	H3 (pembaruan)	H4 (warm-start)
healthcare	Terbatas (duplikasi)	Baik	Layak ($JSD < \tau$)	Baik
project-management	Terbatas (duplikasi)	Baik	Layak ($JSD < \tau$)	Baik
university	Terbatas (duplikasi)	Baik	Layak ($JSD < \tau$)	Baik
Smart Building IoT	Kontras garis-sepele	Baik	Layak ($JSD < \tau$)	Baik

Ringkasan implikasi: hanya *workforce* dan *e-document* yang layak menjadi tumpuan vonis H_1 (permit unik memadai); ketiga dataset kecil terbatas untuk H_1 karena duplikasi tinggi namun tetap sah untuk $H_2/H_3/H_4$; *e-document* tidak layak untuk H_3 karena rantai derau melampaui τ , sedangkan *Smart Building IoT* justru layak — menjadi sarana menunjukkan mekanisme H_3 yang tidak dapat diperlihatkan pada *e-document*.

Lampiran F Konfigurasi dan Kalibrasi Algoritma

F.1 Parameter Tiap Algoritma

Tabel F.1 Parameter kunci CoDA-EDA dan enam baseline

Algoritma	Parameter kunci (nilai)	Catatan
CoDA-EDA	$NP = 100$; $p_0 = 3/D$; $mi_rows = 1500$; $w_{fp} = 1, 0$; $w_{wsc} = 0, 1$	Pohon Chow-Liu order-1 disusun via algoritma Prim; struktur hanya dipicu drift
compact-GA	$NP = 100$; $p_0 = 3/D$; laju $\pm 1/NP$	Univariat; representasi biner identik dengan CoDA-EDA
compact-PSO	$NP = 100$; $w = 0, 7$; $c_1 = c_2 = 1, 5$; $p_0 = 3/D$	Posisi-kecepatan diadaptasi ke ruang kebijakan diskret
BOA	$NP = 100$; $cand = 8$; $top_vars = 400$	Jaringan Bayes via BIC; kandidat induk dibatasi 400 variabel paling informatif agar layak pada D besar
iBOA	$NP = 100$; $k_{order} = 2$	Pohon dependensi inkremental (Chow-Liu); pembanding langsung H3
ACO	$NP = 100$ (semut); $\rho = 0, 1$ (evaporasi feromon)	Parameter mengikuti nilai baku publikasi asli
UBK	$NP = 100$; α adaptif $= 0, 8 \cdot \max(0, 05; 1 - t/200)$; $derau = 0, 3$	Strategi berburu berbasis populasi

Prior kejarangan $p_0 = 3/D$ bersifat sadar-dimensi (aturan ABAC nyata umumnya 2–5 pasangan atribut-nilai), sehingga jumlah bit aktif di awal pencarian tetap konstan meskipun D berbeda antar-dataset.

F.2 Anggaran Evaluasi

Kesetaraan perbandingan dijaga dengan menyetarakan anggaran evaluasi fungsi fitness (bukan jumlah iterasi) antar-algoritma, dihitung untuk keseluruhan proses *separate-and-conquer*. Pada eksperimen kualitas (H_1) semua-dataset, anggaran

ditetapkan seragam: maksimum 20.000 evaluasi total, 4.000 evaluasi per aturan, dan paling banyak 12 aturan; pencarian berhenti lebih awal ketika seluruh permit telah tercakup atau tidak ada aturan yang menambah cakupan. Akibatnya, dataset berdimensi besar (*workforce*, *e-document*) memakai anggaran mendekati penuh, sedangkan dataset kecil berhenti lebih awal karena permit uniknya sedikit.

Tabel F.2 Anggaran evaluasi terpakai per dataset (eksperimen H1)

Dataset	D	Evals terpakai (min–maks)	Rata-rata	Jumlah aturan
workforce	436	16.400–20.100	20.003	4–5
e-document	1907	16.400–20.100	20.003	4–5
healthcare	46	4.001–12.300	4.656	1–3
project- management	51	4.001–20.001	9.328	1–5
university	54	4.001–20.100	11.413	1–5

F.3 Parameter Siklus Pembaruan (H3 dan H4)

Tabel F.3 Parameter simulasi siklus pembaruan lintas jendela

Parameter	Nilai	Keterangan
Ambang pemicu drift (τ)	0, 01	Struktur Chow-Liu dipelajari ulang bila divergensi Jensen-Shannon $> \tau$
Jumlah jendela	5	Log dibagi menjadi lima jendela berurutan (empat siklus pembaruan)
Anggaran pembelajaran awal	9.000 evaluasi	Jendela pertama / kondisi cold
Anggaran per pembaruan	3.000 evaluasi	Tiap siklus pembaruan (warm/always)
Anggaran per aturan	3.000 evaluasi	Batas per aturan pada siklus pembaruan
Ukuran populasi virtual (NP)	100	Sama seperti eksperimen H1

F.4 Ringkasan Uji Monotonisitas (Kalibrasi)

Uji monotonisitas dipakai untuk memverifikasi bahwa *baseline* diimplementasikan secara sah (kualitas tidak menurun ketika anggaran dinaikkan) sekaligus mengidentifikasi tuas hiperparameter yang benar-benar menggerakkan cakupan. Dua tuas diuji seragam pada seluruh algoritma.

Tabel F.4 Efek tuas hiperparameter terhadap kualitas (dataset e-document)

Tuas	Nilai diuji	Efek terhadap CoDA-EDA	Kesimpulan
Bobot false positive (w_{fp})	1, 0/2, 0/4, 0	Cakupan $0,186 \rightarrow 0,187$ (praktis datar)	Bukan tuas cakupan; ditetapkan $w_{fp} = 1, 0$
Anggaran jumlah aturan	4/8/12	F-score $0,362 \rightarrow 0,571 \rightarrow 0,568$ (monoton naik)	Tuas cakupan sesungguhnya; jumlah aturan diperlakukan sebagai parameter rekayasa

Karena kenaikan anggaran aturan menaikkan F-score secara monoton (tanpa penurunan), implementasi kerangka penambangan dinyatakan sah. Nilai baku eksperimen ditetapkan $w_{fp} = 1, 0$; jumlah aturan efektif pada anggaran baku berkisar 4–5.

Lampiran G Rincian Konstruksi Dataset

G.1 Parameter Konstruksi

Tabel G.1 Parameter injeksi korelasi, drift, dan pembangkitan log

Parameter	Nilai
Tingkat korelasi – target NMI	rendah = 0,05; sedang = 0,35; tinggi = 0,65
Skenario drift – target JSD (bersih)	stasioner = 0,00; rendah = 0,10; sedang = 0,25
Entri per jendela	2.000
Jumlah jendela	5 (empat transisi/siklus pembaruan)
Rasio permit	0,30 (proporsi keputusan izin pada log)
Seed dataset	0, 1, 2
Total konfigurasi	$5 \text{ dataset} \times 3 \text{ korelasi} \times 3 \text{ drift} \times 3 \text{ seed} = 135$

Kalibrasi korelasi bersifat *closed-loop*: parameter penggandeng distribusi bersama dicari melalui pencarian biner hingga NMI yang terukur pada log final (bukan pada distribusi teoretis) mendekati target, dengan ukuran log kalibrasi disamakan dengan ukuran log final untuk menekan bias sampel-hingga.

G.2 Struktur Manifest

Tabel G.2 Medan (field) manifest per konfigurasi dataset

Medan	Keterangan
tag	Pengenal konfigurasi, mis. <code>workforce_corr-sedang_drift-rendah_seed-0</code>
dataset, corr, drift, seed	Identitas dataset dasar, tingkat korelasi, skenario drift, dan seed

Tabel G.2 Medan (field) manifest per konfigurasi dataset (lanjutan)

Medan	Keterangan
<code>uattr, rattr</code>	Pasangan atribut subjek dan objek yang dipilih untuk injeksi korelasi
<code>alpha</code>	Parameter penggandeng distribusi bersama hasil kalibrasi biner
<code>target_nmi, observed_nmi</code>	Target <i>NMI</i> dan <i>NMI</i> yang benar-benar terukur pada log final
<code>target_jsd, observed_jsd</code>	Target <i>JSD</i> dan <i>JSD</i> terukur per jendela (untuk perhitungan drift bersih)
<code>permit_ratio duplication</code>	Rasio permit teramati pada log
<code>_rate</code>	Tingkat duplikasi entri (indikator keragaman efektif data)

G.3 Laporan Validasi — Kalibrasi Korelasi

Tabel G.3 NMI teramati per dataset dan tingkat korelasi (rerata lintas drift dan seed; target dalam kurung)

Dataset	<i>NMI</i> @ rendah (0,05)	<i>NMI</i> @ sedang (0,35)	<i>NMI</i> @ tinggi (0,65)
workforce	0,050 (0,05)	0,351 (0,35)	0,638 (0,65)
e-document	0,057 (0,05)	0,348 (0,35)	0,653 (0,65)
healthcare	0,048 (0,05)	0,354 (0,35)	0,477 (0,65)
project-management	0,057 (0,05)	0,349 (0,35)	0,652 (0,65)
university	0,044 (0,05)	0,335 (0,35)	0,653 (0,65)

NMI teramati mendekati target di seluruh dataset, menegaskan mekanisme injeksi korelasi bekerja sebagaimana dirancang; simpangan kalibrasi terbesar pada satu konfigurasi sebesar 0,34 (akibat bias sampel-hingga pada rasio sel/sampel yang tinggi), sedangkan rerata per dataset tetap dekat target.

G.4 Laporan Validasi — Lantai Derau, Drift Bersih, dan Duplikasi

Tabel G.4 Lantai derau JSD, drift bersih terkalibrasi, tingkat duplikasi, dan status validasi per dataset

Dataset	Lantai derau JSD	Drift bersih ($\rightarrow 0, 10$)	Drift bersih ($\rightarrow 0, 25$)	Duplikasi	Status (OK/total)
workforce	0,017	0,094	0,236	16–57%	11/27 OK
e-document	0,132	0,091	0,228	5–37%	24/27 OK
healthcare	0,003	0,102	0,172	95–99%	0/27 OK
project-management	0,001	0,099	0,175	92–94%	0/27 OK
university	0,009	0,089	0,181	68–88%	0/27 OK

Dua pola menegaskan temuan pra-eksperimen. Pertama, lantai derau JSD *e-document* sangat tinggi (0,132) — jauh melampaui ambang $\tau = 0,01$ — sehingga pelaporan drift bersih wajib dan mekanisme H_3 tidak dapat ditunjukkan pada dataset ini; nilai absolut lantai derau ini bergantung pada ukuran jendela pengukuran. Kedua, ketiga dataset kecil berduplikasi 68–99% dan seluruh konfigurasinya ditandai peringatan (0/27 OK), mengonfirmasi status mereka sebagai konteks pelengkap dan bukan tumpuan vonis H_1 .

Lampiran H Kutipan Kode Kunci

H.1 Fungsi Fitness Selaras-F1

Skor satu aturan berbentuk

$$\frac{TP^2}{TP + w_{fp} \cdot FP} - w_{wsc} \cdot WSC,$$

yang setara dengan $TP \times$ presisi sehingga menghargai cakupan dan presisi sekaligus (selaras F1); bentuk penalti linier ditolak karena optimumnya aturan bercakupan nol (modul codaeda/problem.py).

```
def fitness(self, x, uncovered):

    self.evals += 1

    # aturan tanpa aksi tidak pernah cocok
    if x[self.act_idx].sum() == 0:

        return -1e9

    m = self.matches(x)

    tp = int((m & self.label & uncovered).sum())

    if tp == 0:

        # aturan tanpa permit baru tidak berguna
        # (separate-and-conquer); tanpa
        # dominasi ini aturan KOSONG mengalahkan
        # aturan bermanfaat ber-FP kecil.

        return -1e9 - self.w_wsc * self.wsc(x)

    fp = int((m & ~self.label).sum())

    # Skor = TP x presisi = TP^2 / (TP + w_fp*FP):
```

```

# selaras F1 (cakupan DAN presisi).*

# Penalti linier (TP - w_fp*FP) keliru:
# optimumnya aturan super-presisi
# bercakupan nol, sehingga pencarian lebih kuat
# justru MENURUNKAN F1 kebijakan.

return (tp * tp) / (tp + self.w_fp * fp)
- self.w_wsc * self.wsc(x)

```

H.2 Pohon Chow-Liu order-1 via Algoritma Prim (Memori $O(D)$)

Struktur dependensi disusun sebagai pohon rentang maksimum menggunakan algoritma Prim, yang hanya menyimpan larik sepanjang D (bukan matriks mutual information $D \times D$); mutual information dihitung baris-demi-baris atas representasi terkemas-bit (modul `codaeda/algos.py`).

```

def chow_liu_prim(A, rng, max_rows=1536, eps=1e-6):
    n, D = A.shape
    idx = rng.choice(n, max_rows, replace=False)
    if n > max_rows else np.arange(n)

    # Bit-packing: kolom biner dikemas 8 baris/byte;
    # ko-okurensi via AND bitwise + tabel popcount,
    # tanpa membentuk matriks MI D x D maupun salinan
    # float data.

    # (disusun per blok baris)
    P = np.packbits(A[idx], axis=0)
    cnt1 = _POPCNT[P].sum(axis=0).astype(float);
    p1 = cnt1 / n

    # mutual information baris-ke-i terhadap
    # semua variabel
    def mi_row(i):
        j11 = _POPCNT[P & P[:, i:i+1]].sum(axis=0) / n
        # ... hitung j10, j01, j00
        # lalu MI = SUM p*log2(p / (p_i * p_j)) ...

```

```

        return mi

# Algoritma Prim: HANYA larik sepanjang D
# (bukan matriks D x D) -> memori O(D)
parent = np.full(D, -1); best = np.full(D, -np.inf);
intree = np.zeros(D, bool)
cur = 0; intree[cur] = True

for _ in range(D - 1):
    # satu baris MI dihitung saat perlu
    m = mi_row(cur)
    upd = (~intree) & (m > best);
    best[upd] = m[upd]; bestfrom[upd] = cur
    nxt = int(np.argmax(np.where(intree,
    -np.inf, best)))
    parent[nxt] = bestfrom[nxt]; intree[nxt] = True;
    cur = nxt

return parent, cp0, cp1, wgt
# induk + tabel probabilitas bersyarat per variabel

```

H.3 Mekanisme Deteksi Drift

Pergeseran distribusi antar-jendela diukur sebagai Jensen-Shannon divergence atas marginal pasangan atribut-nilai; struktur dependensi hanya dipelajari ulang ketika pergeseran melampaui ambang τ , sedangkan parameter (*Probability Vector*) tetap diperbarui tiap siklus.

```

# distribusi frekuensi pasangan atribut-nilai
# pada satu jendela
def av_marginal(A):
    m = A.mean(axis=0).astype(np.float64); s = m.sum()
    return m / s if s > 0 else np.full(len(m),
    1.0 / max(1, len(m)))

# Jensen-Shannon divergence
def jsd(p, q, eps=1e-12):
    m = 0.5 * (p + q)

```

```

    kl = lambda a: float(np.sum(np.where(a > 0,
    a * np.log2((a+eps)/(m+eps)), 0.0)))
    return max(0.0, min(1.0,
    0.5 * kl(p) + 0.5 * kl(q)))

# Pemicu (di run_sequential): struktur dipelajari
# ulang HANYA bila drift > tau

# distribusi jendela berjalan
cur      = av_marginal(prob.A)

# pergeseran thd jendela referensi
delta    = 0.0 if ref is None else jsd(ref, cur)

# tau = 0,01 (baku)
relearn  = (mode == "always") or (delta > tau)

if relearn:
    # perbarui struktur; jika tidak: PV saja
    T = chow_liu_prim(prob.A, rng)

```